

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

Phạm Hoàng Hùng-23520574

Trần Quốc Hưng-23520579

BÁO CÁO THỰC TẬP DOANH NGHIỆP
Thiết kế và hiện thực mô hình Bộ tăng tốc
Vision Transformer trên FPGA

TP. HỒ CHÍ MINH, <THÁNG/NĂM>

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

BÁO CÁO THỰC TẬP DOANH NGHIỆP
Thiết kế và hiện thực mô hình Bộ tăng tốc
Vision Transformer trên FPGA

<Công ty thực tập>

<Người hướng dẫn tại công ty>

<Giảng viên hướng dẫn>

<Tên sinh viên>

<MSSV> - <LỚP>

TP. HỒ CHÍ MINH, <THÁNG/NĂM>

PHIẾU XÁC NHẬN THỰC TẬP

Công ty:.....xác nhận:

Anh/chị: Sinh ngày: ...

Sinh viên năm thứ.....Khoa:Trường:

Đã thực tập tại:

Thời gian thực tập:

Vị trí thực tập:

Nhiệm vụ được giao và hướng dẫn:

Cán bộ phụ trách và hướng dẫn:

Chức vụ:

Đánh giá quá trình thực tập của sinh viên:

Các kết quả sinh viên đã thực hiện được:

Các tiêu chí đánh giá	Điểm đánh giá (Theo thang điểm 10)	Ghi chú
1. Năng lực chuyên môn		
2. Chất lượng công việc		
3. Tinh thần trách nhiệm		
4. Tính chủ động sáng tạo		
5. Tính kỷ luật		
6.		
Tổng điểm:		

Đánh giá khác:

TRƯỞNG ĐƠN VỊ

NHÂN SỰ

..., ngày...tháng ...năm ...
CÁN BỘ QUẢN LÝ TRỰC TIẾP

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Họ và tên sinh viên:

MSSV:

Công ty thực tập:

Thời gian thực tập:

Vị trí thực tập:

Nhiệm vụ được giao:

Đánh giá quá trình thực tập của sinh viên:

Các kết quả sinh viên đã thực hiện được:

Điểm:

Bằng chữ:

....., ngày.....thángnăm

Giáo viên hướng dẫn

MỤC LỤC

CHƯƠNG 1 — GIỚI THIỆU VÀ PHẠM VI	2
1.1 Bối cảnh Vision Transformer và tăng tốc phần cứng	2
1.2 Bài toán end-to-end ViT inference trên FPGA	3
1.3 Mục tiêu của báo cáo	4
1.4 Phạm vi thiết kế và phạm vi đánh giá	5
1.4.1 Mô hình, cấu hình thực thi và nền tảng phần cứng	5
1.4.2 Phạm vi dữ liệu, quy tắc đo và snapshot	6
1.4.3 Các nội dung ngoài phạm vi	7
1.5 Phương pháp tiếp cận và nguồn bằng chứng	7
1.6 Cấu trúc báo cáo	9
CHƯƠNG 2 — NỀN TẢNG LÝ THUYẾT	10
2.1 Vision Transformer	10
2.1.1 Ảnh đầu vào, patchify và patch embedding	10
2.1.2 CLS token và positional embedding	11
2.1.3 Transformer encoder	12
2.1.4 Multi-Head Self-Attention	15
a) Đầu vào của khối Multi-Head Self-Attention	15
b) Chia thành nhiều attention head	16
c) Tính điểm attention	16
d) Tính đầu ra của từng head	17
e) Ghép các head và chiếu đầu ra	17
f) Công thức tổng quát của Multi-Head Self-Attention	17
g) Vai trò của Multi-Head Self-Attention trong ViT	18

h) Liên hệ với triển khai phần cứng	18
2.1.5 MLP, residual connection và classifier	18
a) Khối MLP trong Transformer Encoder	19
b) Residual Connection	21
c) Đầu ra sau 12 Encoder Layer	23
d) Classifier	23
e) Liên hệ với triển khai phần cứng	24
2.2 Các phép toán nền tảng	24
2.2.1 GEMM và các dạng nhân ma trận của ViT	24
a) Phép toán GEMM tổng quát	25
b) Patch Projection	26
c) Q, K và V Projection	26
d) Nhân ma trận trong Self-Attention	27
e) GEMM trong khối MLP	27
f) GEMM trong lớp Classifier	28
g) Tổng hợp các dạng GEMM trong ViT-Base/Patch16-224	28
h) Đặc điểm tính toán cần hỗ trợ khi ánh xạ xuống phần cứng	29
2.2.2 LayerNorm	30
2.2.3 Softmax ổn định số	31
2.2.4 GELU	35
2.2.5 Vector operation và layout transformation	37
a) Vector Operation	37
b) Layout Transformation	38
2.2.6 Argmax	39

2.3 Biểu diễn số và tính toán mixed-precision	43
2.3.1 FP16, FP32 và chuyển đổi kiểu dữ liệu	43
2.3.2 Làm tròn, tích lũy và các điểm cần kiểm soát sai khác số học	45
2.4 FPGA SoC, DDR và AXI	47
2.4.1 Phân hoạch PS–PL	47
a) Processing System	47
b) Programmable Logic	48
c) Nguyên tắc phân hoạch PS–PL	49
d) Control path và data path	50
e) Vai trò của DDR trong phân hoạch PS–PL	51
f) Phân hoạch PS–PL trong đề tài	52
2.4.2 AXI4-Lite và AXI memory-mapped	52
a) AXI4-Lite	52
b) AXI4 Memory-Mapped	55
c) Các kênh cơ bản của AXI4	55
d) Memory-Mapped Addressing	57
e) Phân biệt AXI4-Lite và AXI4 Memory-Mapped	59
f) Liên hệ với thiết kế ViT Accelerator	60
2.4.3 VALID/READY, burst, FIFO và backpressure	60
a) Cơ chế VALID/READY	60
b) Burst Transfer	62
c) FIFO	62
d) Backpressure	63
e) Quan hệ giữa FIFO và Backpressure	64

f) Quan hệ giữa Burst và FIFO	65
g) Ảnh hưởng đến hiệu năng của accelerator	67
h) Vai trò trong hệ thống ViT Accelerator	68
i) Tổng kết	69
2.6 Nhóm công trình liên quan	69
2.6.1 ViA: A Novel Vision-Transformer Accelerator Based on FPGA	69
2.6.2 ViTA: A Vision Transformer Inference Accelerator for Edge Applications	70
2.6.3 A High Speed Reconfigurable Architecture for Softmax and GELU in Vision Transformer	71
2.6.4 HG-PIPE: Vision Transformer Acceleration with Hybrid-Grained Pipeline	72
2.6.5 MDS-ViT: A Multi-Data Stream FPGA-Based Vision Transformer Accelerator	73
2.7 Ma trận so sánh related work	74
2.8 Định vị phạm vi so với related work	76
2.8.1 Ràng buộc DSP=0 và mixed floating-point	76
2.8.2 Tích hợp GEMM và non-GEMM trong cùng NPU	77
2.8.3 Memory architecture là khác biệt lớn nhất về throughput	77
2.8.4 M8 ưu tiên RTL-to-board validation và bottleneck characterization	78
2.8.5 Giới hạn khi so sánh với các công trình đã công bố	78
2.8.6 Định vị tổng thể của thiết kế M8	79
CHƯƠNG 3 — PHÂN TÍCH WORKLOAD VÀ ÁNH XẠ	80
3.1 Đồ thị thực thi end-to-end của ViT	80

3.2 Bảng tensor shape xuyên suốt mô hình	80
3.3 Phân loại GEMM và non-GEMM	82
3.3.1 Nhóm GEMM	82
3.3.2 Nhóm non-GEMM	83
3.4 Template lệnh, descriptor và lịch lệnh M8	84
3.5 Không gian dữ liệu và vòng đời tensor	86
3.5.1 MODEL	86
3.5.2 INPUT	86
3.5.3 TABLE/command parameter	86
3.5.4 SCRATCH/intermediate tensor	86
3.5.5 LOGITS, PROBS và đầu ra phân loại	87
3.6 Phân hoạch host, PS và PL	87
3.7 Ánh xạ kiểu dữ liệu theo từng đường dữ liệu	88
3.8 Ví dụ minh họa luồng xử lý từ ảnh thô	89
3.8.1 Mục tiêu, phạm vi và input mẫu	89
3.8.2 Định dạng ảnh gốc, resize, crop và chuẩn hóa	89
3.8.3 Tạo tensor đầu vào và ghi vào vùng INPUT	89
3.8.4 Patchify, patch projection, CLS và positional embedding	90
3.8.5 Một encoder block minh họa	90
3.8.6 Final LayerNorm, classifier và Argmax	90
3.8.7 Thu nhận logits/probability/nhãn dự đoán	90
3.8.8 Bảng trace ảnh thô → tensor → command → engine → đầu ra	90
CHƯƠNG 4 — KIẾN TRÚC PHẦN CỨNG	92
4.1 Sơ đồ top-level NPU	92

4.2 Control plane	94
4.2.1 Thanh ghi điều khiển và trạng thái	94
4.2.2 Sequencer	95
4.2.3 Command controller	96
4.2.4 Command descriptor và cơ chế dispatch	97
4.2.5 Handshake, lỗi và quy tắc hoàn tất command	97
4.3 Khuôn giải thích bắt buộc cho mỗi thuật toán/phần tử xử lý	98
4.4 GEMM engine và các thuật toán GEMM	99
4.4.1 Vai trò và taxonomy GEMM trong ViT	99
4.4.2 Hình học logical tile và ánh xạ sang lượt tính vật lý	101
4.4.3 Scheduler, operand routing và vòng lặp GEMM	101
4.4.4 Nạp activation, weight và bias	102
4.4.5 Packed-B panel và ping-pong buffer	103
4.4.6 FP16 dot-stream array	103
4.4.7 Thuật toán một dot stream: khởi tạo, multiply, accumulate, kết thúc	103
4.4.8 Accumulation, conversion, rounding và result FIFO	104
4.4.9 Đường ghi kết quả và đồng bộ với controller	104
4.4.10 Patch projection GEMM	105
4.4.11 Q projection GEMM	105
4.4.12 K projection GEMM	105
4.4.13 V projection GEMM	105
4.4.14 QK^T GEMM	105
4.4.15 PV GEMM	106
4.4.16 Output projection GEMM	106

4.4.17 FC1 GEMM	106
4.4.18 FC2 GEMM	106
4.4.19 Classifier GEMM	106
4.4.20 Ví dụ minh họa một GEMM tile/panel	106
4.5 Non-GEMM engines và các thuật toán non-GEMM	107
4.5.1 Tổ chức chung, shared arithmetic và buffer cục bộ	107
4.5.2 Vector engine	109
4.5.2.1 Positional add	110
4.5.2.2 Residual add	110
4.5.2.3 Attention scale (mask disabled) và phép toán từng phần tử	110
4.5.2.4 Luồng đọc-tính-ghi và điều kiện kết thúc	110
4.5.3 Layout engine	110
4.5.3.1 Copy CLS và copy patch layout	111
4.5.3.2 Split Q/K/V theo attention head	111
4.5.3.3 K transpose	111
4.5.3.4 Head merge	111
4.5.3.5 Địa chỉ, stride và iterator layout	111
4.5.4 LayerNorm engine	111
4.5.4.1 Mean	111
4.5.4.2 Variance	112
4.5.4.3 Reciprocal square root/normalization	112
4.5.4.4 Gamma, beta và output writeback	112
4.5.4.5 Ví dụ một token LayerNorm	113
4.5.5 Softmax engine	113

4.5.5.1 Tìm max theo hàng	113
4.5.5.2 Trừ max; attention scale/mask ở lệnh Vector trước Softmax	113
4.5.5.3 Exponential/xấp xỉ theo phương pháp M8	113
4.5.5.4 Sum, reciprocal và normalization	113
4.5.5.5 Ví dụ một hàng attention score	114
4.5.6 GELU engine	114
4.5.6.1 Công thức và phương pháp hiện thực	114
4.5.6.2 Trình tự tính toán một phân tử	114
4.5.6.3 Chuỗi FC1 → GELU → FC2	114
4.5.7 Argmax engine	114
4.5.7.1 Khởi tạo max và chỉ số lớp	115
4.5.7.2 So sánh/cập nhật tuần tự	115
4.5.7.3 Xuất nhãn và trạng thái hoàn tất	115
4.6 Đồng bộ giữa controller, GEMM, non-GEMM và memory frontend	115
4.7 Danh sách hình/bảng kỹ thuật của chương	116
CHƯƠNG 5 — BỘ NHỚ, AXI, TÍCH HỢP SOC VÀ RUNTIME	116
5.1 Kiến trúc PS–PL và Vivado Block Design	117
5.2 Clock, reset và các miền đồng bộ	117
5.3 Bản đồ địa chỉ và vai trò từng vùng DDR	118
5.4 Memory frontend	119
5.4.1 Arbitration và ownership	119
5.4.2 Address context	119
5.4.3 Activation cache và bias cache	119
5.4.4 Read router, write router và result path	119

5.5 AXI memory adapter	120
5.5.1 Read request FIFO	120
5.5.2 Read response FIFO	120
5.5.3 Scalar read, line-fill và read-ahead	120
5.5.4 AXI beat, burst và addressing	120
5.5.5 Đường ghi, write response và backpressure	121
5.6 Ví dụ minh họa một luồng xử lý liên quan đến bộ nhớ	121
5.6.1 Mục tiêu, command mẫu và tensor mẫu	121
5.6.2 Sinh địa chỉ input/activation/weight/bias	121
5.6.3 Read request → AXI transaction → read response	121
5.6.4 Nạp dữ liệu vào cache/panel/buffer ping-pong	121
5.6.5 Cấp toán hạng cho GEMM hoặc non-GEMM	121
5.6.6 Result FIFO → scratch writeback	121
5.6.7 Non-GEMM đọc tensor trung gian kế tiếp	122
5.6.8 Timeline request → response → compute → writeback	122
5.7 Luồng runtime cho một ảnh	122
5.7.1 Chuẩn bị model, command table và input	122
5.7.2 Khởi tạo và kích hoạt job	122
5.7.3 Theo dõi trạng thái thực thi	122
5.7.4 Thu thập đầu ra và log bằng chứng	123
CHƯƠNG 6 — PHƯƠNG PHÁP KIỂM CHỨNG VÀ KẾT QUẢ CUỐI CÙNG	123
6.1 Quy tắc trình bày snapshot cuối cùng	123
6.2 Chiến lược kiểm chứng	123
6.2.1 Mức phép toán	123

6.2.2 Mức engine	124
6.2.3 Mức tích hợp	124
6.2.4 Mức implementation	124
6.2.5 Mức board và tập dữ liệu	124
6.3 Phương pháp đánh giá tập ảnh	124
6.3.1 Nguồn dữ liệu, số lượng ảnh và tiêu chí chọn mẫu	124
6.3.2 Tiền xử lý và nhãn tham chiếu	124
6.3.3 Quy trình chạy một ảnh và thu log	124
6.3.4 Quy tắc ghi nhận pass/fail, lỗi và dữ liệu thiếu	125
6.3.5 Khả năng tái lập kết quả	125
6.4 Kết quả implementation và timing cuối cùng	125
6.5 Bảng kết quả cuối cùng trên tập ảnh	127
6.5.1 Tham chiếu với software CPU	128
6.6 So sánh vector số và kết quả phân loại	128
6.7 Hình, bảng, log và receipt minh chứng	129
CHƯƠNG 7 — PHÂN TÍCH KỸ THUẬT VÀ GIỚI HẠN	130
7.1 Phân bố thời gian thực thi và runtime counters	130
7.2 Mức sử dụng GEMM và đường cấp toán hạng	133
7.3 Memory/AXI, data movement và backpressure	135
7.4 Timing, critical path và tài nguyên	137
7.5 Sai khác số học và phạm vi diễn giải	140
7.6 Các giới hạn của phạm vi	143
7.7 Hướng tối ưu/khảo sát tiếp theo	145
7.7.1 Tối ưu memory frontend	145

7.7.2 Tăng burst efficiency	146
7.7.3 Tăng request concurrency	146
7.7.4 Prefetch	146
7.7.5 Ping-pong buffering	147
7.7.6 Giảm scratch DDR round-trip	147
7.7.7 Pipeline shared FP32 non-GEMM datapath	148
7.7.8 Khảo sát selective DSP	148
7.7.9 Standalone Cortex-A53 runtime	148
7.7.10 Numerical checkpoint analysis	149
CHƯƠNG 8 — KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	149
8.1 Tóm tắt phạm vi báo cáo	149
8.2 Các giới hạn cần lưu ý khi đọc kết quả cuối cùng	150
8.3 Hướng phát triển	151
TÀI LIỆU THAM KHẢO	152
PHỤ LỤC	153

CHƯƠNG 1 — GIỚI THIỆU VÀ PHẠM VI

1.1 Bối cảnh Vision Transformer và tăng tốc phần cứng

Trong những năm gần đây, kiến trúc **Transformer** đã được mở rộng từ xử lý ngôn ngữ tự nhiên sang các bài toán thị giác máy tính. Một trong những kiến trúc tiêu biểu là **Vision Transformer (ViT)**, trong đó ảnh đầu vào được chia thành các vùng nhỏ gọi là *patch*, chuyển thành chuỗi token và xử lý bằng các khối Transformer Encoder.

Khác với mạng nơ-ron tích chập truyền thống, phần lớn khối lượng tính toán của ViT tập trung vào các phép toán tuyến tính và nhân ma trận trong cơ chế **Multi-Head Self-Attention (MHSA)** và **Multi-Layer Perceptron (MLP)**. Với cấu hình **ViT-Base/Patch16-224**, mô hình sử dụng ảnh đầu vào kích thước (224x224), patch kích thước (16x16), 196 patch ảnh, một CLS token, hidden dimension bằng 768 và 12 Transformer Encoder layer. Khối lượng tính toán cho một lần suy luận nằm ở mức hàng chục GFLOP, trong đó các phép nhân ma trận chiếm tỷ trọng lớn.

Bên cạnh các phép nhân ma trận, quá trình suy luận ViT còn chứa nhiều phép toán không thuộc nhóm GEMM (*non-GEMM*) như **LayerNorm**, **Softmax**, **GELU**, **residual/vector operations**, **layout transformation** và **Argmax**. Các phép toán này có đặc điểm truy cập dữ liệu, độ song song và yêu cầu số học khác với GEMM. Vì vậy, việc chỉ xây dựng một bộ nhân ma trận nhanh chưa đủ để hình thành một hệ thống tăng tốc ViT hoàn chỉnh.

Một thách thức khác đến từ dung lượng tham số và dữ liệu trung gian. Trong quá trình suy luận, phần cứng phải liên tục trao đổi trọng số, activation và kết quả trung gian giữa bộ nhớ ngoài và các khối tính toán. Khi số đơn vị tính toán tăng lên nhưng hệ thống bộ nhớ không cung cấp dữ liệu với tốc độ tương ứng, các khối tính toán có thể phải chờ dữ liệu và hiệu năng thực tế sẽ thấp hơn đáng kể so với thông lượng tính toán lý thuyết.

FPGA là một nền tảng phù hợp để khảo sát bài toán này do có khả năng cấu hình kiến trúc dữ liệu, mức song song, pipeline và hệ thống bộ nhớ theo yêu cầu cụ thể của mô hình. So với việc chỉ thực thi mô hình bằng bộ xử lý đa dụng, FPGA cho phép xây dựng trực tiếp các datapath chuyên dụng cho GEMM và các phép toán non-GEMM. Đồng thời, so với ASIC, FPGA vẫn giữ được tính tái cấu hình và thuận lợi cho quá trình nghiên cứu, thử nghiệm kiến trúc và kiểm chứng trên phần cứng thực.

Đối với nền tảng **Zynq UltraScale+ MPSoC**, FPGA fabric trong Programmable Logic (PL) còn được tích hợp cùng Processing System (PS) chứa bộ xử lý ARM và DDR controller. Điều này cho phép xây dựng một hệ thống tăng tốc hoàn chỉnh gồm CPU điều khiển, NPU trong PL, bộ nhớ DDR và các giao tiếp AXI giữa các thành phần.

Do đó, bài toán tăng tốc ViT trên FPGA không chỉ là bài toán thiết kế một kernel tính toán, mà là bài toán **thiết kế hệ thống ở mức end-to-end**, bao gồm kiến trúc tính toán, tổ chức bộ nhớ, giao tiếp PS-PL, điều khiển thực thi, tích hợp RTL, timing closure và kiểm chứng trên board vật lý.

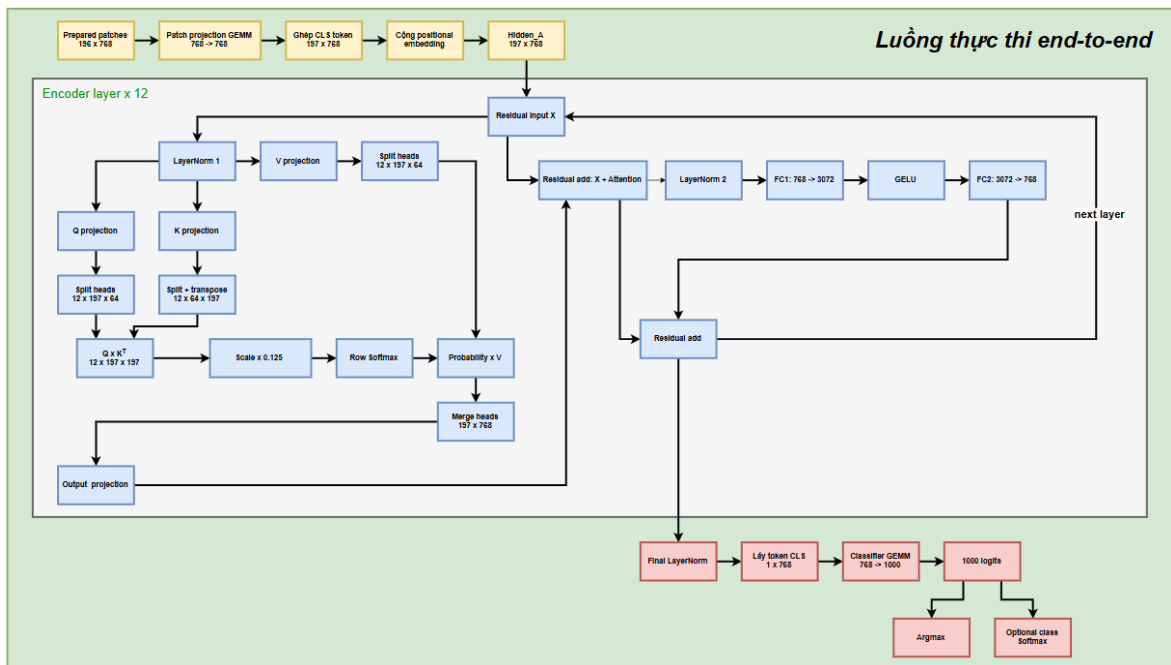
1.2 Bài toán end-to-end ViT inference trên FPGA

Bài toán được xem xét trong báo cáo là xây dựng một hệ thống phần cứng có khả năng thực hiện chuỗi tính toán của **ViT-Base/Patch16-224** từ tensor đầu vào đã được tiền xử lý cho đến kết quả phân lớp cuối cùng.

Khái niệm *end-to-end* trong phạm vi báo cáo không có nghĩa toàn bộ quy trình từ giải mã tệp JPEG đến suy luận đều được thực hiện trong PL. Các bước như đọc ảnh, resize, normalize và chuẩn bị model package hiện vẫn được thực hiện ở phía host. Phần tăng tốc được nghiên cứu bắt đầu từ khi tensor đầu vào và dữ liệu mô hình đã được đặt trong vùng nhớ DDR mà NPU có thể truy cập.

Luồng thực thi tổng quát được xác định như sau:

Ảnh đầu vào → **tiền xử lý trên host** → **tensor đầu vào trong DDR** → **NPU thực hiện chuỗi ViT** → **logits** → **Argmax** → **lớp dự đoán**.



Để thực hiện được luồng này, hệ thống phải giải quyết đồng thời nhiều lớp bài toán.

Trước hết là **bài toán tính toán**. Những phép nhân ma trận xuất hiện trong patch embedding, Q/K/V projection, attention, output projection, hai lớp fully connected của MLP và classifier cần được ánh xạ sang một kiến trúc GEMM phù hợp với tài nguyên FPGA.

Tiếp theo là **bài toán non-GEMM**. Nếu LayerNorm, Softmax, GELU và các phép residual tiếp tục được chuyển về CPU xử lý sau mỗi lớp, dữ liệu phải di chuyển nhiều lần giữa PS, DDR và PL. Điều này làm suy giảm ý nghĩa của một accelerator end-to-end. Vì vậy, phiên bản M8 đưa các phép toán non-GEMM chính vào PL thông qua các engine phần cứng chuyên dụng hoặc dùng chung tài nguyên số học.

Một yêu cầu khác là **bài toán điều phối toàn mô hình**. ViT-Base chứa 12 encoder layer cùng các bước xử lý trước và sau encoder. Hệ thống do đó cần một cơ chế sequencer và descriptor để mô tả thứ tự thực hiện, kích thước tensor, địa chỉ toán hạng và loại engine được sử dụng. Trong cấu hình M8, full inference schedule được biểu diễn bằng **249 command descriptor**, bao phủ các giai đoạn chính của một lần suy luận.

Ở mức hệ thống, NPU phải giao tiếp với Processing System và DDR thông qua AXI. Hai đường truyền được tách theo chức năng:

- **Control path:** ARM A53 điều khiển NPU thông qua giao tiếp AXI4-Lite và register bank.
- **Data path:** NPU sử dụng giao tiếp AXI4 master 128 bit để truy cập dữ liệu trong DDR thông qua hệ thống interconnect của Zynq UltraScale+.

Bài toán còn chịu một ràng buộc thiết kế quan trọng: phiên bản được khảo sát đặt mục tiêu **không sử dụng DSP block trong PL**. Các datapath số học của accelerator do đó phải được tổng hợp từ LUT, FF và các tài nguyên logic còn lại. Ràng buộc này không nhằm tạo ra kiến trúc có tốc độ cao nhất có thể trên ZU-5EV, mà nhằm khảo sát khả năng xây dựng một NPU ViT khi tài nguyên DSP được bảo toàn cho các chức năng khác.

Như vậy, bài toán nghiên cứu có thể được phát biểu ngắn gọn như sau:

Thiết kế, triển khai và kiểm chứng trên FPGA một NPU chuyên dụng có khả năng thực hiện chuỗi suy luận ViT-Base/Patch16-224 với GEMM và các phép non-GEMM chính được xử lý trong PL, kết nối với DDR thông qua AXI, hoạt động dưới ràng buộc DSP=0 và có thể được đánh giá bằng các bảng chứng thu được từ RTL, Vivado và board Genesys ZU-5EV.

1.3 Mục tiêu của báo cáo

Mục tiêu tổng quát của báo cáo là trình bày một cách có hệ thống quá trình xây dựng và kiểm chứng một **NPU chuyên dụng cho Vision Transformer trên FPGA**, từ phân tích workload của mô hình đến thiết kế RTL, tích hợp SoC và thử nghiệm trên phần cứng thực.

Các mục tiêu cụ thể gồm:

Thứ nhất, xây dựng kiến trúc tăng tốc cho các phép toán GEMM của ViT.

Thiết kế cần hỗ trợ các phép nhân ma trận xuất hiện xuyên suốt ViT-Base/Patch16-224, sử dụng mixed precision phù hợp và tuân thủ ràng buộc không sử dụng DSP.

Thứ hai, tích hợp các phép toán non-GEMM chính vào Programmable Logic.

Các phép toán gồm Vector, Layout, LayerNorm, Softmax, GELU và Argmax được tổ chức thành các engine phần cứng nhằm giảm sự phụ thuộc vào CPU trong quá trình thực thi từng layer.

Thứ ba, xây dựng cơ chế điều khiển cho toàn bộ quá trình suy luận.

Hệ thống cần có sequencer, command controller và descriptor format để điều phối các engine theo đúng thứ tự của mô hình ViT.

Thứ tư, xây dựng hệ thống truy cập bộ nhớ và tích hợp PS–PL.

NPU phải có khả năng nhận lệnh điều khiển từ PS, tự truy cập vùng nhớ DDR chứa model, input và intermediate tensor, sau đó ghi kết quả trở lại DDR.

Thứ năm, hoàn thiện luồng từ RTL đến FPGA vật lý.

Thiết kế được đưa qua các bước simulation, synthesis, implementation, timing analysis, bitstream generation và lập trình lên board Digilent Genesys ZU-5EV.

Thứ sáu, xây dựng phương pháp kiểm chứng nhiều cấp.

Tính đúng đắn được đánh giá từ các khối số học cơ bản, các engine riêng lẻ, mức hệ thống cho đến thực thi trên board.

Thứ bảy, đánh giá kiến trúc dựa trên số liệu đo thay vì chỉ dựa trên thông lượng lý thuyết.

Báo cáo xem xét đồng thời mức sử dụng tài nguyên, timing, hành vi truy cập bộ nhớ, trạng thái hoạt động của GEMM và kết quả đầu ra của mô hình nhằm xác định các giới hạn thực tế của hệ thống.

Mục tiêu của đồ án vì vậy không phải chứng minh kiến trúc M8 có tốc độ cao hơn các accelerator ViT hiện đại, mà là xây dựng một **baseline RTL-to-board có thể kiểm chứng**, từ đó xác định rõ những điểm giới hạn cần tiếp tục tối ưu.

1.4 Phạm vi thiết kế và phạm vi đánh giá

Để tránh đồng nhất giữa những chức năng đã được hiện thực, những kết quả đã được đo và những hướng phát triển mới chỉ được đề xuất, phạm vi của báo cáo được giới hạn như sau.

1.4.1 Mô hình, cấu hình thực thi và nền tảng phần cứng

Mô hình được sử dụng là **ViT-Base/Patch16-224** cho bài toán image classification. Cấu hình được cố định với ảnh đầu vào (224x224), patch (16x16), batch size bằng 1 và kích thước tensor được xác định trước theo kiến trúc của mô hình.

Thiết kế không hướng đến khả năng tự động chạy mọi biến thể Transformer. Sequencer và memory layout hiện được xây dựng cho workload ViT-Base/Patch16-224 cụ thể.

Chiến lược biểu diễn số sử dụng **mixed FP16/FP32**. Các operand GEMM được chuyển sang FP16 tại datapath thích hợp trong khi accumulation và các phép toán non-GEMM sử dụng miền FP32 theo cấu hình của thiết kế M8.

Nền tảng phần cứng được sử dụng là:

- Board: **Digilent Genesys ZU-5EV**.
- FPGA/MPSoC: **xczu5ev-sfvc784-1-e**.
- Công cụ thiết kế: **Xilinx Vivado 2023.2**.
- Tần số mục tiêu của PL: **50 MHz**.
- Batch size: **1**.

- Data interface chính: **AXI4 128 bit**.
- Control interface: **AXI4-Lite**.
- Ràng buộc tài nguyên: **DSP = 0**.

Runtime được sử dụng cho snapshot hiện tại là luồng **host-driven thông qua XSCT/JTAG**. Host chịu trách nhiệm chuẩn bị dữ liệu, nạp model/input, khởi tạo hệ thống, phát lệnh START, chờ DONE và đọc kết quả. Việc xây dựng một application hoàn chỉnh chạy trực tiếp trên Cortex-A53 chưa thuộc phiên bản được đánh giá.

1.4.2 Phạm vi dữ liệu, quy tắc đo và snapshot

Báo cáo sử dụng nhiều nguồn bằng chứng khác nhau. Để tránh việc một giá trị ước lượng bị trình bày như kết quả thực nghiệm, các thông tin được phân thành bốn mức:

- **MEASURED**: số liệu được lấy trực tiếp từ simulation, Vivado report, performance counter hoặc quá trình thực thi trên board.
- **VERIFIED**: thông tin được đối chiếu trực tiếp từ RTL, configuration, descriptor hoặc cấu trúc project.
- **ESTIMATED**: giá trị được tính toán hoặc suy ra từ mô hình và thông số hệ thống.
- **USER-REPORTED**: kết quả đã được ghi nhận trong snapshot dự án nhưng chưa có đủ raw artifact để kiểm chứng độc lập tại thời điểm tổng hợp báo cáo.

Đối với kiểm chứng board, phạm vi bằng chứng chính bao gồm:

- chương trình phần cứng đã được tạo bitstream và chạy trên Genesys ZU-5EV;
- full schedule gồm 249 command có thể đi đến trạng thái hoàn thành;
- kết quả phân lớp Top-1 có thể được đọc từ phần cứng;
- vector logits/probability có thể được so sánh với reference;
- timing và resource utilization được lấy từ báo cáo Vivado;
- các performance counter được sử dụng để phân tích mức hoạt động và stall của datapath.

Snapshot cuối sử dụng trong báo cáo gồm 1.000 ảnh đầu tiên của tập ImageNet ILSVRC2012 validation (chỉ số 1–1000). Kết quả tổng hợp ghi nhận 806 ảnh dự đoán đúng và 194 ảnh sai, tương ứng Top-1 accuracy 80,6%. Kết quả này được trình bày tách biệt với kiểm chứng bit-exact/numerical-vector vì một job hoàn tất và dự đoán Top-1 đúng không đồng nghĩa toàn bộ tensor trung gian trùng tuyệt đối với software reference.

Việc đánh giá được thực hiện theo hai hướng độc lập:

1. **Đánh giá tính đúng đắn**, bao gồm command execution, trạng thái phần cứng, Top-1 prediction và sai khác số học so với reference.
2. **Đánh giá đặc tính phần cứng**, bao gồm timing closure, resource utilization và các counter phản ánh hành vi thực thi.

Cách phân chia này đặc biệt quan trọng vì việc một thiết kế hoàn thành toàn bộ schedule không đồng nghĩa với việc toàn bộ vector số học đã khớp hoàn toàn với software reference.

1.4.3 Các nội dung ngoài phạm vi

Những nội dung sau không thuộc phạm vi hiện thực và đánh giá chính của phiên bản M8:

- ViT-Large, DeiT, Swin Transformer, BERT hoặc các kiến trúc khác ngoài ViT-Base/Patch16-224.
- Batch size lớn hơn 1.
- Dynamic input shape.
- Quantization INT8/INT4.
- Pruning hoặc model compression.
- Model-agnostic compiler.
- Dynamic graph scheduling.
- Tự động sinh descriptor cho nhiều loại mô hình.
- Linux kernel driver cho NPU.
- Runtime hoàn toàn độc lập trên Cortex-A53.
- Production-ready deployment.
- Tối ưu cực đại về throughput.
- So sánh nhằm chứng minh M8 nhanh hơn các accelerator FPGA/GPU/ASIC hiện đại.

Các nội dung trên có thể được sử dụng làm định hướng phát triển nhưng không được xem là chức năng đã hoàn thành của hệ thống hiện tại.

1.5 Phương pháp tiếp cận và nguồn bằng chứng

Báo cáo áp dụng phương pháp tiếp cận theo hướng **design–implement–verify–measure**, trong đó kiến trúc không chỉ được mô tả ở mức ý tưởng mà phải được kiểm tra qua các artifact cụ thể của quá trình phát triển FPGA.

Quy trình được thực hiện qua năm giai đoạn chính.

Phân tích workload

Trước tiên, ViT-Base/Patch16-224 được phân rã thành các phép toán thành phần. Các phép toán được chia thành hai nhóm chính:

- **GEMM**, gồm các linear projection, attention matrix multiplication, MLP và classifier;
- **non-GEMM**, gồm LayerNorm, Softmax, GELU, vector/residual operation, layout transformation và Argmax.

Việc phân loại này là cơ sở để lựa chọn kiến trúc datapath, mức song song và cách chia sẻ tài nguyên.

Thiết kế RTL

Các khối phần cứng được xây dựng từ mức arithmetic primitive đến mức engine và hệ thống. GEMM engine chịu trách nhiệm cho các phép nhân ma trận, trong khi các non-GEMM engine xử lý những phép toán còn lại. Sequencer và command controller điều phối thứ tự thực thi của toàn mô hình.

Memory frontend và AXI adapter chịu trách nhiệm kết nối các engine với DDR. Register bank cung cấp đường điều khiển giữa PS và PL.

Kiểm chứng nhiều cấp

Quá trình kiểm chứng được tổ chức theo nhiều tầng:

Leaf level → Engine level → System level → FPGA board level.

Ở leaf level, các primitive số học và bộ chuyển đổi dữ liệu được kiểm tra riêng. Ở engine level, GEMM và từng engine non-GEMM được kiểm tra bằng testbench tương ứng. Sau đó các thành phần được tích hợp để kiểm tra command flow và memory interface ở mức hệ thống. Cuối cùng, thiết kế được chạy trên board vật lý.

Cách kiểm chứng phân tầng giúp thu hẹp phạm vi lỗi khi kết quả end-to-end khác với software reference.

Synthesis, implementation và physical validation

Thiết kế RTL được tổng hợp và triển khai bằng Vivado 2023.2 cho xczu5ev-sfvc784-1-e. Các báo cáo synthesis và implementation được dùng để xác định:

- LUT utilization;
- FF utilization;
- BRAM utilization;
- DSP utilization;
- timing slack;
- critical path.

Sau khi đạt điều kiện implementation, bitstream và hardware artifact được tạo để lập trình lên board Genesys ZU-5EV.

Đánh giá dựa trên bằng chứng

Các nguồn bằng chứng chính của báo cáo gồm:

- RTL source và configuration;
- descriptor và sequencer schedule;
- ModelSim/XSim testbench;
- Vivado synthesis report;

- Vivado utilization report;
- routed timing report;
- bitstream/XSA;
- board execution log;
- performance counter;
- output tensor và software reference khi có.

Một nguyên tắc quan trọng của báo cáo là **không đồng nhất giữa giá trị thiết kế, giá trị tính toán và giá trị đo được**. Mỗi kết luận định lượng ở các chương sau cần được gắn với nguồn bằng chứng tương ứng.

Cách tiếp cận này cho phép báo cáo không chỉ trả lời câu hỏi “*NPU có chạy hay không?*”, mà còn trả lời các câu hỏi sâu hơn như “*điểm nghẽn nằm ở đâu?*”, “*giới hạn đến từ datapath hay memory?*”, “*kết quả nào đã được đo trực tiếp?*” và “*kết quả nào mới chỉ là ước lượng hoặc hướng phát triển?*”.

1.6 Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức như sau:

Chương 2 — Nền tảng lý thuyết trình bày các kiến thức cần thiết để hiểu hệ thống, bao gồm Vision Transformer, patch embedding, self-attention, MLP, các phép toán non-GEMM, biểu diễn số FP16/FP32, kiến trúc FPGA/Zynq UltraScale+, giao thức AXI và nguyên lý của các kiến trúc tính toán ma trận.

Chương 3 — Khảo sát tài liệu và công trình liên quan tổng hợp các nghiên cứu về tăng tốc Transformer và Vision Transformer trên FPGA, các kiến trúc GEMM, systolic array, mixed precision và những kỹ thuật tổ chức bộ nhớ có liên quan. Từ đó xác định vị trí và mục tiêu của thiết kế được trình bày trong báo cáo.

Chương 4 — Thiết kế hệ thống trình bày kiến trúc tổng thể của NPU, kết nối PS–PL, GEMM engine, các non-GEMM engine, memory subsystem, AXI adapter, command controller, sequencer, descriptor và luồng dữ liệu của một lần suy luận ViT.

Chương 5 — Hiện thực và triển khai mô tả quá trình chuyển kiến trúc sang RTL, cấu trúc project Vivado, synthesis, implementation, constraint, timing closure, bitstream/XSA, boot flow và phương pháp đưa thiết kế lên Genesys ZU-5EV.

Chương 6 — Kiểm chứng và kết quả trình bày phương pháp verification từ leaf-level đến board-level, kết quả synthesis và implementation, kết quả thực thi trên FPGA, kiểm tra đầu ra mô hình và các số liệu performance counter thu được trong quá trình thực nghiệm.

Chương 7 — Đánh giá và thảo luận phân tích các kết quả thu được, bao gồm trade-off của kiến trúc DSP=0, sai khác số học, critical path, memory feeding, mức sử dụng datapath và mối quan hệ giữa timing bottleneck với runtime bottleneck.

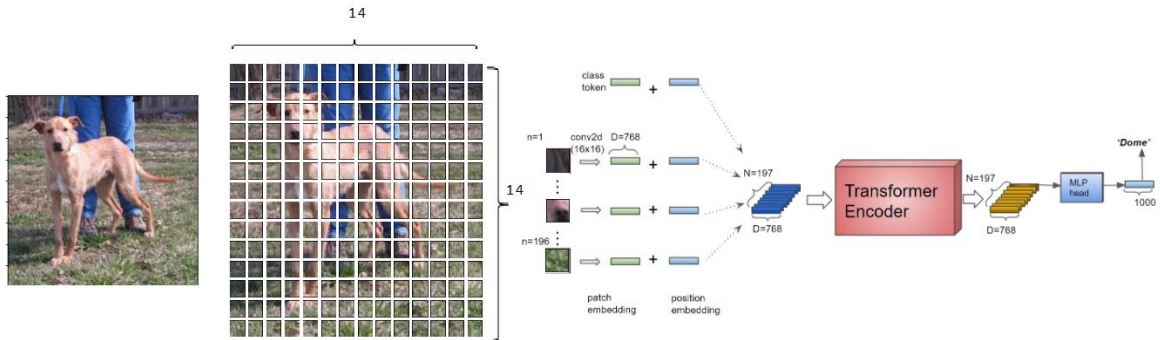
Chương 8 — Kết luận và hướng phát triển tổng hợp những nội dung đã hoàn thành, các giới hạn còn tồn tại và những hướng cải tiến tiếp theo như tối ưu memory frontend, cải thiện numerical accuracy, tăng timing margin, phát triển runtime trên Cortex-A53 và khảo sát các cấu hình kiến trúc khác.

CHƯƠNG 2 — NỀN TẢNG LÝ THUYẾT

2.1 Vision Transformer

2.1.1 Ảnh đầu vào, patchify và patch embedding

Trong Vision Transformer (ViT), ảnh đầu vào không được xử lý trực tiếp dưới dạng toàn bộ ma trận ảnh. Thay vào đó, ảnh được chia thành nhiều vùng nhỏ có kích thước cố định gọi là **patch**. Cách biểu diễn này cho phép mỗi patch đóng vai trò tương tự một “token” trong mô hình Transformer. Ý tưởng này được giới thiệu trong công trình gốc về ViT của Dosovitskiy và cộng sự.



Với mô hình **ViT-Base/Patch16-224**, ảnh đầu vào có kích thước:

$$224 \times 224 \times 3$$

trong đó 3 là số kênh RGB. Ảnh được chia thành các patch có kích thước:

$$16 \times 16 \times 3$$

Số patch thu được là:

$$N = 224/16 \times 224/16 = 14 \times 14 = 196$$

Mỗi patch sau đó được làm phẳng thành một vector có:

$$16 \times 16 \times 3 = 768$$

phần tử. Vector này được đưa qua một phép biến đổi tuyến tính để tạo **patch embedding** có kích thước 768 chiều:

$$E_i = X_i W_E + b_E$$

Trong đó X_i là patch thứ i , còn W_E và b_E là trọng số và bias của lớp patch embedding. Sau bước này, 196 patch được biểu diễn dưới dạng 196 vector đặc trưng, mỗi vector có chiều 768.

Tiếp theo, mô hình thêm một **CLS token** vào đầu chuỗi và cộng thêm **positional embedding** để cung cấp thông tin vị trí cho từng token. Vì vậy đầu vào của Transformer Encoder có kích thước:

$$[1, 197, 768]$$

2.1.2 CLS token và positional embedding

Sau khi ảnh đầu vào được chia thành 196 patch và mỗi patch được ánh xạ thành một vector embedding có kích thước 768, Vision Transformer bổ sung thêm hai thành phần trước khi đưa dữ liệu vào Transformer Encoder: **CLS token** và **Positional Embedding**.

CLS Token

CLS token (Classification Token) là một vector có thể học được với kích thước:

$$1 \times 768$$

Token này được chèn vào vị trí đầu tiên của chuỗi patch embedding. Với 196 patch ban đầu, sau khi thêm CLS token, số lượng token trở thành:

$$196 + 1 = 197$$

Do đó kích thước của chuỗi sau bước này là:

$$[197, 768]$$

Có thể biểu diễn như sau:

$$X_{\text{patch}} \in \mathbb{R}^{196 \times 768} \quad X_{\text{CLS}} \in \mathbb{R}^{1 \times 768} \quad X_{\text{intermediate}} = [X_{\text{CLS}}; X_{\text{patch}}] \in \mathbb{R}^{197 \times 768}$$

CLS token được truyền qua toàn bộ các lớp Transformer Encoder cùng với các patch token. Sau encoder cuối cùng, vector tương ứng với CLS token được sử dụng làm đặc trưng tổng hợp của ảnh và được đưa vào lớp classifier để thực hiện phân loại.

Positional Embedding

Cơ chế self-attention của Transformer không tự cung cấp thông tin về vị trí không gian của các patch. Do đó ViT sử dụng **Positional Embedding** để bổ sung thông tin vị trí cho từng token.

Với ViT-Base/Patch16-224, Positional Embedding là một tham số có thể học được với kích thước:

$$197 \times 768$$

tương ứng với:

- 1 vị trí dành cho CLS token;
- 196 vị trí dành cho các patch token.

Positional Embedding được cộng trực tiếp với chuỗi embedding sau khi đã thêm CLS token:

$$X_{input} = X_{intermediate} + E_{pos}$$

trong đó:

$$X_{intermediate} \in \mathbb{R}^{197 \times 768}$$

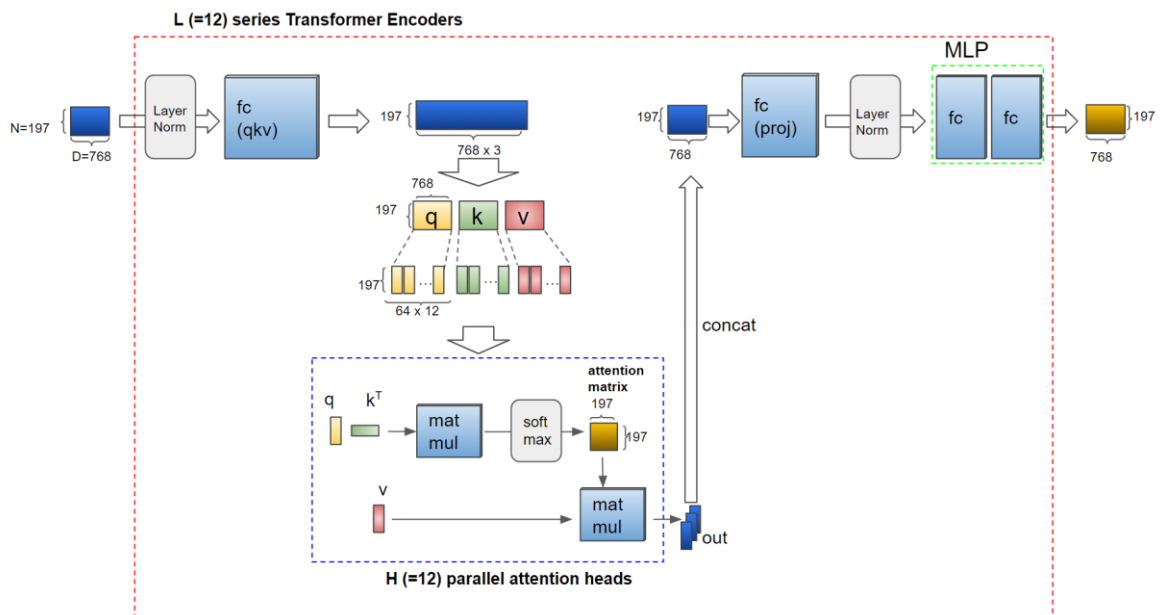
và

$$E_{pos} \in \mathbb{R}^{197 \times 768}$$

Do đó đầu ra cuối cùng trước khi đưa vào Transformer Encoder vẫn có kích thước:

$$X_{input} \in \mathbb{R}^{197 \times 768}$$

2.1.3 Transformer encoder



Sau giai đoạn Patch Embedding, CLS Token và Positional Embedding, dữ liệu đầu vào của Vision Transformer có kích thước:

$$X_0 \in \mathbb{R}^{197 \times 768}$$

trong đó 197 là tổng số token gồm 196 patch token và 1 CLS token, còn 768 là kích thước đặc trưng của mỗi token.

Đối với mô hình **ViT-Base/Patch16-224**, phần Transformer Encoder gồm **12 encoder layer nối tiếp nhau**. Mỗi encoder layer nhận vào tensor có kích thước 197×768 và tạo ra tensor đầu ra có cùng kích thước. Vì vậy, mặc dù dữ liệu trải qua nhiều phép biến đổi bên trong encoder, số lượng token và kích thước embedding vẫn được duy trì xuyên suốt 12 lớp.

Kiến trúc ViT sử dụng dạng **Pre-LayerNorm**, nghĩa là Layer Normalization được thực hiện trước khối Attention và trước khối MLP. Ở mức tổng quát, một encoder layer có thể được mô tả bởi:

$$Y_l = X_l + \text{MSA}(\text{LN}_1(X_l))$$

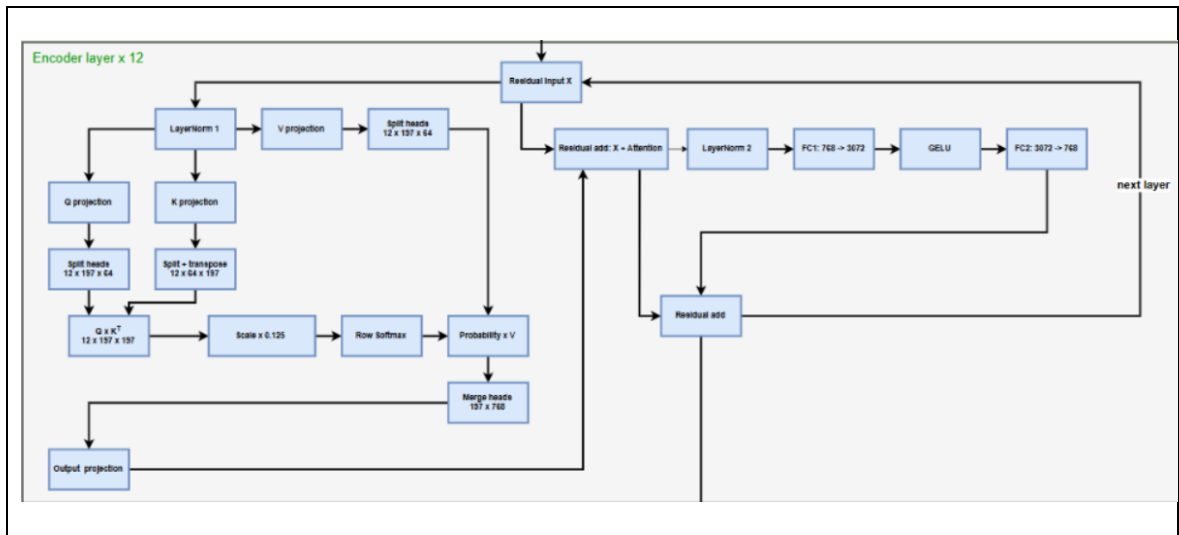
và:

$$X_{l+1} = Y_l + \text{MLP}(\text{LN}_2(Y_l))$$

Trong đó:

- X_l : đầu vào của encoder layer thứ l ;
- LN : phép Layer Normalization;
- MSA : Multi-Head Self-Attention;
- MLP : mạng feed-forward bên trong Transformer;
- Y_l : kết quả sau nhánh Attention và phép cộng residual;
- X_{l+1} : đầu ra của encoder layer hiện tại và đồng thời là đầu vào của encoder layer tiếp theo

Với ViT-Base/Patch16-224, quá trình xử lý có dạng:



Như vậy, toàn bộ encoder có thể xem như một chuỗi gồm 12 khối xử lý có cùng cấu trúc nhưng sử dụng các **bộ trọng số riêng biệt cho từng layer**.

Các thành phần chính của một encoder layer

Ở mức kiến trúc, mỗi encoder layer có thể chia thành hai nhánh tính toán chính:

Thành phần	Chức năng chính	Kích thước đầu vào/đầu ra
LayerNorm 1	Chuẩn hóa dữ liệu trước Attention	197×768
Multi-Head Self-Attention	Học mối quan hệ giữa các token	197×768
Residual Add 1	Cộng lại dữ liệu đầu vào	197×768
LayerNorm 2	Chuẩn hóa trước MLP	197×768
MLP	Biến đổi đặc trưng theo từng token	197×768
Residual Add 2	Tạo đầu ra của encoder layer	197×768

Hình minh họa kiến trúc encoder cho thấy dữ liệu 197×768 trước tiên đi qua LayerNorm và khối Attention. Trong khối Attention, embedding 768 chiều được phân chia thành **12 attention head**, mỗi head có kích thước 64 chiều:

$$768=12\times 64$$

Sau khi các head được xử lý, kết quả được ghép lại để khôi phục kích thước 768 chiều trước khi tiếp tục qua projection và nhánh MLP. Tuy nhiên, cơ chế tính toán chi tiết của Q, K, V và Multi-Head Self-Attention sẽ được trình bày riêng ở mục **2.1.4**.

Tương tự, cấu trúc hai lớp fully-connected của MLP, hàm kích hoạt GELU, residual connection và phần classifier cuối mô hình sẽ được trình bày ở mục **2.1.5**.

Vai trò của Transformer Encoder

Transformer Encoder là phần chính chịu trách nhiệm trích xuất và biến đổi đặc trưng của ảnh. Thông qua 12 encoder layer liên tiếp, mỗi token có thể trao đổi thông tin với các token khác thông qua cơ chế self-attention, trong khi MLP tiếp tục biến đổi đặc trưng của từng token.

Sau encoder layer cuối cùng, tensor vẫn có kích thước:

$$197 \times 768$$

Trong đó vector tương ứng với CLS token chứa thông tin tổng hợp của ảnh và được sử dụng ở giai đoạn phân loại cuối cùng.

2.1.4 Multi-Head Self-Attention

Trong Vision Transformer, **Multi-Head Self-Attention (MHSA)** là khối chịu trách nhiệm mô hình hóa quan hệ giữa các token trong chuỗi đầu vào. Khác với CNN, nơi đặc trưng chủ yếu được trích xuất thông qua vùng lân cận cục bộ, self-attention cho phép mỗi token tương tác trực tiếp với tất cả các token còn lại. Nhờ đó, mô hình có thể học được cả thông tin cục bộ lẫn phụ thuộc dài hạn trên toàn ảnh.

a) Đầu vào của khối Multi-Head Self-Attention

Sau khi qua bước embedding, đầu vào của mỗi encoder layer trong ViT-Base/Patch16-224 có dạng:

$$X \in \mathbb{R}^{197 \times 768}$$

trong đó:

- **197** là số token, gồm 196 patch token và 1 CLS token,
- **768** là số chiều đặc trưng của mỗi token.

Trước khi đưa vào self-attention, đầu vào X được chuẩn hóa qua LayerNorm. Từ tensor này, mô hình sử dụng ba phép chiếu tuyến tính độc lập để tạo ra ba ma trận:

- **Query (Q)**
- **Key (K)**
- **Value (V)**

Cụ thể:

$$\begin{aligned} Q &= XW_Q + b_Q \\ K &= XW_K + b_K \\ V &= XW_V + b_V \end{aligned}$$

với:

$$W_Q, W_K, W_V \in \mathbb{R}^{768 \times 768}$$

Khi đó, kích thước của ba ma trận thu được là:

$$Q, K, V \in \mathbb{R}^{(197 \times 768)}$$

b) Chia thành nhiều attention head

Trong mô hình ViT-Base, khối self-attention sử dụng **12 attention head song song**. Do embedding dimension bằng 768 nên mỗi head nhận một phần không gian đặc trưng có kích thước:

$$d_{\text{head}} = 768 / 12 = 64$$

Như vậy, các ma trận Q, K, V được tách thành 12 head:

$$Q \rightarrow \{Q_1, Q_2, \dots, Q_{12}\}$$

$$K \rightarrow \{K_1, K_2, \dots, K_{12}\}$$

$$V \rightarrow \{V_1, V_2, \dots, V_{12}\}$$

với:

$$Q_i, K_i, V_i \in \mathbb{R}^{197 \times 64}$$

Việc chia thành nhiều head giúp mô hình học đồng thời nhiều kiểu quan hệ khác nhau giữa các token. Mỗi head có thể tập trung vào một dạng thông tin riêng, chẳng hạn như quan hệ vị trí, hình dạng, biên hoặc mức tương đồng giữa các vùng ảnh.

c) Tính điểm attention

Đối với từng head, cơ chế self-attention được thực hiện bằng cách nhân ma trận Query với chuyển vị của Key:

$$S_i = Q_i K_i^T$$

Khi đó:

$$S_i \in \mathbb{R}^{197 \times 197}$$

Ma trận S_i biểu diễn mức độ liên quan giữa từng token với tất cả các token còn lại. Tuy nhiên, để tránh giá trị tích vô hướng quá lớn khi số chiều tăng lên, kết quả này được chuẩn hóa theo hệ số d_{head} :

$$S^i = 64 Q_i K_i^T / \text{can}(64)$$

Sau đó, hàm softmax được áp dụng theo từng hàng để thu được trọng số attention:

$$A_i = \text{Softmax}(64 Q_i K_i^T)$$

trong đó:

$$A_i \in \mathbb{R}^{197 \times 197}$$

Mỗi hàng của ma trận A_i biểu diễn phân bố trọng số mà một token gán cho toàn bộ các token khác trong chuỗi đầu vào.

d) Tính đầu ra của từng head

Sau khi có ma trận trọng số attention, đầu ra của mỗi head được tính bằng cách nhân ma trận attention với ma trận Value:

$$O_i = A_i V_i$$

với:

$$O_i \in \mathbb{R}^{197 \times 64}$$

Ý nghĩa của phép tính này là: đầu ra của mỗi token được tạo thành từ tổ hợp có trọng số của tất cả các token trong chuỗi, trong đó trọng số được quyết định bởi mức độ liên quan do cơ chế attention học được.

e) Ghép các head và chiếu đầu ra

Kết quả của 12 head sau đó được ghép lại theo chiều đặc trưng:

$$O = \text{Concat}(O_1, O_2, \dots, O_{12})$$

nên:

$$O \in \mathbb{R}^{(197 \times 768)}$$

Tiếp theo, một phép chiếu tuyến tính đầu ra được áp dụng:

$$Y = OW + b_O$$

với:

$$W_O \in \mathbb{R}^{768 \times 768}$$

Kết quả cuối cùng của khối Multi-Head Self-Attention là:

$$Y \in \mathbb{R}^{(197 \times 768)}$$

Tensor này sau đó được cộng residual với đầu vào của nhánh attention để tạo thành đầu ra trung gian của encoder layer.

f) Công thức tổng quát của Multi-Head Self-Attention

Toàn bộ quá trình MHSA có thể viết gọn dưới dạng:

$$\text{head}_i = \text{Softmax}(\text{dhead}_i \text{Q}_i \text{K}_i^T) \text{V}_i$$

$$\text{MHSA}(\text{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_{12}) \text{W}_O$$

Trong trường hợp ViT-Base/Patch16-224:

- số head: **12**
- kích thước mỗi head: **64**
- số token: **197**
- embedding dimension: **768**

g) Vai trò của Multi-Head Self-Attention trong ViT

Khối MHSA là thành phần cốt lõi giúp Vision Transformer xử lý ảnh dưới dạng chuỗi token. Thay vì chỉ khai thác quan hệ cục bộ như tích chập, self-attention cho phép mô hình xây dựng quan hệ toàn cục giữa các patch trong ảnh. Điều này đặc biệt quan trọng với các bài toán phân loại ảnh, vì thông tin quyết định nhãn có thể phân bố ở nhiều vùng khác nhau và không nhất thiết nằm gần nhau về mặt không gian.

Ngoài ra, việc sử dụng nhiều head song song giúp mô hình tăng khả năng biểu diễn. Mỗi head học một dạng quan hệ riêng, và khi tổng hợp lại, đầu ra của MHSA mang thông tin phong phú hơn so với attention đơn đầu.

h) Liên hệ với triển khai phần cứng

Từ góc độ phần cứng, khối Multi-Head Self-Attention bao gồm một chuỗi các phép toán chính:

1. **Linear projection** để tạo Q, K, V,
2. **Matrix multiplication** để tính QKT,
3. **Softmax** để chuẩn hóa trọng số attention,
4. **Matrix multiplication** giữa attention và V,
5. **Concatenation** và **output projection**.

Trong đó, các phép nhân ma trận lớn như XW_Q , XW_K , XW_V , QKT , $\text{attention} \times V$ và output projection là những phần có chi phí tính toán cao nhất. Đây cũng chính là lý do trong thiết kế accelerator cho ViT, các khối GEMM và các engine hỗ trợ non-GEMM như Softmax trở thành các thành phần quan trọng cần được tối ưu.

2.1.5 MLP, residual connection và classifier

Sau khối Multi-Head Self-Attention, dữ liệu trong mỗi encoder layer tiếp tục được xử lý bởi khối **MLP (Multi-Layer Perceptron)**. Đồng thời, Vision Transformer sử dụng các **residual connection** để duy trì thông tin từ đầu vào của từng nhánh và giúp quá trình huấn luyện ổn định

hơn. Sau khi hoàn thành toàn bộ 12 encoder layer, đặc trưng của CLS token được đưa vào **classifier** để thực hiện phân loại ảnh.

a) Khối MLP trong Transformer Encoder

Trong ViT-Base/Patch16-224, khối MLP gồm hai lớp fully connected nối tiếp nhau. Đầu vào của MLP vẫn có kích thước:

$$X \in \mathbb{R}^{197 \times 768}$$

Trước khi đưa vào MLP, dữ liệu được chuẩn hóa bằng LayerNorm. Lớp fully connected đầu tiên mở rộng chiều đặc trưng từ 768 lên 3072:

$$H = XW_1 + b_1$$

với:

$$W_1 \in \mathbb{R}^{768 \times 3072}$$

Do đó:

$$H \in \mathbb{R}^{(197 \times 3072)}$$

Sau lớp tuyến tính đầu tiên, hàm kích hoạt **GELU (Gaussian Error Linear Unit)** được áp dụng:

$$H' = \text{GELU}(H)$$

Tiếp theo, lớp fully connected thứ hai giảm kích thước đặc trưng từ 3072 trở lại 768:

$$Y = H'W_2 + b_2$$

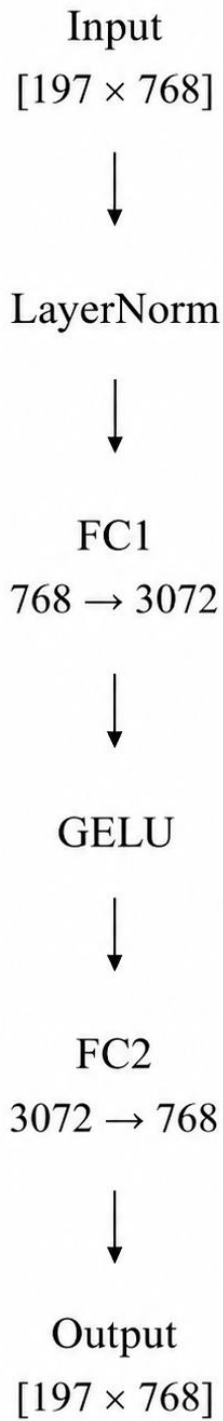
trong đó:

$$W_2 \in \mathbb{R}^{3072 \times 768}$$

và:

$$Y \in \mathbb{R}^{(197 \times 768)}$$

Như vậy, MLP có cấu trúc tổng quát:



Việc mở rộng chiều đặc trưng lên 3072 cho phép MLP học các phép biến đổi phi tuyến có mức biểu diễn cao hơn, trong khi lớp thứ hai đưa dữ liệu trở lại embedding dimension 768 để duy trì kích thước thống nhất giữa các encoder layer.

b) Residual Connection

Vision Transformer sử dụng residual connection ở cả hai nhánh chính của encoder layer: sau Multi-Head Self-Attention và sau MLP.

Với nhánh Attention:

$$Y = X + \text{MSA}(\text{LN1}(X))$$

Với nhánh MLP:

$$Z = Y + \text{MLP}(\text{LN2}(Y))$$

Trong đó:

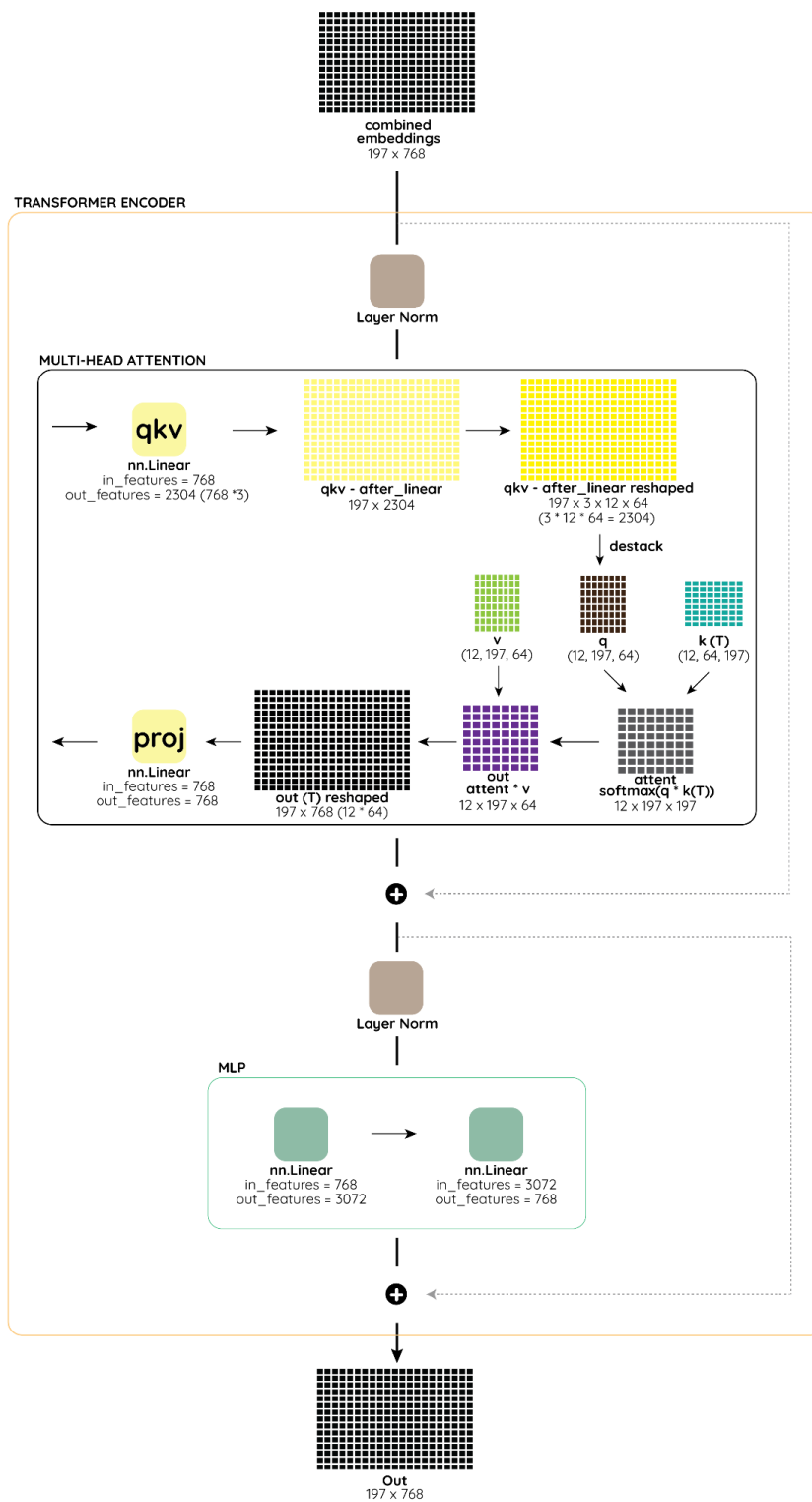
- X: đầu vào encoder layer;
- Y: đầu ra sau nhánh Attention;
- Z: đầu ra cuối cùng của encoder layer.

Residual connection thực hiện phép cộng phần tử giữa đầu vào của nhánh và kết quả tính toán của nhánh đó. Vì vậy, kích thước hai tensor phải giống nhau:

$$197 \times 768$$

Residual connection giúp bảo toàn thông tin ban đầu của token, cải thiện quá trình lan truyền gradient và giúp các Transformer có nhiều layer được huấn luyện ổn định hơn.

Luồng của một encoder layer có thể tóm tắt như sau:



Sau mỗi encoder layer, kích thước đầu ra vẫn giữ nguyên:

$$Z \in \mathbb{R}^{197 \times 768}$$

và được đưa trực tiếp sang encoder layer tiếp theo.

c) Đầu ra sau 12 Encoder Layer

Sau khi dữ liệu đi qua toàn bộ 12 encoder layer, đầu ra vẫn là một chuỗi gồm 197 token:

$$X_{enc} \in \mathbb{R}^{197 \times 768}$$

Trong đó token đầu tiên là CLS token:

$$X_{CLS} \in \mathbb{R}^{1 \times 768}$$

CLS token đã trao đổi thông tin với các patch token thông qua các lớp self-attention xuyên suốt 12 encoder layer. Vì vậy, vector này được sử dụng làm biểu diễn tổng hợp của toàn bộ ảnh cho bước phân loại.

Trước khi đưa vào classifier, output encoder được xử lý bằng **Final LayerNorm**.

d) Classifier

Đối với mô hình ViT-Base/Patch16-224 được sử dụng cho ImageNet-1K, classifier nhận vector CLS có kích thước 768 và ánh xạ sang 1000 giá trị tương ứng với 1000 lớp của ImageNet:

$$L = X_{CLS} W_{cls} + b_{cls}$$

với:

$$W_{cls} \in \mathbb{R}^{768 \times 1000}$$

Do đó:

$$L \in \mathbb{R}^{(1 \times 1000)}$$

Các giá trị này được gọi là **logits**. Mỗi phần tử biểu diễn mức điểm mà mô hình gán cho một lớp tương ứng.

Để xác định kết quả phân loại Top-1, mô hình chọn vị trí có giá trị logit lớn nhất:

$$y^* = \arg\max(L)$$

Trong đó y^* là chỉ số của lớp được dự đoán.

Nếu cần xác suất của toàn bộ 1000 lớp, có thể áp dụng Softmax:

$$P_i = \frac{e^{L_i}}{\sum_{j=1}^{1000} e^{L_j}}$$

Tuy nhiên, nếu mục tiêu chỉ là lấy kết quả Top-1 thì có thể sử dụng trực tiếp Argmax trên logits mà không cần tính Softmax.

e) Liên hệ với triển khai phần cứng

Từ góc nhìn phần cứng, MLP và classifier chứa các phép nhân ma trận lớn nên phù hợp với khối **GEMM Engine**. Các phép cộng residual có thể được thực hiện bởi **Vector Engine**, trong khi GELU và Argmax được xử lý bởi các engine chuyên biệt tương ứng.

Có thể ánh xạ các phép toán như sau:

Thành phần ViT	Phép toán chính	Khối phần cứng tương ứng
FC1	$197 \times 768 \times 768 \times 3072$	GEMM Engine
GELU	Hàm kích hoạt phi tuyến	GELU Engine
FC2	$197 \times 3072 \times 3072 \times 768$	GEMM Engine
Residual Add	Cộng phần tử	Vector Engine
Final LayerNorm	Chuẩn hóa	LayerNorm Engine
Classifier	$1 \times 768 \times 768 \times 1000$	GEMM Engine
Argmax	Tìm phần tử lớn nhất	Argmax Engine
Class Softmax	Chuẩn hóa logits	Softmax Engine, nếu được sử dụng

2.2 Các phép toán nền tảng

2.2.1 GEMM và các dạng nhân ma trận của ViT

Trong Vision Transformer, phần lớn khối lượng tính toán đến từ các phép **nhân ma trận tổng quát (General Matrix Multiplication – GEMM)**. Các phép chiếu tuyến tính để tạo Q, K, V, phép tính attention, hai lớp fully connected trong MLP và lớp classifier cuối đều có thể quy về cùng một dạng tính toán cơ bản. Vì vậy, GEMM là một trong những phép toán quan trọng nhất cần được hỗ trợ khi hiện thực ViT trên phần cứng.

a) Phép toán GEMM tổng quát

Xét hai ma trận:

$$A \in \mathbb{R}^{M \times K}$$

và:

$$B \in \mathbb{R}^{K \times N}$$

phép nhân ma trận tạo ra:

$$C = AB$$

với:

$$C \in \mathbb{R}^{M \times N}$$

Mỗi phần tử của ma trận kết quả được xác định bởi tích vô hướng giữa một hàng của A và một cột của B:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}$$

Trong các lớp tuyến tính của ViT, phép tính thường đi kèm với bias:

$$C = AB + b$$

trong đó vector b được cộng vào từng hàng của ma trận kết quả.

Như vậy, xét ở mức tính toán cơ bản, một GEMM gồm ba thao tác chính:



Ba tham số M, N và K quyết định kích thước và khối lượng tính toán của một GEMM. Đây cũng là ba tham số quan trọng khi xây dựng bộ tăng tốc có khả năng sử dụng lại cho nhiều phép nhân ma trận khác nhau trong ViT.

b) Patch Projection

Sau khi ảnh $224 \times 224 \times 3$ được chia thành 196 patch kích thước $16 \times 16 \times 3$, mỗi patch được làm phẳng thành vector 768 phần tử. Toàn bộ các patch có thể biểu diễn dưới dạng:

$$X_{\text{patch}} \in \mathbb{R}^{196 \times 768}$$

Patch embedding được thực hiện bằng một phép chiếu tuyến tính:

$$E = X_{\text{patch}} W_E + b_E$$

với:

$$W_E \in \mathbb{R}^{768 \times 768}$$

và:

$$E \in \mathbb{R}^{196 \times 768}$$

Do đó, patch projection cũng có thể được biểu diễn dưới dạng một GEMM.

c) Q, K và V Projection

Trong mỗi Transformer Encoder, đầu vào sau LayerNorm có kích thước:

$$X \in \mathbb{R}^{197 \times 768}$$

Ba phép chiếu tuyến tính tạo Query, Key và Value được tính như sau:

$$Q = XW_Q + b_Q \quad K = XW_K + b_K \quad V = XW_V + b_V$$

trong đó:

$$W_Q, W_K, W_V \in \mathbb{R}^{768 \times 768}$$

Do đó:

$$Q, K, V \in \mathbb{R}^{(197 \times 768)}$$

Ba phép chiếu này có cùng dạng GEMM:

$$[197 \times 768] \times [768 \times 768] \rightarrow [197 \times 768]$$

Điểm khác biệt chủ yếu nằm ở bộ trọng số, bias và vùng nhớ chứa kết quả.

d) Nhân ma trận trong Self-Attention

Sau khi tạo Q, K và V, mỗi tensor được chia thành 12 attention head. Với embedding dimension bằng 768, kích thước của mỗi head là:

$$d_{\text{head}} = 768 / 12 = 64$$

Do đó:

$$Q_i, K_i, V_i \in \mathbb{R}^{197 \times 64}$$

Để tạo attention score, mỗi head thực hiện phép nhân:

$$S_i = Q_i K_i^T$$

với:

$$[197 \times 64] \times [64 \times 197] \rightarrow [197 \times 197]$$

Sau bước scale và Softmax, ma trận attention tiếp tục được nhân với V_i :

$$O_i = A_i V_i$$

tương ứng:

$$[197 \times 197] \times [197 \times 64] \rightarrow [197 \times 64]$$

Như vậy, mặc dù kích thước của các ma trận khác với phép Q/K/V projection, cả hai phép toán vẫn có thể quy về cùng dạng GEMM tổng quát với các giá trị M, N và K khác nhau.

Sau khi kết quả của 12 head được ghép lại thành tensor 197×768 , một phép output projection tiếp tục được thực hiện:

$$Y = OW + b_O$$

với:

$$[197 \times 768] \times [768 \times 768] \rightarrow [197 \times 768]$$

e) GEMM trong khối MLP

MLP của ViT-Base gồm hai lớp fully connected.

Lớp thứ nhất mở rộng embedding dimension từ 768 lên 3072:

$$H = XW_1 + b_1$$

tương ứng:

$$[197 \times 768] \times [768 \times 3072] \rightarrow [197 \times 3072]$$

Sau khi qua GELU, lớp fully connected thứ hai đưa chiều đặc trưng trở lại 768:

$$Y = HW_2 + b_2$$

tương ứng:

$$[197 \times 3072] \times [3072 \times 768] \rightarrow [197 \times 768]$$

Hai phép nhân này có kích thước lớn và chiếm một phần đáng kể khối lượng tính toán của mỗi encoder layer.

f) GEMM trong lớp Classifier

Sau encoder layer cuối cùng và Final LayerNorm, CLS token được lấy ra với kích thước:

$$X_{CLS} \in \mathbb{R}^{1 \times 768}$$

Đối với ImageNet-1K, classifier ánh xạ vector này thành 1000 logits:

$$L = X_{CLS} W_{cls} + b_{cls}$$

với:

$$W_{cls} \in \mathbb{R}^{768 \times 1000}$$

Do đó:

$$[1 \times 768] \times [768 \times 1000] \rightarrow [1 \times 1000]$$

Đây vẫn là cùng một dạng GEMM, nhưng có $M=1$.

g) Tổng hợp các dạng GEMM trong ViT-Base/Patch16-224

Các phép nhân ma trận chính có thể tổng hợp như sau:

Phép toán	Ma trận A	Ma trận B	Kết quả
Patch Projection	196×768	768×768	196×768
Q Projection	197×768	768×768	197×768
K Projection	197×768	768×768	197×768

V Projection	197×768	768×768	197×768
QKT, mỗi head	197×64	64×197	197×197
Attention × V, mỗi head	197×197	197×64	197×64
Output Projection	197×768	768×768	197×768
MLP – FC1	197×768	768×3072	197×3072
MLP – FC2	197×3072	3072×768	197×768
Classifier	1×768	768×1000	1×1000

Bảng trên cho thấy các phép nhân ma trận trong ViT có kích thước rất khác nhau nhưng đều có thể biểu diễn dưới dạng:

$$[M \times K] \times [K \times N] \rightarrow [M \times N]$$

Đây là cơ sở để sử dụng **một primitive GEMM tổng quát và tái sử dụng cùng một khối tính toán cho nhiều giai đoạn khác nhau của mô hình**, thay vì xây dựng một phần cứng riêng cho từng phép Q projection, FC1, FC2 hay classifier.

h) Đặc điểm tính toán cần hỗ trợ khi ánh xạ xuống phần cứng

Từ các phép GEMM trên có thể rút ra một số yêu cầu tính toán cơ bản đối với bộ tăng tốc:

- Hỗ trợ các giá trị M, N và K khác nhau tùy từng lớp.
- Hỗ trợ phép tích lũy nhiều tích $A_{m,k}B_{k,n}$ theo chiều K.
- Hỗ trợ bias đối với các lớp linear projection.
- Xử lý được các kích thước không nhất thiết chia hết cho kích thước song song vật lý của phần cứng.
- Cho phép tái sử dụng cùng một primitive GEMM cho projection, attention, MLP và classifier.
- Cần phối hợp giữa khả năng tính toán và việc cung cấp dữ liệu từ bộ nhớ, do lượng operand của các GEMM lớn.

2.2.2 LayerNorm

Layer Normalization (LayerNorm) được sử dụng nhiều lần trong Vision Transformer để chuẩn hóa đặc trưng của từng token trước các khối Multi-Head Self-Attention và MLP. Với ViT-Base/Patch16-224, mỗi token có kích thước đặc trưng:

$$D=768$$

Giả sử vector đặc trưng của một token là:

$$x=[x_0, x_1, \dots, x_{D-1}]$$

LayerNorm trước tiên tính giá trị trung bình:

$$\mu = \frac{1}{D} \sum_{i=0}^{D-1} x_i$$

Sau đó tính phương sai:

$$\sigma^2 = \frac{1}{D} \sum_{i=0}^{D-1} (x_i - \mu)^2$$

Để tránh phép chia cho 0 hoặc giá trị quá nhỏ, một hằng số ϵ được cộng vào phương sai. Giá trị chuẩn hóa của mỗi phần tử được xác định bởi:

$$x_i^{\wedge} = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Cuối cùng, LayerNorm thực hiện phép biến đổi affine bằng hai tham số có thể học được γ và β :

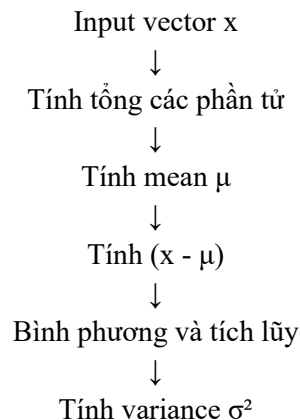
$$y_i = \gamma x_i^{\wedge} + \beta_i$$

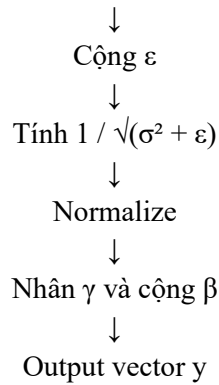
Do đó, kích thước đầu vào và đầu ra của LayerNorm không thay đổi:

$$[197 \times 768] \rightarrow [197 \times 768]$$

Trình tự tính toán

Ở mức thuật toán, LayerNorm có thể phân chia thành các bước:





Có thể thấy LayerNorm không chỉ bao gồm phép cộng và nhân thông thường mà còn cần:

- phép tích lũy;
- phép chia hoặc reciprocal;
- phép căn bậc hai nghịch đảo;
- phép nhân và cộng affine.

Đây là đặc điểm quan trọng khi ánh xạ LayerNorm sang phần cứng, bởi cùng một vector đầu vào cần được sử dụng qua nhiều giai đoạn tính toán khác nhau.

Yêu cầu tính toán đối với phần cứng

Từ thuật toán LayerNorm có thể rút ra các yêu cầu:

- hỗ trợ tích lũy trên toàn bộ chiều đặc trưng;
- duy trì đủ độ chính xác khi tính mean và variance;
- hỗ trợ reciprocal và inverse square root;
- hỗ trợ các phép nhân và cộng FP32;
- cho phép tái sử dụng dữ liệu đầu vào ở nhiều giai đoạn khác nhau.

Cách lưu trữ dữ liệu trung gian, cách thực hiện inverse square root và cách tổ chức datapath LayerNorm sẽ được trình bày trong chương kiến trúc phần cứng.

2.2.3 Softmax ổn định số

Softmax được sử dụng trong khối Self-Attention để chuyển các attention score thành các trọng số có tổng bằng 1.

Với vector đầu vào:

$$x=[x_0, x_1, \dots, x_{N-1}]$$

Softmax thông thường được định nghĩa:

$$p_i = \frac{e^{x_i}}{\sum_{j=0}^{N-1} e^{x_j}}$$

Tuy nhiên, việc tính trực tiếp e^{x_i} có thể gây **overflow** khi x_i lớn. Vì vậy, trong thực tế thường sử dụng dạng Softmax ổn định số.

Đầu tiên xác định:

$$m = \max_j x_j$$

Sau đó tính:

$$p_i = \frac{e^{x_i - m}}{\sum_{j=0}^{N-1} e^{x_j - m}}$$

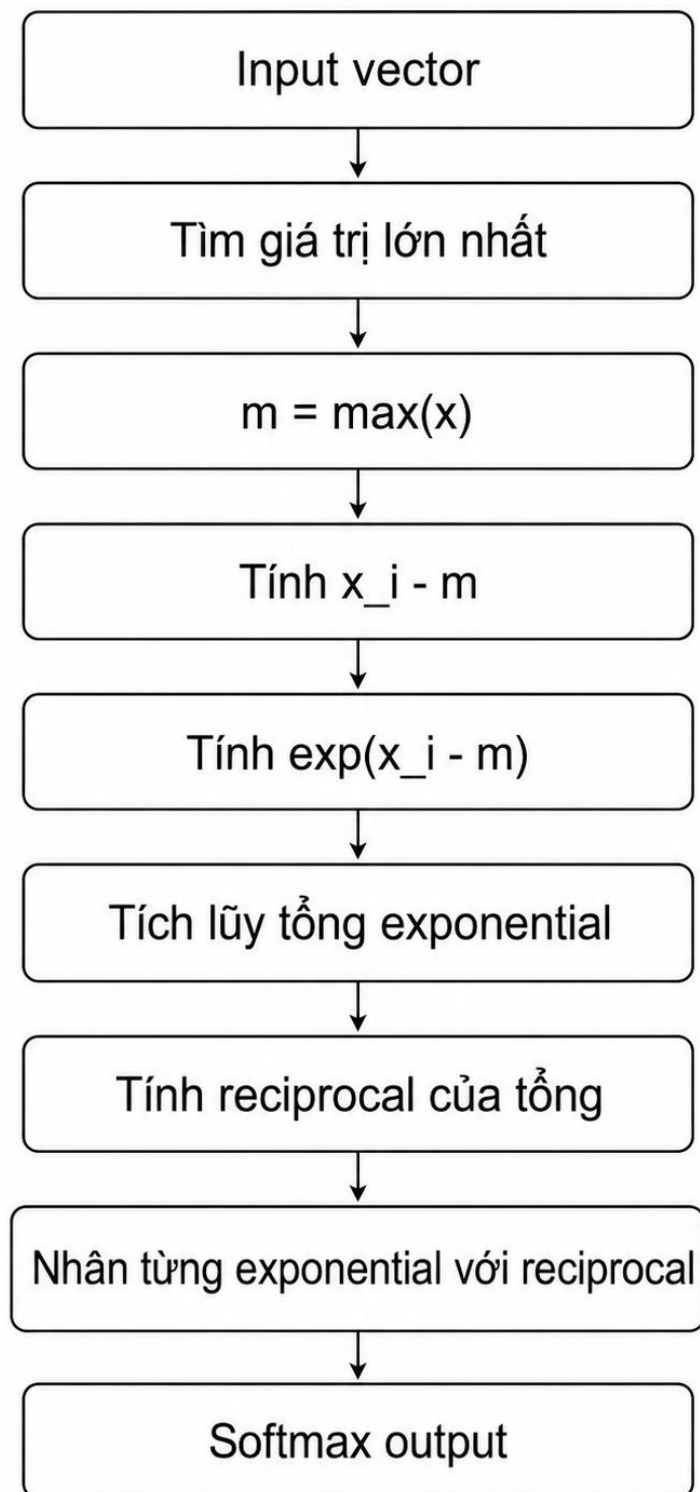
Do:

$$x_i - m \leq 0$$

nên giá trị exponential không vượt quá 1, giúp giảm nguy cơ overflow.

Trình tự tính toán

Stable Softmax có thể biểu diễn thành các giai đoạn:



Tương đương:

$$\begin{aligned} e_i &= x_i - m \\ S &= \sum_i e_i \\ r &= 1/S \\ p_i &= e_i r \end{aligned}$$

Trong Self-Attention của ViT-Base, Softmax thường được áp dụng trên từng hàng của attention score matrix:

$$[197 \times 197]$$

nghĩa là mỗi hàng gồm 197 attention score được chuẩn hóa độc lập.

Đặc điểm khi hiện thực số

Hàm exponential là một hàm phi tuyến tương đối phức tạp đối với phần cứng số. Vì vậy, khi triển khai accelerator, phép ex có thể được chuyển đổi thành một chuỗi các phép toán số học đơn giản hơn như:

- range reduction;
- nhân và cộng;
- xấp xỉ đa thức;
- scaling theo lũy thừa của 2.

Tương tự, phép chia:

$$1/S$$

có thể được chuyển thành bài toán tính reciprocal và sau đó nhân:

$$pi = ei \times reciprocal(S)$$

Mục này chỉ mô tả yêu cầu thuật toán; phương pháp xấp xỉ exponential và reciprocal cụ thể sẽ được trình bày trong phần thiết kế khối Softmax.

Yêu cầu tính toán đối với phần cứng

Softmax yêu cầu:

- phép so sánh để tìm maximum;
- phép trừ;
- exponential hoặc phép xấp xỉ tương đương;
- phép tích lũy;
- reciprocal;
- phép nhân chuẩn hóa.

Ngoài ra, dữ liệu exponential thường cần được giữ lại cho tới khi tổng S được xác định, sau đó mới có thể tạo output cuối cùng.

2.2.4 GELU

Gaussian Error Linear Unit (GELU) là hàm kích hoạt phi tuyến được sử dụng giữa hai lớp fully connected của MLP trong Vision Transformer.

Khác với ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

GELU tạo một hàm chuyển tiếp mềm hơn.

Dạng lý thuyết của GELU là:

$$\text{GELU}(x) = x \Phi(x)$$

trong đó $\Phi(x)$ là hàm phân phối tích lũy của phân phối chuẩn.

Có thể biểu diễn:

$$\text{GELU}(x) = (x/2) [1 + \text{erf}(x/\sqrt{2})]$$

Trong khối MLP của ViT-Base, GELU được áp dụng lên tensor:

$$[197 \times 3072]$$

sau lớp fully connected thứ nhất:

$$[197 \times 768] \rightarrow [197 \times 3072] \rightarrow \text{GELU}$$

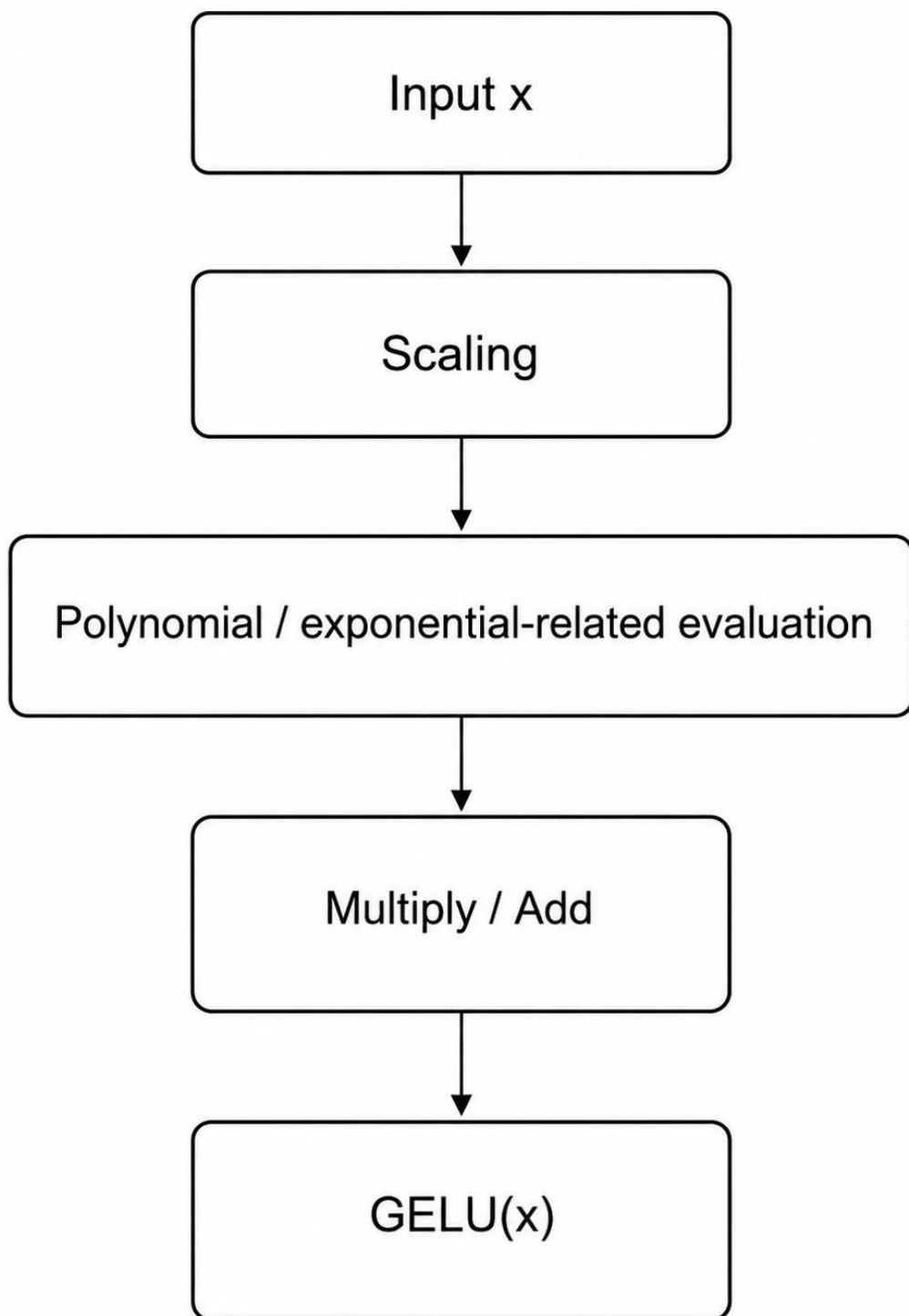
Đặc điểm tính toán

Không giống residual add hay phép scale đơn giản, GELU chứa các hàm phi tuyến như:

$$\text{erf}(x)$$

hoặc các biểu diễn tương đương dựa trên exponential/polynomial.

Việc hiện thực trực tiếp hàm erf trong phần cứng có chi phí cao. Vì vậy, khi chuyển sang thiết kế số, GELU thường được biến đổi hoặc xấp xỉ bằng một chuỗi các primitive đơn giản hơn:



Các primitive chủ yếu bao gồm:

- FP32 multiply;
- FP32 add/subtract;
- reciprocal;

- polynomial evaluation;
- exponential-related calculation.

Điều này cho phép GELU chia sẻ một số primitive số học với các phép toán Non-GEMM khác thay vì yêu cầu một cấu trúc toán học hoàn toàn độc lập.

Yêu cầu tính toán đối với phần cứng

Khối GELU cần:

- hỗ trợ số dấu và cả vùng giá trị âm/dương;
- duy trì độ chính xác phù hợp với FP32 intermediate data;
- hỗ trợ chuỗi phép nhân và cộng liên tiếp;
- có phương pháp xấp xỉ hàm phi tuyến phù hợp với tài nguyên FPGA.

Chi tiết cách biến đổi GELU thành các bước số học cụ thể sẽ được trình bày ở phần kiến trúc khối GELU.

2.2.5 Vector operation và layout transformation

Ngoài GEMM và các hàm phi tuyến, ViT còn chứa nhiều phép toán có độ phức tạp thấp hơn nhưng xuất hiện thường xuyên. Trong thiết kế accelerator, có thể chia chúng thành hai nhóm chính: **vector operation** và **layout transformation**.

a) Vector Operation

Vector operation thực hiện cùng một phép toán trên nhiều phần tử của tensor một cách độc lập.

Một ví dụ quan trọng là **residual addition**:

$$Y_i = A_i + B_i$$

Hai tensor phải có cùng kích thước. Trong Transformer Encoder, phép cộng residual xuất hiện sau nhánh Attention:

$$Y = X + \text{MSA}(\text{LN}(X))$$

và sau nhánh MLP:

$$Z = Y + \text{MLP}(\text{LN}(Y))$$

Ngoài phép cộng, một số giai đoạn của ViT cần phép scale:

$$Y_i = \alpha X_i$$

Ví dụ attention score được scale bởi:

$$d_{head1}$$

Với ViT-Base:

$$d_{head}=64$$

nên:

$$1/\text{can}(64)=81$$

Một dạng khác là scale kết hợp với mask hoặc phép cộng:

$$Y_i=\alpha X_i+B_i$$

Như vậy, các vector primitive cơ bản cần hỗ trợ gồm:

ADD:

$$Y = A + B$$

SCALE:

$$Y = \alpha A$$

SCALE + ADD/MASK:

$$Y = \alpha A + B$$

Đặc điểm của các phép toán này là không có tích lũy theo chiều K như GEMM. Mỗi phần tử output chỉ phụ thuộc vào một hoặc một số ít phần tử input tương ứng.

b) Layout Transformation

Không phải mọi bước trong ViT đều làm thay đổi giá trị số học của tensor. Một số bước chỉ thay đổi **cách tổ chức và ánh xạ chỉ số dữ liệu**.

Ví dụ, tensor:

$$Q \in \mathbb{R}^{197 \times 768}$$

cần được biểu diễn thành 12 attention head:

$$Q \rightarrow Q_1, Q_2, \dots, Q_{12}$$

với:

$$Q_i \in \mathbb{R}^{197 \times 64}$$

Về mặt số học, giá trị của từng phần tử không thay đổi. Thay đổi nằm ở cách ánh xạ:

$$(\text{token}, \text{feature})$$

sang:

(head,token,head feature)

Tương tự, sau khi hoàn thành attention cho 12 head, các kết quả cần được ghép lại:

$$12 \times [197 \times 64] \rightarrow [197 \times 768]$$

Các operation thuộc nhóm này có thể bao gồm:

- split tensor;
- merge tensor;
- transpose;
- reorder;
- reshape-like transformation;
- strided copy.

Về mặt tổng quát, layout transformation có thể được xem là ánh xạ:

source index $\rightarrow$ destination index

mà không nhất thiết thay đổi giá trị bit của dữ liệu.

Yêu cầu tính toán đối với phần cứng

Vector operation yêu cầu chủ yếu:

- FP32 addition;
- multiplication với scalar;
- mask;
- xử lý dữ liệu theo từng phần tử.

Trong khi đó, Layout Transformation chủ yếu yêu cầu:

- tính toán địa chỉ;
- stride;
- bộ đếm index;
- đọc dữ liệu tại địa chỉ nguồn;
- ghi nguyên giá trị đó tới địa chỉ đích.

Việc tách hai nhóm này khỏi GEMM giúp tránh sử dụng khối nhân ma trận lớn cho những phép toán đơn giản hoặc chỉ liên quan đến việc tổ chức dữ liệu.

2.2.6 Argmax

Ở cuối mô hình ViT, classifier tạo ra vector logits gồm 1000 phần tử đối với ImageNet-1K:

$$L=[l_0, l_1, \dots, l_{999}]$$

Mỗi logit thể hiện mức điểm mà mô hình gán cho một lớp.

Để xác định lớp dự đoán Top-1, chỉ cần tìm chỉ số của phần tử có giá trị lớn nhất:

$$c^* = \operatorname{argmax}_i(l_i)$$

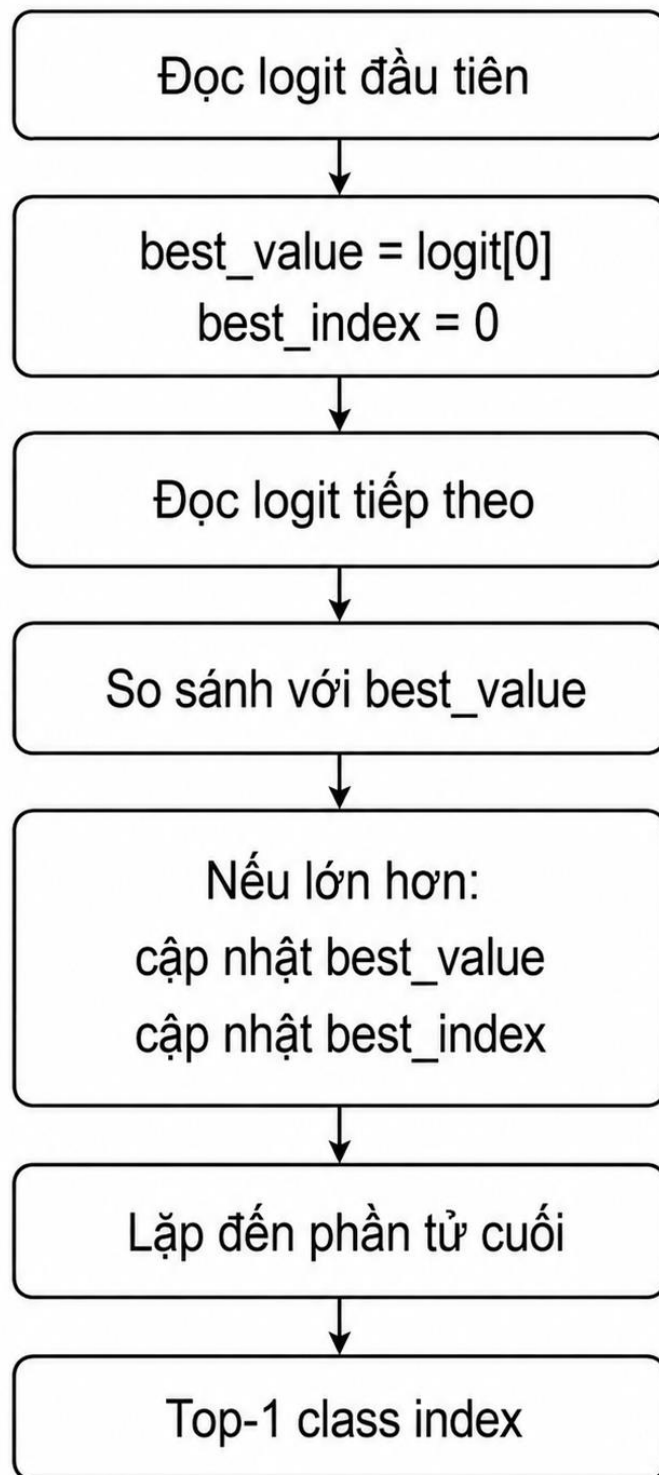
Trong đó:

$$c^* \in \{0, 1, \dots, 999\}$$

là class index cuối cùng.

Trình tự tính toán

Argmax có thể được thực hiện tuần tự:



Thuật toán chỉ cần duy trì hai giá trị: **best_value** và: **best_index**

Tại mỗi bước: **if xi>best_value**

thì: **best_value=xi best_index=i**

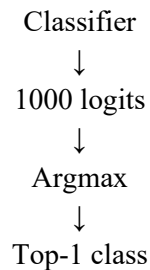
Quan hệ giữa Argmax và Softmax

Một đặc điểm quan trọng là Softmax là một hàm đơn điệu theo logits. Vì vậy:

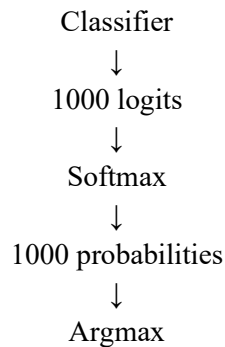
$$\text{argimax}_i = \text{argimax}_{\text{Softmax}(l)} i$$

Do đó, nếu mục tiêu chỉ là xác định **Top-1 classification**, không bắt buộc phải thực hiện Softmax trên 1000 logits trước Argmax.

Có thể sử dụng trực tiếp:



thay vì:



Softmax ở đầu ra chỉ cần thiết nếu hệ thống yêu cầu xác suất của từng lớp.

Yêu cầu tính toán đối với phần cứng

Argmax là một trong những primitive đơn giản hơn trong accelerator. Nó chủ yếu cần:

- bộ so sánh FP32;
- thanh ghi lưu giá trị lớn nhất;
- thanh ghi lưu chỉ số;
- bộ đếm phần tử;
- xử lý các giá trị số đặc biệt nếu cần.

Không cần multiplier hoặc adder FP32 cho phép toán chính.

2.3 Biểu diễn số và tính toán mixed-precision

2.3.1 FP16, FP32 và chuyển đổi kiểu dữ liệu

Trong chuẩn IEEE 754, số dấu phẩy động binary32 (FP32) gồm 1 bit dấu, 8 bit số mũ và 23 bit phần trị; binary16 (FP16) gồm 1 bit dấu, 5 bit số mũ và 10 bit phần trị. FP32 vì vậy có miền biểu diễn và độ chính xác phần trị lớn hơn FP16, trong khi FP16 giảm một nửa dung lượng lưu trữ của mỗi phần tử. Với Vision Transformer, lựa chọn kiểu dữ liệu không chỉ ảnh hưởng đến độ chính xác của phép nhân ma trận mà còn chi phối lưu lượng đọc trọng số, dung lượng DDR và tài nguyên phần cứng cho datapath.

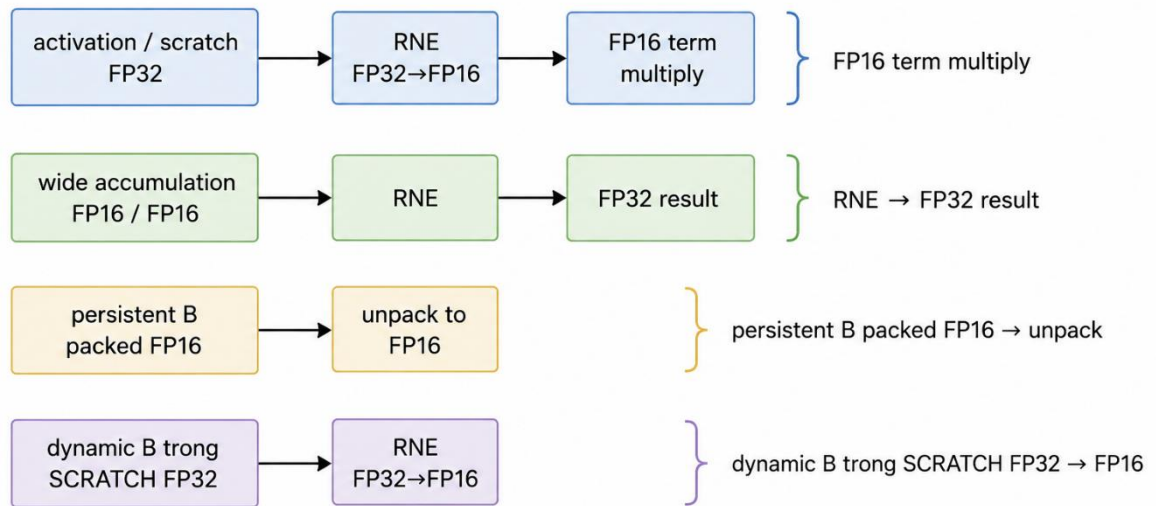
Thiết kế M8 không phải là hệ thống “toàn FP16” cũng không phải là hệ thống “toàn FP32”. Mixed-precision ở đây được hiểu theo ba lớp: **kiểu lưu trữ trong package/DDR, kiểu của toán hạng khi đi vào GEMM**, và **kiểu của tensor trung gian sau khi tính**. Phân biệt ba lớp này tránh một nhầm lẫn thường gặp: một activation có thể được lưu ở FP32 trong SCRATCH, được đổi sang FP16 chỉ khi trở thành một term của GEMM, sau đó kết quả dot product lại được xuất về FP32.

Dữ liệu hoặc đường xử lý	Biểu diễn trong M8	Ý nghĩa triển khai
prepared_input, activation, residual, scratch và các ma trận attention động	FP32	Được lưu/trao đổi dưới dạng word 32 bit; các tensor QK và PV động không dùng packed persistent-weight mode.
Persistent WEIGHT_B của GEMM	FP16, đóng gói hai half trong một word 32 bit	Trọng số B được chuyển một lần khi tạo package; layout là K16/N2 để phù hợp với cách cấp panel của GEMM.
Bias GEMM, gamma/beta LayerNorm và các tensor model không thuộc WEIGHT_B	FP32	Giữ nguyên trên đường FP32 để dùng cho bias, affine và các engine non-GEMM.
Toán hạng term của GEMM	FP16	Activation FP32 và B động FP32 được đổi sang FP16 tại GEMM ingress; persistent B đã packed FP16 được giải nén half trực tiếp.
Kết quả dot product, output GEMM và tensor chuyển giữa các engine	FP32	Kết quả được chuẩn hóa về FP32 trước khi ghi vào SCRATCH hoặc dùng cho bước sau.
LayerNorm, Softmax, GELU, Vector và Argmax	FP32	Các engine non-GEMM không chạy ở FP16; Layout chỉ sao chép nguyên bit

		của word 32 bit nên không tạo phép ép kiểu.
--	--	---

Trong package mixed-precision của dự án, các tensor persistent WEIGHT_B được đổi từ FP32 sang binary16 và sắp xếp theo block K16/N2. Hai giá trị FP16 của hai cột logic được đặt trong một container u32; cách đóng gói này là quyết định về **lưu trữ và cấp phát dữ liệu**, không làm cho toàn bộ đường dữ liệu trở thành FP16. Các tensor còn lại, bao gồm input đã chuẩn bị, vẫn giữ word FP32. Vì vậy, việc giảm dung lượng chủ yếu tập trung vào trọng số B được đọc lặp lại nhiều lần bởi GEMM, trong khi các tensor cần dùng cho residual, normalization và attention động vẫn có độ phân giải FP32.

Ở thời điểm thực thi, đường kiểu dữ liệu của một GEMM điển hình có thể tóm tắt như sau:



M8 dùng tám dot-product stream vật lý. Mỗi stream nhận các term FP16, nhưng không ghi kết quả FP16 ra bộ nhớ. Điểm này rất quan trọng khi mô tả kiến trúc: FP16 được dùng để giảm chi phí của term multiplication và lưu trữ persistent B; còn kết quả tích lũy và giao tiếp với các engine sau GEMM được đưa về FP32. Với các GEMM projection như Q, K, V, output projection, FC1, FC2 và classifier, B thường là persistent packed FP16. Ngược lại, QK^T và PV sử dụng A/B động trong SCRATCH; hai toán hạng vẫn bắt đầu ở FP32 và được chuyển sang FP16 ngay tại ingress.

Cần dùng thuật ngữ “datapath FP32 của M8” một cách chính xác. Các word FP32 tuân theo cách mã hóa floating-point 32 bit và các toán tử có quy tắc RNE, nhưng báo cáo không nên khẳng định toàn bộ datapath tương đương đầy đủ với mọi hành vi tùy chọn của IEEE 754 nếu chưa có chứng minh compliance. Cụ thể, arithmetic leaf của M8 xử lý một số trường hợp subnormal theo contract riêng: adder coi exponent bằng 0 là zero, còn multiplier flush subnormal đầu vào và underflow đầu ra thành signed zero. Đây là đặc tính cần nêu rõ vì nó ảnh hưởng trực tiếp đến cách xây dựng golden/reference và diễn giải sai khác số học.

2.3.2 Làm tròn, tích lũy và các điểm cần kiểm soát sai khác số học

Làm tròn là bước đưa một giá trị thực hoặc giá trị có độ chính xác cao hơn về tập giá trị biểu diễn được của một định dạng đích. M8 sử dụng quy tắc **round-to-nearest, ties-to-even (RNE)** tại các ranh giới định dạng chính: giá trị gần nhất được chọn; nếu hai giá trị lân cận cách đều, phần trị có bit thấp nhất chẵn được chọn. Quy tắc này hạn chế độ lệch có hệ thống khi một lượng lớn giá trị được làm tròn lặp lại. Ký hiệu $\mathcal{R}_{16}(\cdot)$ và $\mathcal{R}_{32}(\cdot)$ dưới đây lần lượt biểu thị việc làm tròn theo contract FP16 và FP32 của thiết kế.

Với một phần tử đầu ra GEMM, mô hình số học khái quát là:

$$\hat{a}_k = \mathcal{R}_{16}(a_k), \quad \hat{b}_k = \mathcal{R}_{16}(b_k)$$

$$\hat{y} = \mathcal{R}_{32} \left(\sum_{k=0}^{K-1} \hat{a}_k \hat{b}_k \right)$$

Trong đó:

- $\mathcal{R}_{16}(\cdot)$: làm tròn/chuyển dữ liệu sang FP16, ví dụ RNE.
- $\mathcal{R}_{32}(\cdot)$: làm tròn kết quả tích lũy về FP32.
- K : chiều tích lũy của phép nhân ma trận.
- $\hat{a}_k, \hat{b}_k$: toán hạng sau khi chuyển về FP16.
- $\hat{y}$: một phần tử đầu ra GEMM.

Biểu thức trên không nên bị hiểu là tích lũy tuần tự với một lần làm tròn FP32 sau từng hạng tử. Đó là điểm khác biệt quan trọng của M8. Dot stream nhân các term FP16, biểu diễn product hữu hạn thành addend fixed-point có căn chỉnh số mũ, rồi tích lũy bằng carry-save accumulator rộng 93 bit. Chỉ khi gặp term cuối của dot product, tổng sum + carry mới được carry-propagate, phân tích và đóng gói về binary32 bằng RNE. Nhờ vậy, phần tích lũy của một dot product không bị mất bit do làm tròn FP32 lặp lại theo từng term; sai khác chính tại GEMM đến từ việc lượng tử hóa toán hạng sang FP16, các quy tắc special-value và lần chuẩn hóa kết quả cuối.

Các điểm làm tròn và biến đổi số học cần được ghi rõ trong báo cáo được tổng hợp ở Bảng 2.5.

Vị trí	Hành vi M8	Ý nghĩa đối với sai khác số học

Tạo package persistent B	FP32 → FP16 theo RNE, ties-to-even; gradual underflow, signed zero và qNaN chuẩn hóa được xử lý ở bước đóng gói	Đây là sai số lượng tử hóa cố định của trọng số persistent; không phát sinh lại mỗi lần đọc trọng số đó.
GEMM ingress	Activation FP32 và B động FP32 được đổi sang FP16 theo RNE; B packed FP16 đi thẳng vào stream, không FP16→FP32→FP16	Cần kiểm soát đúng boundary và thứ tự packed-half; QK ^T /PV có sai khác do cả hai toán hạng động đều đi qua boundary này.
Dot-product stream	FP16 term multiply → căn chỉnh → CSA 93 bit → RNE binary32 ở cuối dot product	Không mô tả đây là “FP32 MAC mỗi chu kỳ”; kết quả FP32 chỉ xuất hiện sau pha finalize.
Bias và output GEMM	Bias/output đi trên đường FP32 sau khi kết quả dot được thu nhận	Sai khác bias phải được tách khỏi sai số do lượng tử hóa term FP16.
Vector, LayerNorm, Softmax và GELU	Các phép add/mul FP32 thực hiện theo từng state riêng, không dùng fused multiply-add	Mỗi phép toán là một điểm làm tròn FP32; thứ tự thao tác của RTL có ý nghĩa.
Layout và bộ nhớ đệm cục bộ	Layout copy nguyên bit; BRAM LayerNorm/Softmax giữ word FP32 đã đọc hoặc đã tính	Di chuyển qua DDR/BRAM không tự tạo rounding; chỉ có phép toán hoặc converter mới làm đổi giá trị.

Đối với non-GEMM, sai khác số học không chỉ đến từ precision FP16. LayerNorm của M8 tính tổng và phương sai theo thứ tự tuần tự FP32, dùng biased variance với mẫu số H, rồi áp dụng epsilon và phép xấp xỉ reciprocal-square-root. Mỗi phép cộng hoặc nhân là một state riêng, do đó mỗi state tạo một rounding point FP32; không thể thay nó bằng một công thức có fused multiply-add rồi kỳ vọng vector trùng bit. `inv_std` được tạo từ seed và ba vòng lặp Newton, nên đây là một phép xấp xỉ được xác định bởi đúng hằng số, thứ tự phép toán và số vòng lặp của RTL.

Softmax cũng cần được diễn giải theo implementation thay vì chỉ theo công thức toán học. Engine trước hết lấy max của từng hàng để ổn định số, sau đó tính $\exp(x_i - \max)$ bằng range reduction và đa thức Horner bậc 8, cộng dồn tuần tự theo FP32, rồi lấy reciprocal bằng seed và bốn vòng lặp Newton trước khi normalize. GELU sử dụng approximant kiểu erf với chuỗi reciprocal, đa thức và exponential xấp xỉ. Vì vậy, “bit-exact” của các khối này chỉ nên được hiểu là trùng với reference RTL cùng thứ tự operation/rounding; nó không đồng nghĩa với việc bằng chính xác hàm exp, erf hoặc GELU toán học lý tưởng.

Vector engine là ví dụ rõ về thứ tự làm tròn. Với ADD, thiết kế thực hiện một phép FP32_ADD(a, b). Với SCALE_MASK, thiết kế thực hiện $s = \text{FP32_MUL}(a, \text{scalar})$, sau đó mới thực hiện $y = \text{FP32_ADD}(s, b)$ nếu mask được bật. Hai phép này có hai rounding point riêng và không được thay

bằng FMA trong golden model. Layout không có rounding vì chỉ thay đổi địa chỉ/stride; Argmax dùng so sánh số FP32, chính sách > nghiêm ngặt và giữ index nhỏ hơn khi các logit bằng nhau.

Để kiểm soát sai khác số học, báo cáo cần tách ba mức kiểm tra thay vì chỉ nhìn vào nhân cuối cùng:

1. **Kiểm tra boundary dữ liệu.** Xác minh package, descriptor GEMM_FP16, layout K16/N2, thứ tự packed-half và converter FP32→FP16. Bước này giúp phân biệt lỗi đóng gói/địa chỉ với sai số arithmetic.
2. **Kiểm tra checkpoint tensor.** So sánh tensor sau các ranh giới quan trọng như projection, QK^T , Softmax, PV, LayerNorm, GELU và logits để tìm layer đầu tiên phân kỳ. Các chỉ số tối thiểu gồm sai số tuyệt đối lớn nhất, sai số tương đối có mẫu số bảo vệ và số phần tử vượt ngưỡng đã công bố trước.
3. **Kiểm tra special values.** Có test riêng cho NaN, $\pm\text{Inf}$, ± 0 , subnormal, overflow/underflow và tail/length boundary. GEMM có các cờ invalid, overflow, alignment/length và subnormal-flushed; nhiều engine non-GEMM lại chuyển special case thành word FP32 mà không có cờ numerical-error riêng, nên trạng thái done không đủ để kết luận numerical pass.

Cuối cùng, cần báo cáo độc lập ba khái niệm: job phân cứng hoàn tất không lỗi giao thức, nhân Top-1 có khớp nhãn tham chiếu hay không, và vector logits/probability có đạt tiêu chí numerical comparison hay không. Một nhãn Top-1 đúng không chứng minh toàn bộ vector số học tương đương; ngược lại, một sai khác vector nhỏ cũng cần được truy vết về đúng rounding boundary, approximant hoặc thứ tự tích lũy trước khi kết luận nguyên nhân.

2.4 FPGA SoC, DDR và AXI

2.4.1 Phân hoạch PS–PL

Nền tảng Genesys ZU-5EV sử dụng SoC FPGA thuộc họ Xilinx Zynq UltraScale+ MPSoC, trong đó hệ thống được chia thành hai miền phần cứng chính là **Processing System (PS)** và **Programmable Logic (PL)**. Cách tổ chức này cho phép kết hợp khả năng xử lý linh hoạt của bộ xử lý ARM với khả năng thực hiện song song và tùy biến cao của logic FPGA trên cùng một chip.

a) Processing System

Processing System (PS) là phần xử lý cố định được tích hợp sẵn trong Zynq UltraScale+ MPSoC. Như minh họa trong hình, PS bao gồm nhiều thành phần như bộ xử lý ARM, bộ điều khiển bộ nhớ và các peripheral hệ thống.

Trong phạm vi đề tài, thành phần quan trọng nhất là **Application Processing Unit (APU)** sử dụng các lõi **ARM Cortex-A53**. CPU được sử dụng để thực hiện các chức năng điều khiển cấp hệ thống như:

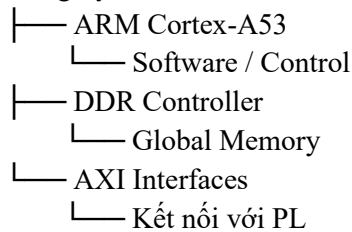
- khởi tạo hệ thống và accelerator;
- cấu hình các thanh ghi điều khiển;
- cung cấp thông tin về vùng nhớ và tham số thực thi;
- khởi động một tác vụ inference;
- theo dõi trạng thái hoàn tất hoặc lỗi;

- phối hợp với DDR và các peripheral của hệ thống.

PS cũng chứa bộ điều khiển **DDR**, cho phép sử dụng bộ nhớ ngoài làm vùng lưu trữ dung lượng lớn cho trọng số mô hình, ảnh đầu vào và các tensor trung gian.

Có thể xem vai trò của PS trong hệ thống như sau:

Processing System



PS phù hợp với các tác vụ có tính điều khiển, tuần tự và cần khả năng lập trình bằng phần mềm.

b) Programmable Logic

Programmable Logic (PL) là vùng logic có thể cấu hình của FPGA. PL cung cấp các tài nguyên như:

- LUT và Flip-Flop;
- Block RAM;
- UltraRAM;
- DSP;
- các tài nguyên interconnect và I/O.

Khác với PS, cấu trúc bên trong PL không cố định mà được người thiết kế mô tả bằng HDL và tổng hợp thành phần cứng chuyên dụng.

Trong đề tài này, **ViT Accelerator được hiện thực trong PL**. Accelerator đảm nhiệm các phép toán có khối lượng tính toán lớn của Vision Transformer, bao gồm:

- GEMM;
- LayerNorm;
- Softmax;
- GELU;
- các phép vector;
- biến đổi layout;
- Argmax.

Mặc dù PL của Zynq UltraScale+ có sẵn các DSP slice, thiết kế trong đề tài đặt ra ràng buộc:

DSP usage=0

Do đó, các phép nhân và các primitive số học của accelerator được ánh xạ chủ yếu lên LUT, carry logic và các tài nguyên logic khả trình khác.

c) Nguyên tắc phân hoạch PS–PL

Mục tiêu của phân hoạch PS–PL là đưa từng loại công việc về phần cứng phù hợp nhất.

Các tác vụ điều khiển và phần mềm được giữ ở PS, trong khi các phép toán chuyên dụng có mức độ lặp lại cao được chuyển sang PL.

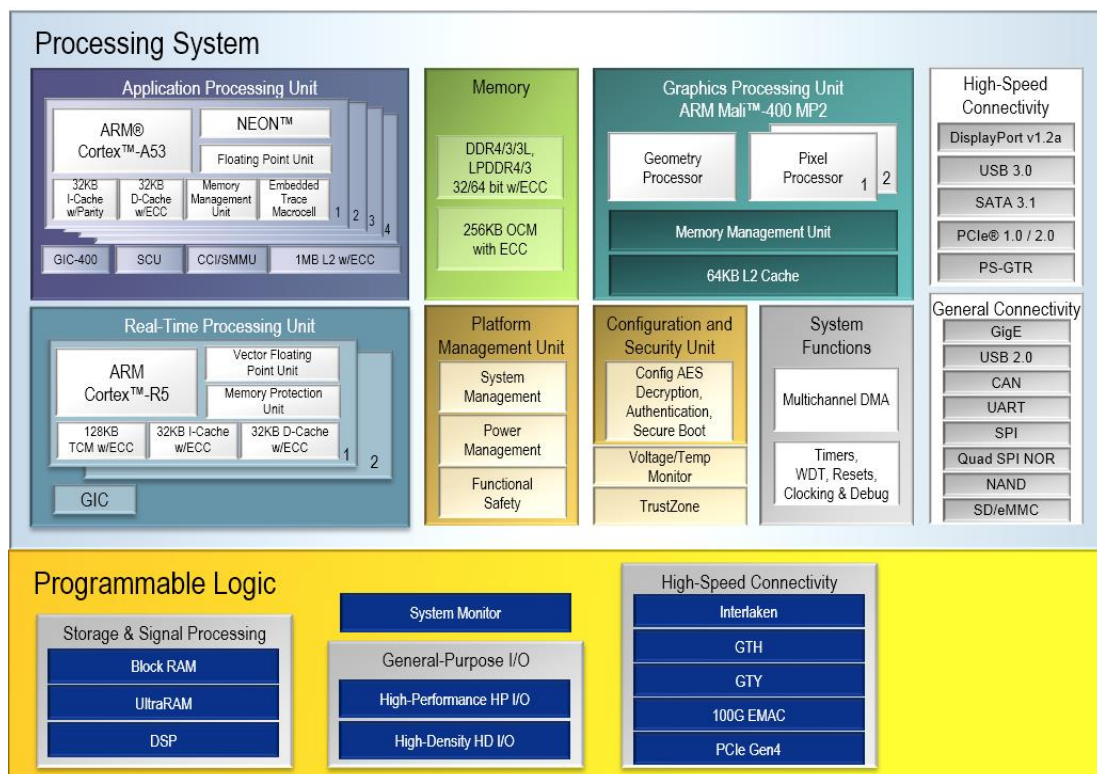
Có thể tổng hợp như sau:

Thành phần	Nhiệm vụ chính
PS – ARM Cortex-A53	Khởi tạo và điều khiển hệ thống
PS – DDR Controller	Quản lý bộ nhớ DDR ngoài
PS – AXI Infrastructure	Kết nối PS với PL
PL – ViT Accelerator	Thực thi các kernel của ViT
PL – Local memory/buffer	Lưu dữ liệu tạm và tăng khả năng tái sử dụng
PL – Control logic	Điều phối các engine phần cứng

Trong hệ thống ViT của đề tài, CPU không trực tiếp thực hiện từng phép nhân ma trận. Thay vào đó, PS cấu hình accelerator và cung cấp thông tin cần thiết để bắt đầu một job. Sau đó các khối phần cứng trong PL tự điều phối quá trình thực thi.

Luồng tổng quát có thể biểu diễn:

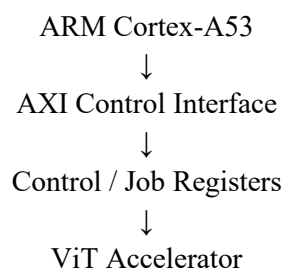
Zynq UltraScale+ MPSoC



d) Control path và data path

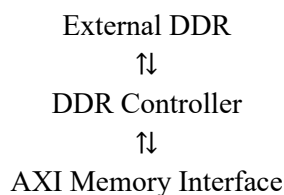
Trong hệ thống SoC, cần phân biệt hai loại luồng thông tin.

Control path được sử dụng để CPU cấu hình accelerator:



Lượng dữ liệu trên đường này tương đối nhỏ, chủ yếu là các thanh ghi cấu hình, lệnh bắt đầu và trạng thái hoạt động.

Ngược lại, **data path** được sử dụng để truyền các tensor và trọng số có dung lượng lớn:



↕
ViT Accelerator

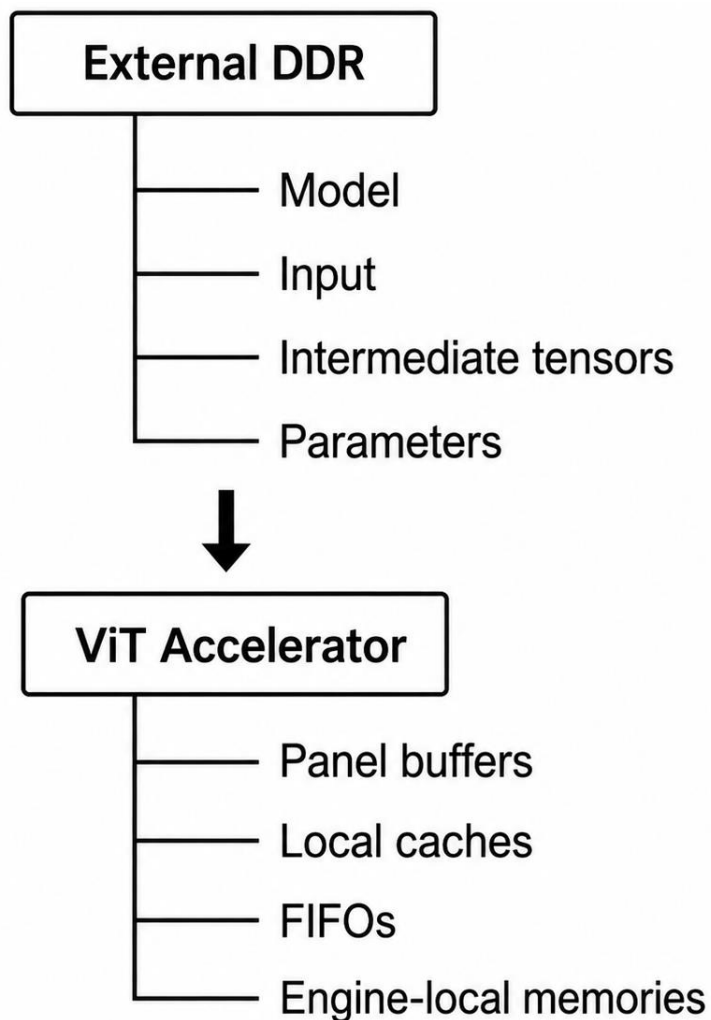
Việc tách control path và data path giúp CPU không phải trực tiếp chuyển từng phần tử tensor vào accelerator.

e) Vai trò của DDR trong phân hoạch PS–PL

Một mô hình ViT có lượng trọng số và dữ liệu trung gian lớn nên không thể lưu toàn bộ trong tài nguyên BRAM của PL.

Vì vậy, hệ thống sử dụng DDR làm **global memory**, trong khi các bộ nhớ bên trong PL chỉ đóng vai trò bộ đệm cục bộ.

Có thể xem cấu trúc bộ nhớ theo hai tầng:



Việc tổ chức này giúp accelerator truy cập lượng dữ liệu lớn từ DDR nhưng vẫn có khả năng tái sử dụng dữ liệu cục bộ để giảm số lần truy cập bộ nhớ ngoài.

f) Phân hoạch PS–PL trong đề tài

Đối với hệ thống được xây dựng trong đề tài, việc phân chia nhiệm vụ có thể tóm tắt như sau:

PS: System Control + DDR PL: ViT Computation

CPU Cortex-A53 trong PS chịu trách nhiệm quản lý và khởi tạo hệ thống, trong khi ViT Accelerator được triển khai trong PL để thực hiện các phép toán của mô hình. Hai miền trao đổi thông tin thông qua giao tiếp AXI, còn DDR đóng vai trò bộ nhớ chung cho dữ liệu mô hình.

Cách phân hoạch này cho phép tận dụng đồng thời hai ưu điểm: **tính linh hoạt của phần mềm trên PS** và **khả năng tùy biến, song song hóa của phần cứng trên PL**.

2.4.2 AXI4-Lite và AXI memory-mapped

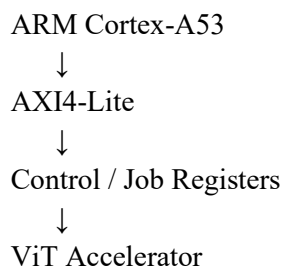
Trong hệ thống SoC FPGA, giao tiếp giữa Processing System (PS), Programmable Logic (PL) và bộ nhớ thường được thực hiện thông qua chuẩn **AMBA AXI (Advanced eXtensible Interface)**. Trong phạm vi hệ thống ViT Accelerator, hai dạng giao tiếp quan trọng nhất là **AXI4-Lite** và **AXI4 Memory-Mapped**.

Hai giao tiếp này đều sử dụng mô hình ánh xạ bộ nhớ, tuy nhiên chúng phục vụ hai mục đích khác nhau: AXI4-Lite phù hợp với truyền các giá trị điều khiển có kích thước nhỏ, trong khi AXI4 Memory-Mapped phù hợp với truyền lượng dữ liệu lớn như trọng số và tensor.

a) AXI4-Lite

AXI4-Lite là phiên bản đơn giản hóa của AXI4, thường được sử dụng để truy cập các thanh ghi điều khiển và trạng thái của một IP phần cứng.

Trong hệ thống ViT Accelerator, AXI4-Lite có thể được sử dụng cho luồng:



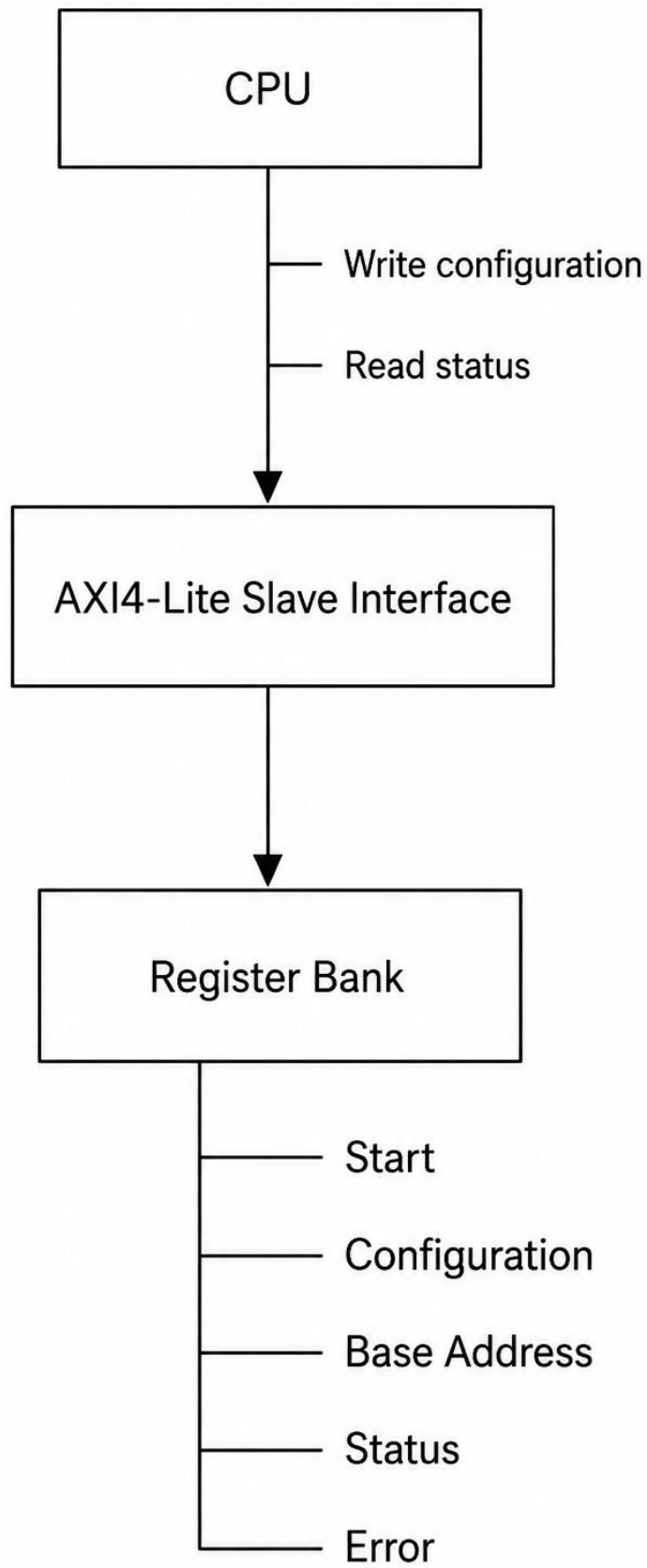
Thông qua các thanh ghi này, CPU có thể thực hiện các thao tác như:

- ghi cấu hình cho accelerator;
- cung cấp địa chỉ cơ sở của các vùng dữ liệu;

- thiết lập chế độ hoạt động;
- phát tín hiệu bắt đầu một job;
- đọc trạng thái busy, done hoặc error;
- đọc các thông tin kết quả hoặc bộ đếm hiệu năng nếu cần.

AXI4-Lite không hỗ trợ burst transaction và thường chỉ truyền một word dữ liệu cho mỗi transaction. Do đó, giao tiếp này phù hợp với các dữ liệu điều khiển có dung lượng nhỏ nhưng không phù hợp để truyền các tensor lớn.

Có thể mô tả luồng điều khiển như sau:

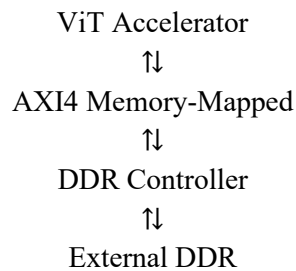


Trong kiến trúc này, ViT Accelerator đóng vai trò **AXI4-Lite slave**, còn CPU trong PS đóng vai trò master khi truy cập các thanh ghi điều khiển.

b) AXI4 Memory-Mapped

Đối với các dữ liệu có kích thước lớn như model weights, input image và intermediate tensors, hệ thống cần một giao tiếp có băng thông cao hơn. Vì vậy, **AXI4 Memory-Mapped** được sử dụng cho đường truyền dữ liệu giữa accelerator và DDR.

Luồng dữ liệu tổng quát có dạng:



Khác với AXI4-Lite, AXI4 Memory-Mapped hỗ trợ:

- burst transaction;
- nhiều beat dữ liệu trong một request;
- nhiều transaction đang chờ xử lý;
- các kênh đọc và ghi độc lập;
- khả năng truyền dữ liệu với băng thông cao.

Điều này phù hợp với các workload như GEMM, nơi accelerator cần đọc liên tục các block dữ liệu A, B, bias hoặc ghi các tensor kết quả.

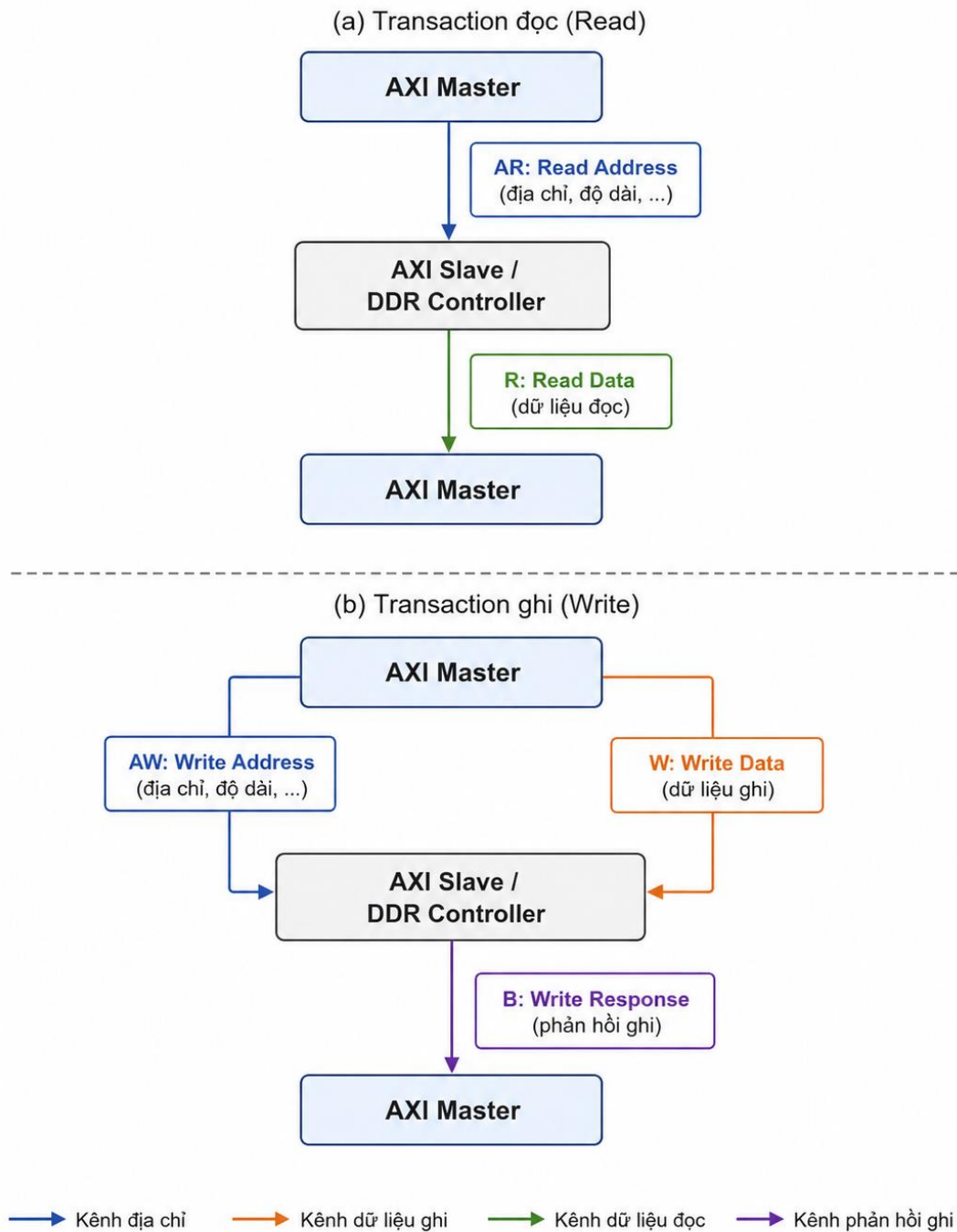
Trong cấu trúc này, accelerator thường đóng vai trò **AXI master**, tự phát sinh địa chỉ đọc/ghi đến DDR dựa trên command và thông tin địa chỉ đã được cấu hình.

c) Các kênh cơ bản của AXI4

AXI4 Memory-Mapped sử dụng năm kênh độc lập:

Kênh	Chức năng
AW – Write Address	Truyền địa chỉ ghi
W – Write Data	Truyền dữ liệu ghi

B – Write Response	Phản hồi kết quả ghi
AR – Read Address	Truyền địa chỉ đọc
R – Read Data	Truyền dữ liệu đọc

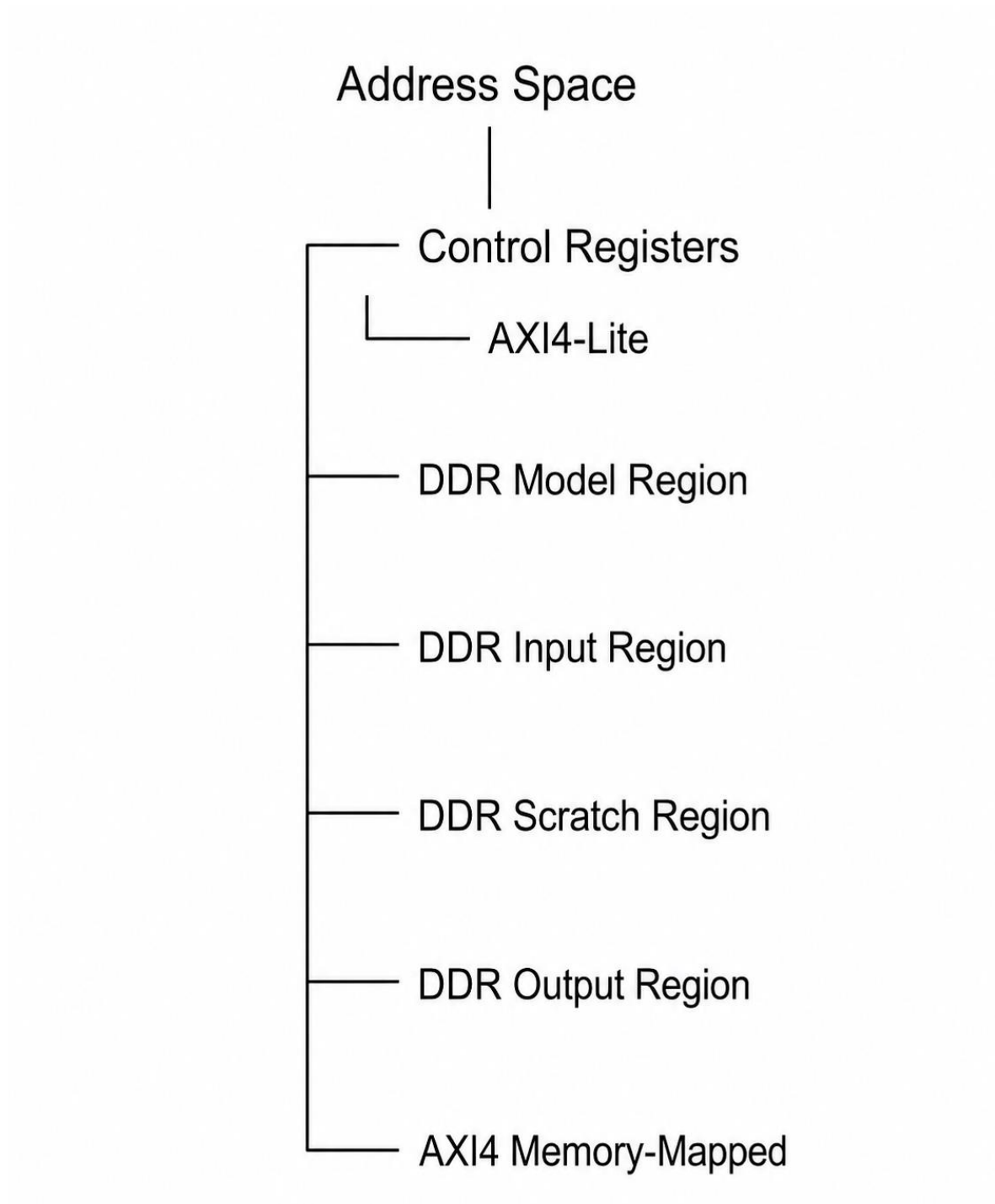


Việc tách các kênh địa chỉ, dữ liệu và response giúp giao thức AXI hỗ trợ pipeline và cho phép nhiều transaction được xử lý linh hoạt hơn.

d) Memory-Mapped Addressing

Cả AXI4-Lite và AXI4 Memory-Mapped đều sử dụng cơ chế **memory-mapped addressing**. Điều này có nghĩa là một peripheral hoặc vùng nhớ được gán một khoảng địa chỉ trong không gian địa chỉ của hệ thống.

Ví dụ ở mức khái niệm:



CPU có thể điều khiển accelerator bằng cách đọc hoặc ghi vào vùng thanh ghi tương ứng, trong khi accelerator có thể truy cập DDR bằng cách phát sinh địa chỉ memory-mapped đến vùng dữ liệu cần thiết.

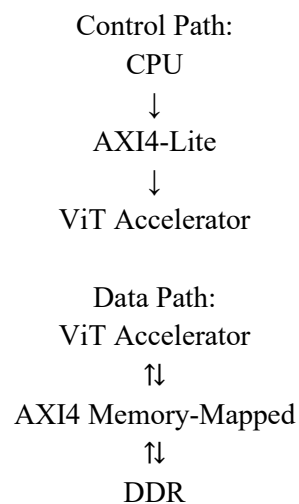
Cách tổ chức này giúp các thành phần trong SoC giao tiếp với nhau bằng một mô hình địa chỉ thống nhất.

e) Phân biệt AXI4-Lite và AXI4 Memory-Mapped

Hai giao tiếp có thể được tổng hợp như sau:

Đặc điểm	AXI4-Lite	AXI4 Memory-Mapped
Mục đích chính	Control / Status	Truyền dữ liệu lớn
Burst	Không	Có
Độ phức tạp	Thấp	Cao hơn
Bảng thông yêu cầu	Thấp	Cao
Dữ liệu điển hình	Register, mode, start, status	Weight, tensor, input, output
Vai trò trong ViT Accelerator	CPU điều khiển accelerator	Accelerator truy cập DDR

Trong hệ thống ViT, hai loại giao tiếp này bổ sung cho nhau:



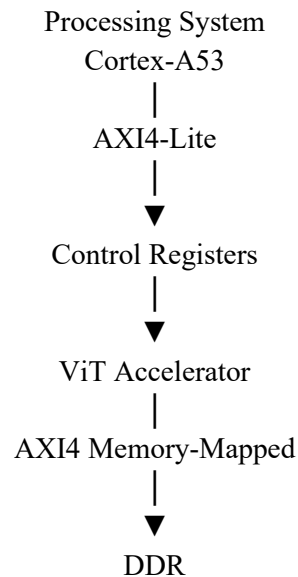
Việc tách hai luồng giúp tránh sử dụng đường điều khiển để vận chuyển lượng dữ liệu lớn, đồng thời cho phép accelerator truy cập bộ nhớ độc lập với CPU trong quá trình tính toán.

f) Liên hệ với thiết kế ViT Accelerator

Từ góc độ thiết kế hệ thống, có thể xem AXI4-Lite và AXI4 Memory-Mapped là hai lớp giao tiếp phục vụ hai nhiệm vụ khác nhau.

AXI4-Lite cung cấp **control plane**, dùng để cấu hình và khởi động accelerator. Trong khi đó, AXI4 Memory-Mapped cung cấp **data plane**, phục vụ quá trình đọc trọng số, activation và ghi kết quả.

Luồng tổng thể có thể biểu diễn:



Cách tổ chức này là cơ sở để xây dựng một SoC trong đó CPU đảm nhiệm điều khiển cấp cao, còn accelerator trong PL có khả năng tự truy cập bộ nhớ và thực hiện các phép toán ViT với mức can thiệp của CPU được giảm xuống.

2.4.3 VALID/READY, burst, FIFO và backpressure

Trong hệ thống AXI và các giao tiếp streaming nội bộ của accelerator, dữ liệu không được truyền chỉ dựa trên xung clock mà còn cần cơ chế xác nhận giữa phía gửi và phía nhận. Các khái niệm **VALID/READY**, **burst**, **FIFO** và **backpressure** là những thành phần quan trọng để đảm bảo dữ liệu được truyền đúng thứ tự, không bị mất và cho phép các khối có tốc độ xử lý khác nhau vẫn có thể hoạt động cùng nhau.

a) Cơ chế VALID/READY

AXI sử dụng cơ chế handshake dựa trên hai tín hiệu chính là VALID và READY.

- VALID = 1 cho biết phía gửi đã có dữ liệu hoặc thông tin hợp lệ.
- READY = 1 cho biết phía nhận đã sẵn sàng tiếp nhận dữ liệu.

- Một lần truyền dữ liệu chỉ thực sự xảy ra khi cả hai tín hiệu đều bằng 1 trong cùng một chu kỳ clock.

Có thể biểu diễn:

$$\text{transfer} = \text{VALID} \wedge \text{READY}$$

Transfer xảy ra khi:

$\text{VALID} = 1$ và $\text{READY} = 1$

Ví dụ, nếu producer có dữ liệu nhưng consumer chưa sẵn sàng:

$\text{VALID} = 1$

$\text{READY} = 0$

thì dữ liệu chưa được truyền. Producer phải giữ dữ liệu và VALID ổn định cho đến khi consumer đưa READY lên mức 1.

Ngược lại, nếu:

$\text{VALID} = 0$

$\text{READY} = 1$

thì consumer đang sẵn sàng nhưng producer chưa có dữ liệu, do đó cũng không có transaction xảy ra.

Có thể tóm tắt bằng bảng sau:

VALID	READY	Trạng thái
0	0	Không có dữ liệu và phía nhận chưa sẵn sàng
0	1	Phía nhận sẵn sàng nhưng chưa có dữ liệu
1	0	Có dữ liệu nhưng phía nhận chưa thể nhận
1	1	Dữ liệu được truyền

Cơ chế này được sử dụng không chỉ trong AXI mà còn rất phù hợp với các giao tiếp nội bộ giữa các khối phần cứng, chẳng hạn giữa memory subsystem và compute engine hoặc giữa compute engine và result path.

b) Burst Transfer

Đối với dữ liệu lớn như tensor hoặc trọng số mô hình, việc phát một request riêng cho từng word sẽ tạo ra nhiều overhead về địa chỉ và điều khiển. AXI4 vì vậy hỗ trợ **burst transaction**, cho phép nhiều beat dữ liệu được truyền liên tiếp sau một lần phát địa chỉ.

Ví dụ, thay vì:

Address 0 → Data 0

Address 1 → Data 1

Address 2 → Data 2

Address 3 → Data 3

với bốn request độc lập, burst cho phép:

Read Address



Data Beat 0

Data Beat 1

Data Beat 2

Data Beat 3

Một burst gồm một chuỗi các **beat**, trong đó mỗi beat tương ứng với một đơn vị dữ liệu trên bus.

Việc sử dụng burst đem lại một số lợi ích:

- giảm số lần gửi địa chỉ;
- giảm overhead giao thức;
- tăng hiệu quả sử dụng bus;
- phù hợp với việc đọc/ghi các vùng dữ liệu liên tục;
- hỗ trợ tốt các phép toán như GEMM, nơi nhiều operand nằm liên tiếp trong bộ nhớ.

Ở mức khái niệm, hiệu quả truyền dữ liệu có thể được cải thiện vì phần chi phí thiết lập transaction được chia sẻ cho nhiều beat dữ liệu thay vì lặp lại cho từng word.

c) FIFO

FIFO (First-In First-Out) là bộ đệm trong đó dữ liệu được đọc ra theo đúng thứ tự đã được ghi vào.

Một FIFO thường được đặt giữa hai khối có tốc độ xử lý hoặc latency khác nhau:

Producer



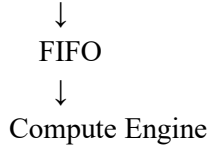
FIFO



Consumer

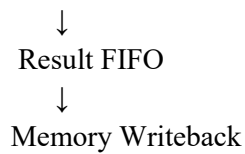
Ví dụ:

DDR / Memory Frontend



hoặc:

Compute Engine



FIFO giúp **tách rời producer và consumer**. Producer có thể tiếp tục ghi dữ liệu vào FIFO trong một khoảng thời gian ngay cả khi consumer tạm thời xử lý chậm hơn.

Các trạng thái quan trọng thường gồm: empty, full, read enable, write enable

Khi FIFO rỗng, consumer không thể đọc thêm dữ liệu. Khi FIFO đầy, producer phải tạm dừng việc ghi.

FIFO đặc biệt hữu ích để:

- hấp thụ chênh lệch tốc độ ngắn hạn giữa hai khối;
- giảm stall tức thời;
- che một phần latency;
- duy trì dòng dữ liệu ổn định hơn;
- tách timing giữa các khối.

Tuy nhiên, FIFO **không làm tăng bandwidth tối đa của hệ thống**. Nếu tốc độ cung cấp dữ liệu trung bình từ memory thấp hơn tốc độ compute yêu cầu trong thời gian dài, FIFO cuối cùng vẫn sẽ bị rỗng.

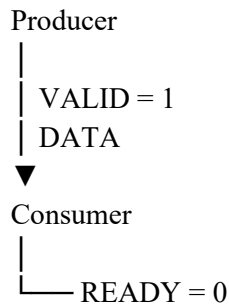
d) Backpressure

Backpressure là cơ chế cho phép phía nhận báo cho phía gửi rằng nó tạm thời không thể nhận thêm dữ liệu.

Trong giao thức VALID/READY, backpressure thường xuất hiện khi:

VALID=1,READY=0

Ví dụ:



Khi đó producer phải giữ dữ liệu và chờ.

Backpressure có thể xuất hiện do nhiều nguyên nhân như:

- FIFO phía sau đã đầy;
- compute engine đang bận;
- memory subsystem chưa hoàn thành request trước;
- result path chưa thể ghi dữ liệu;
- downstream pipeline đang bị stall.

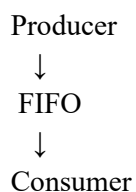
Điểm quan trọng là:

Backpressure không phải là lỗi giao thức mà là một cơ chế flow control bình thường nhằm tránh mất dữ liệu khi các khối hoạt động ở tốc độ khác nhau.

e) Quan hệ giữa FIFO và Backpressure

FIFO và backpressure thường được sử dụng cùng nhau.

Ví dụ:



Nếu consumer tạm thời xử lý chậm:

FIFO occupancy tăng

Producer vẫn có thể tiếp tục gửi dữ liệu cho đến khi:

FIFO = FULL

Khi đó FIFO hạ READY:

READY = 0

và tạo backpressure ngược về producer.

Có thể biểu diễn:

Consumer chậm



FIFO bắt đầu đầy



FIFO FULL



READY = 0



Backpressure



Producer tạm dừng

Khi consumer xử lý bớt dữ liệu:

FIFO không còn full



READY = 1



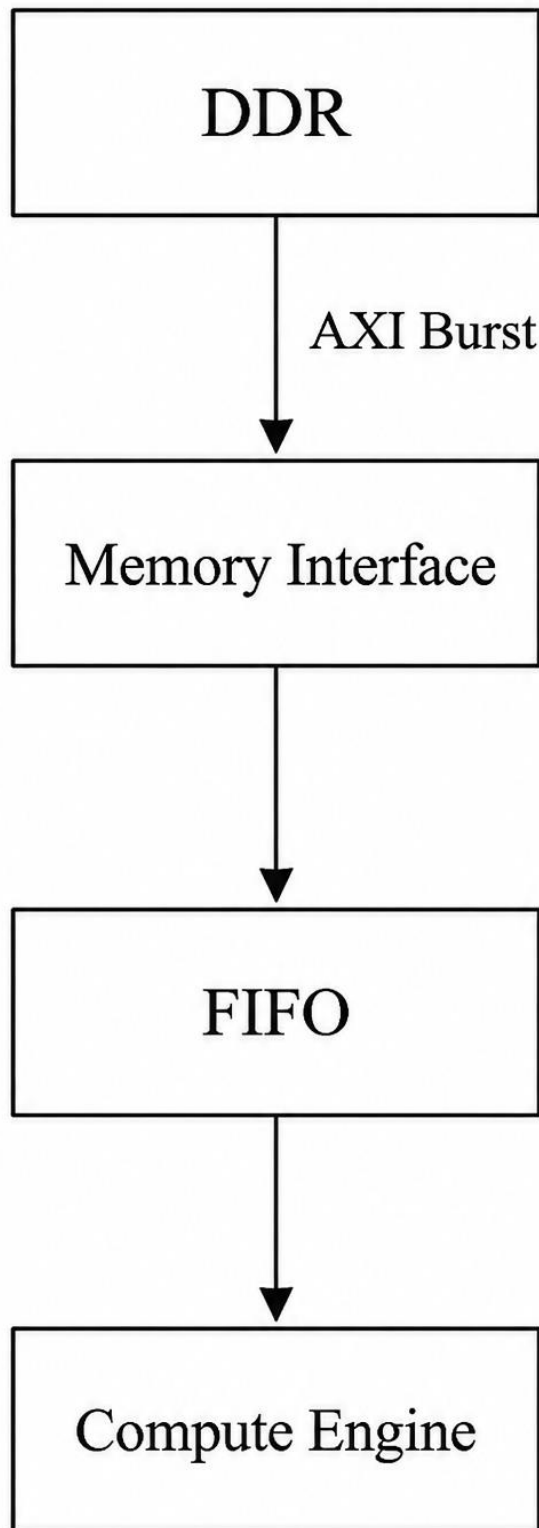
Producer tiếp tục truyền

Cơ chế này đảm bảo dữ liệu không bị mất dù producer và consumer có tốc độ khác nhau.

f) Quan hệ giữa Burst và FIFO

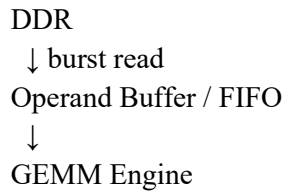
Burst giúp tăng hiệu quả truy cập bộ nhớ, trong khi FIFO giúp hấp thụ dữ liệu được trả về theo từng đợt.

Ví dụ:



DDR có thể trả về nhiều beat dữ liệu liên tiếp trong một burst. FIFO lưu tạm các beat này và cung cấp chúng dần cho compute engine.

Cách tổ chức này đặc biệt phù hợp với workload matrix multiplication:

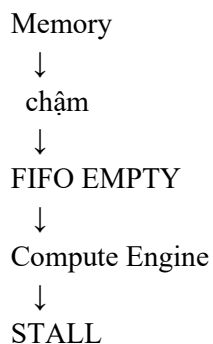


Compute engine không cần phải đồng bộ trực tiếp từng chu kỳ với latency của DDR.

g) Ảnh hưởng đến hiệu năng của accelerator

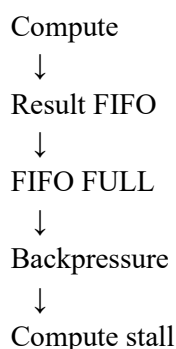
Hiệu năng của accelerator không chỉ phụ thuộc số lượng phần tử tính toán mà còn phụ thuộc khả năng duy trì dòng dữ liệu liên tục.

Giả sử compute engine có thể xử lý dữ liệu với tốc độ cao nhưng memory subsystem không cung cấp operand kịp thời:



Trong trường hợp đó, tăng thêm số lượng Processing Element chưa chắc làm tăng throughput, vì bottleneck nằm ở data delivery.

Ngược lại, nếu compute engine tạo kết quả nhanh hơn memory subsystem có thể ghi:



Do đó, một accelerator hiệu quả cần cân bằng giữa:

Memory bandwidth Buffer capacity

và:

Compute throughput

h) Vai trò trong hệ thống ViT Accelerator

Trong hệ thống ViT, các khái niệm trên xuất hiện ở nhiều cấp khác nhau.

Ở đường truy cập DDR:

DDR

↑↓

AXI Memory Interface

↑↓

Memory Frontend

burst giúp truyền các vùng tensor liên tục với hiệu quả cao hơn.

Ở bên trong accelerator:

Memory Frontend

↑↓

VALID / READY

↑↓

Compute Engine

VALID/READY được sử dụng để đồng bộ việc cung cấp operand và nhận kết quả.

Các FIFO được sử dụng tại những vị trí cần tách tốc độ giữa các khối:

Memory

↓

Operand buffering

↓

Compute

↓

Result buffering

↓

Memory

Khi downstream chưa thể tiếp nhận dữ liệu, cơ chế backpressure lan truyền ngược lên upstream để tạm dừng quá trình truyền dữ liệu.

i) Tổng kết

Bốn khái niệm có thể tóm tắt như sau:

Khái niệm	Chức năng chính
VALID/READY	Xác nhận một transfer hợp lệ
Burst	Truyền nhiều beat dữ liệu trong một transaction
FIFO	Tạm lưu và tách tốc độ giữa producer và consumer
Backpressure	Tạm dừng upstream khi downstream chưa sẵn sàng

Chúng tạo thành cơ sở cho việc xây dựng một data path có khả năng hoạt động ổn định trong FPGA SoC. Đặc biệt đối với accelerator ViT, nơi các phép GEMM yêu cầu lượng dữ liệu lớn, hiệu quả của burst, buffering và flow control có ảnh hưởng trực tiếp đến khả năng khai thác các phần tử tính toán.

2.6 Nhóm công trình liên quan

Các nghiên cứu về tăng tốc Vision Transformer trên FPGA trong những năm gần đây có thể được nhìn nhận theo nhiều hướng khác nhau. Một số công trình tập trung vào ánh xạ toàn bộ Transformer block lên các processing engine và tối ưu locality của dữ liệu; một số ưu tiên giảm truy cập bộ nhớ ngoài trên FPGA tài nguyên hạn chế; một nhóm khác tập trung vào các phép toán non-GEMM như Softmax, GELU và LayerNorm; trong khi các kiến trúc gần đây hơn nghiên cứu pipeline đa mức hoặc tổ chức hệ thống xoay quanh DDR để mở rộng tới các mô hình ViT lớn.

Năm công trình được lựa chọn trong báo cáo đại diện cho năm góc tiếp cận này, đồng thời có quan hệ trực tiếp với các vấn đề được khảo sát trong thiết kế NPU của báo cáo: tổ chức GEMM, xử lý non-GEMM, pipeline, sử dụng LUT/DSP và đặc biệt là data movement giữa bộ nhớ ngoài với compute engine.

2.6.1 ViA: A Novel Vision-Transformer Accelerator Based on FPGA

Wang và cộng sự đề xuất **ViA**, một kiến trúc tăng tốc Vision Transformer trên FPGA nhằm xử lý hai vấn đề mà tác giả cho rằng chưa được xem xét đầy đủ trong các accelerator Transformer trước đó: **data locality của dữ liệu ảnh** và **path dependence do residual/shortcut connection**.

Thay vì sử dụng một processing engine duy nhất cho toàn bộ Transformer, ViA chia quá trình tính toán thành hai processing engine tái sử dụng là **Norm Self-Attention (NSA)** và **Norm MLP (NMP)**. NSA thực hiện LayerNorm và phần Multi-Head Self-Attention, trong khi NMP thực hiện LayerNorm và MLP. Các khối con trong hai engine được tổ chức theo dạng stream nhằm khai thác tính song song giữa các giai đoạn tính toán.

Một đóng góp quan trọng của ViA là kỹ thuật **half-layer mapping**. Residual connection tạo ra dependency giữa dữ liệu đầu vào và đầu ra của nhiều tầng pipeline, dẫn tới nhu cầu lưu intermediate activation trong buffer. ViA khai thác tính đều đặn của residual path trong Transformer block để chia block thành hai nửa và ánh xạ chúng lên NSA và NMP. Theo kết quả của tác giả, phương pháp này làm giảm đáng kể chi phí buffer so với pipeline trực tiếp.

Bên cạnh đó, ViA phân tích quan hệ độc lập giữa các chiều dữ liệu của ViT và thực hiện partition theo các chiều thích hợp để tăng mức song song và giảm kích thước dữ liệu mà mỗi processing engine phải giữ cục bộ. Sau đó, tác giả thực hiện throughput balancing giữa các stage nhằm tránh trường hợp một stage có latency quá lớn làm giới hạn toàn bộ pipeline.

ViA được triển khai trên **Xilinx Alveo U50**, sử dụng Vitis HLS/Vivado. Trong cấu hình đánh giá của tác giả, kiến trúc đạt peak computing performance khoảng **309,6 GOP/s**. Báo cáo của ViA cũng cho thấy energy efficiency cao hơn khoảng 5,2 lần so với NVIDIA Tesla V100 trong phép so sánh được tác giả thiết lập.

Giá trị tham khảo chính của ViA đối với báo cáo này không nằm ở con số throughput tuyệt đối, mà ở cách công trình xem ViT như một **dataflow hoàn chỉnh** thay vì chỉ tập trung vào phép nhân ma trận. Việc phân tách MSA và MLP thành các engine, xử lý residual dependency và cân bằng throughput giữa các stage cho thấy một accelerator ViT hiệu quả phải đồng thời xem xét compute architecture, local buffering và scheduling.

2.6.2 ViTA: A Vision Transformer Inference Accelerator for Edge Applications

Khác với ViA được triển khai trên accelerator card Alveo U50, Nag và cộng sự phát triển **ViTA** với mục tiêu rõ ràng hơn là các **FPGA tài nguyên hạn chế ở edge**.

Tác giả nhận xét rằng các nền tảng như Zynq ZC7020 chỉ có vài trăm DSP và dung lượng BRAM tương đối nhỏ. Do đó, việc cố lưu toàn bộ weight và intermediate activation trên chip như các kiến trúc dành cho FPGA lớn là không khả thi. ViTA vì thế tập trung vào việc giảm trao đổi dữ liệu giữa FPGA và off-chip DRAM.

ViTA sử dụng **post-training INT8 quantization** cho weight và activation. Đối với ViT-B/16, tác giả cho biết việc chuyển sang INT8 chỉ gây suy giảm Top-1 accuracy nhỏ trong thí nghiệm của họ. Kiến trúc lựa chọn **input-stationary dataflow**, giữ activation tại bộ nhớ on-chip trong khi weight được đưa từ DRAM vào theo nhu cầu.

Để che latency của việc nạp weight, hai vùng BRAM được sử dụng luân phiên ở mức cột: khi một cột weight đang tham gia tính toán thì cột tiếp theo có thể được nạp từ off-chip memory. Đây về bản chất là một dạng **double buffering/prefetching** nhằm overlap communication với computation.

Đối với MSA, ViTA sử dụng **head-level coarse-grained pipeline**. Các PE block xử lý Q, K và V của một attention head, trong khi các block khác xử lý (QK^T), Softmax và phép nhân với V của head trước đó. Với MLP, tác giả áp dụng inter-layer optimization để intermediate result có thể được chuyển trực tiếp giữa hai nhóm MAC thay vì phải staging toàn bộ hidden layer ra bộ nhớ ngoài.

ViTA còn tích hợp các unit riêng cho Softmax, LayerNorm, skip connection và nonlinear activation. Vì vậy, đây không phải chỉ là một GEMM accelerator mà là kiến trúc hướng tới thực thi các thành phần chính của Transformer block trên FPGA.

Trên Zynq ZC7020 ở 150 MHz, ViTA báo cáo công suất khoảng **0,88 W**. Đối với ViT-B/16 hoặc DeiT-B với ảnh (224×224), thiết kế đạt khoảng **92,8% hardware utilization efficiency** và **2,75 fps** theo cấu hình của tác giả.

ViTA đặc biệt liên quan tới đề tài của báo cáo vì cả hai cùng phải giải quyết vấn đề **compute resource hạn chế và off-chip memory access**. Tuy nhiên, ViTA giải quyết vấn đề này bằng INT8, input-stationary dataflow và chủ động overlap việc nạp weight, trong khi thiết kế M8 sử dụng mixed FP16/FP32 và hiện vẫn phụ thuộc đáng kể vào đường cấp toán hạng từ DDR.

2.6.3 A High Speed Reconfigurable Architecture for Softmax and GELU in Vision Transformer

Li và cộng sự tiếp cận một khía cạnh khác của ViT acceleration: **các phép toán phi tuyến không thuộc GEMM**.

Trong Transformer, phần lớn số phép tính nằm ở các linear layer, tuy nhiên Softmax và GELU xuất hiện lặp lại trong mỗi encoder block và chứa các phép toán khó hiện thực trực tiếp trên FPGA như exponential, division và nonlinear approximation. Tác giả chỉ ra rằng nếu hiện thực trực tiếp các phép toán này bằng FP32 thì chi phí phần cứng rất lớn.

Công trình đề xuất một datapath có thể **reconfigure để sử dụng chung cho Softmax và GELU**. Softmax trước hết được ổn định số học bằng cách trừ giá trị lớn nhất. Các hàm exponential cơ số (e) sau đó được chuyển sang dạng cơ số 2. Một số phép nhân với hằng số được xấp xỉ bằng tổ hợp **shift-and-add**, trong khi division và exponentiation được biến đổi thành các thao tác phù hợp hơn với phần cứng như leading-one detection và bit shifting.

Kiến trúc gồm các thành phần Input FIFO, Max Unit, Power Unit, Intermediate FIFO, Unified Shift-Add Unit, Rounding LOD và Rounding Output. Việc dùng chung datapath cho hai operator cho phép tăng mức tái sử dụng phần cứng thay vì xây dựng hai accelerator hoàn toàn độc lập.

Một điểm đáng chú ý là công trình sử dụng **INT16 fixed-point** thay cho FP32 cho Softmax và GELU. Kết quả trên ImageNet cho thấy khi đồng thời thay Softmax và GELU bằng phiên bản INT16, Top-1 accuracy giảm khoảng **0,38% đối với ViT-B** và **0,37% đối với ViT-L** trong phép thử của tác giả.

Thiết kế được viết bằng Verilog HDL và triển khai trên **Xilinx XCVU37P** ở 200 MHz. Tác giả báo cáo throughput **102,4 Gbit/s**, sử dụng khoảng 43.940 LUT, 44.861 register và **0 DSP**.

Công trình này có ý nghĩa trực tiếp đối với thiết kế M8 vì chứng minh rằng việc tối ưu accelerator ViT không thể chỉ tập trung vào GEMM. Những operator như Softmax và GELU có đặc điểm tính toán khác biệt và có thể trở thành chi phí đáng kể nếu vẫn được xử lý ngoài accelerator. Đồng thời, việc đạt DSP=0 cho hai operator bằng các biến đổi toán học cho thấy LUT-based implementation là một điểm thiết kế khả thi, mặc dù phương pháp của công trình dựa trên INT16 và approximation, khác với mixed floating-point của M8.

2.6.4 HG-PIPE: Vision Transformer Acceleration with Hybrid-Grained Pipeline

HG-PIPE của Guo và cộng sự tiếp tục mở rộng vấn đề từ tối ưu từng operator sang **tối ưu pipeline end-to-end của toàn bộ ViT**.

Tác giả chia các kiến trúc ViT FPGA trước đó thành hai nhóm chính. **Temporal architecture** sử dụng lại cùng một GEMM engine cho nhiều operator, có ưu điểm tiết kiệm tài nguyên nhưng dễ phát sinh off-chip memory traffic và utilization thấp. Ngược lại, **pipelined architecture** ánh xạ nhiều operator lên các hardware stage riêng biệt để stream activation trực tiếp giữa các stage, nhưng phải đối mặt với nhu cầu tài nguyên lớn và pipeline bubble do dependency trong attention.

HG-PIPE đề xuất **hybrid-grained pipeline**, kết hợp đặc điểm của fine-grained và coarse-grained pipeline. Các FIFO sâu được sử dụng tại những nhánh có thể stream theo tile, trong khi các deep buffer được dùng tại những vị trí cần giữ toàn bộ K hoặc V do dependency của attention. Theo tác giả, phương pháp này làm giảm khoảng **83,3% activation-buffer cost** so với coarse-grained PIPO implementation.

Một đặc điểm quan trọng khác là việc khai thác LUT để giảm áp lực DSP. HG-PIPE sử dụng low-bit quantization cùng các LUT-based approximation cho Exp, GeLU, Reciprocal, Rsqrt và ReQuant. Các nonlinear operator sau tối ưu không còn sử dụng DSP riêng; ở mức toàn hệ thống, tác giả báo cáo giảm khoảng **89,6% DSP usage** so với cấu hình trước tối ưu. Tuy vậy HG-PIPE **không phải kiến trúc DSP=0 toàn hệ thống**: các MAC unit chính vẫn sử dụng tài nguyên tính toán song song và cấu hình cuối vẫn còn DSP.

HG-PIPE được đánh giá trên ZCU102 và VCK190. Trên VCK190, tác giả hiện thực end-to-end ViT acceleration hoàn toàn trong PL và báo cáo khoảng **7118 images/s**, tương đương **17,8 TOP/s** trong cấu hình đánh giá.

HG-PIPE cho thấy một bước phát triển quan trọng của accelerator ViT: khi số lượng compute engine đủ lớn, vấn đề không còn đơn thuần là tốc độ của từng MAC mà chuyển sang **pipeline balance, buffer organization, global dependency và resource roofline**. Đây là điểm tham khảo quan trọng khi phân tích sự khác nhau giữa timing bottleneck và throughput bottleneck trong thiết kế M8.

2.6.5 MDS-ViT: A Multi-Data Stream FPGA-Based Vision Transformer Accelerator

MDS-ViT của Ye và cộng sự năm 2026 đại diện cho một hướng khác so với các kiến trúc cố gắng giữ phần lớn dữ liệu trong on-chip memory. Công trình đặt **DDR ở vị trí trung tâm của dataflow** nhằm giải quyết bài toán mở rộng tới các ViT kích thước lớn.

Tác giả cho rằng chiến lược ưu tiên giữ weight và activation trên BRAM/URAM đạt hiệu quả tốt với các model nhỏ nhưng gặp giới hạn khi số tham số tăng lên hàng trăm triệu. Vì vậy MDS-ViT chủ động lưu intermediate result trong DDR và thiết kế compute pipeline, cache hierarchy và data stream xung quanh mô hình truy cập này.

Compute core chính là **High-Performance Matrix Multiplication (HP_MM)**. Cùng một HP_MM được sử dụng cho các GEMM trong MHA và MLP. MDS-ViT sử dụng nhiều mức parallelism: nhiều HP_MM chạy song song, nhiều PE trong mỗi HP_MM và nhiều multiplier trong một PE.

Bên cạnh GEMM, LayerNorm, Softmax, GELU và residual addition cũng được hiện thực trên FPGA. Một Transformer encoder được phân thành **10 computational step** được điều khiển bằng configuration parameter. Cách tổ chức này cho phép cùng một bitstream hỗ trợ nhiều ViT/DeiT khác nhau thay vì khóa cứng vào một model duy nhất.

Phần đáng chú ý nhất của MDS-ViT là hệ thống bộ nhớ. Thiết kế sử dụng hierarchy:

DDR → URAM → FIFO → HP_MM

trong đó URAM đóng vai trò vừa là cache vừa là bộ chuyển đổi access pattern. Theo kết quả mô phỏng của tác giả, nếu loại bỏ tầng URAM này thì throughput chỉ còn khoảng **27%** so với cấu hình đầy đủ. Công trình còn sử dụng dual-interface DDR access và ping-pong buffer để che latency cũng như tránh một số thao tác transpose riêng biệt.

MDS-ViT sử dụng **INT16 cho feature, weight và inter-layer activation**, accumulation ở 32 bit. Hệ thống được triển khai bằng Vitis HLS/OpenCL trên **Xilinx Alveo U250**, với bốn kernel có thể hoạt động song song. Tác giả báo cáo **28 fps cho ViT-B/16 với ảnh (256x256)** và vẫn hỗ trợ tới ViT-H/16. Trong thử nghiệm 100 class ImageNet, average probability drop so với PyTorch được báo cáo khoảng 0,035%.

MDS-ViT có quan hệ đặc biệt trực tiếp với vấn đề được quan sát trong M8. Kết quả profiling của M8 cho thấy compute engine thường bị chờ operand, trong khi MDS-ViT xem DDR bandwidth và khả năng biến các burst access thành stream đều đặn là một vấn đề kiến trúc trung tâm. Do đó, các

kỹ thuật multi-level caching, prefetch và ping-pong buffering trong MDS-ViT là những tham khảo quan trọng cho hướng tối ưu memory frontend của M8.

2.7 Ma trận so sánh related work

Bảng 2.11 tổng hợp năm công trình được lựa chọn cùng với thiết kế trong báo cáo. Cần lưu ý rằng các giá trị throughput không thể được dùng để xếp hạng trực tiếp do các công trình sử dụng model, precision, FPGA, frequency và phạm vi phần cứng khác nhau.

Bảng 2.11. So sánh các công trình FPGA Vision Transformer liên quan

Công trình	Năm	FPGA/ nền tảng	Precision	Phạm vi tăng tốc	Kiến trúc/ dataflow nổi bật	Non- GEMM	Chiến lược bộ nhớ	Kết quả đại diện
ViA – Wang et al.	2022	Alveo U50	Không lấy low-bit quantization làm trọng tâm chính	Transformer block/ViT accelerator	Hai reuse engine NSA–NMP; stream; half- layer mapping; throughput balancing	LayerNorm, Softmax, GELU nằm trong các engine	Data partition + external memory + local buffer	Peak $\approx$ 309,6 GOP/s ; $5,2\times$ energy efficiency so với V100 theo thiết lập của tác giả
ViTA – Nag et al.	2023	Zynq ZC7020	INT8	MSA + MLP và các operator chính	Input-stationary; head-level coarse pipeline; inter- layer MLP optimization	Có LayerNorm, Softmax, activation, skip connection	Giữ activation on-chip; weight streaming; double-buffer theo cột	2,75 fps cho ViT- B/16/DeiT-B 224 ² ; 0,88 W; HUE $\approx 92,8\%$
Softmax/GELU – Li et al.	2023	XCVU37P	INT16	Chỉ Softmax + GELU	Reconfigurable shared datapath; shift-add; LOD; approximation	Softmax + GELU	FIFO nội bộ	102,4 Gbit/s @200 MHz, 0 DSP ; ViT- B Top-1 loss $\approx 0,38\%$ khi thay cả hai operator

HG-PIPE – Guo et al.	2024	ZCU102/ VCK190	Low-bit, chủ yếu 3/4 bit	End-to-end ViT	Hybrid fine/coarse- grained pipeline; output-stationary MM; distributed control	LayerNorm, Softmax, GELU, ReQuant	Giữ nhiều weight/activation on-chip; deep FIFO + deep buffer	7118 img/s, 17,8 TOP/s trên VCK190; buffer giảm 83,3%; DSP giảm 89,6%
MDS-ViT – Ye et al.	2026	Alveo U250	INT16 , ACC 32 bit	End-to-end, nhiều ViT/DeiT	Multi-data- stream; configurable 10- step flow; nhiều HP_MM	LayerNorm, Softmax, GELU, residual	DDR-centric , DDR– URAM– FIFO, dual DDR interface, ping-pong	28 fps ViT-B/16 256² ; hỗ trợ ViT- Huge; average probability drop ≈0,035% trong test của tác giả
Thiết kế trong báo cáo	2026	Genesys ZU- 5EV	Mixed FP16/FP32	ViT- Base/Patch16- 224 cố định	Descriptor- driven; GEMM + 6 non-GEMM engine; sequencer 249 command	Vector, Layout, LayerNorm, Softmax, GELU, Argmax	DDR qua AXI4 128-bit; memory frontend; chưa có prefetch/ping- pong hoàn chỉnh	Board full- schedule/Top-1 PASS; numerical vector chưa sign- off; DSP=0 toàn thiết kế

Qua bảng so sánh có thể nhận thấy ba xu hướng phát triển chính.

Thứ nhất, các kiến trúc hiệu năng cao ngày càng cố gắng giảm khoảng cách giữa các operator bằng **streaming hoặc pipeline**, thay vì liên tục ghi intermediate tensor về external memory. ViA, ViTA và đặc biệt HG-PIPE đều khai thác hướng này ở các mức độ khác nhau.

Thứ hai, **non-GEMM operator đã trở thành thành phần kiến trúc quan trọng**. Từ accelerator chuyên dụng Softmax/GELU tới HG-PIPE và MDS-ViT, LayerNorm, Softmax và GELU đều được đưa vào FPGA thay vì coi GEMM là toàn bộ workload.

Thứ ba, **memory architecture** có ảnh hưởng trực tiếp tới compute utilization. ViTA cố gắng tránh off-chip traffic; HG-PIPE giữ dữ liệu trong pipeline; trong khi MDS-ViT đi theo hướng ngược lại nhưng có chủ đích: sử dụng DDR như một phần của dataflow và xây dựng hierarchy cache đủ mạnh để che latency.

M8 nằm giữa các hướng này: các operator chính đã được đưa vào PL, nhưng architecture vẫn mang tính **temporal/descriptor-driven** và phụ thuộc nhiều vào DDR. Điều này giải thích tại sao vấn đề cấp operand trở thành bottleneck chính trong kết quả runtime hiện tại. Kiến trúc M8 sử dụng tám FP16 stream, FP32 accumulation/output và toàn bộ datapath không sử dụng DSP; control được thực hiện bằng sequencer 249 command và dữ liệu đi qua AXI4 master 128 bit tới DDR.

2.8 Định vị phạm vi so với related work

Từ năm công trình trên có thể thấy thiết kế trong báo cáo không nên được định vị đơn giản theo tiêu chí “**accelerator nào có FPS cao hơn**”. Các kiến trúc đang khảo sát các điểm khác nhau trong design space.

ViA tập trung vào locality, path dependence và cân bằng pipeline; ViTA tập trung vào edge FPGA và giảm off-chip traffic; Li và cộng sự tập trung vào hardware-efficient non-GEMM; HG-PIPE ưu tiên throughput thông qua spatial pipeline và low-bit quantization; trong khi MDS-ViT ưu tiên scalability bằng kiến trúc DDR-centric.

Thiết kế M8 lựa chọn một điểm thiết kế khác với bốn đặc trưng chính.

2.8.1 Ràng buộc DSP=0 và mixed floating-point

Khác biệt rõ nhất của M8 là ràng buộc **không sử dụng DSP block cho toàn thiết kế**.

Điểm này cần phân biệt với các công trình sử dụng LUT để giảm DSP. HG-PIPE sử dụng LUT-based nonlinear approximation nhưng compute array của hệ thống vẫn khai thác DSP. Công trình Softmax/GELU đạt 0 DSP cho riêng accelerator nonlinear, nhưng không phải một NPU end-to-end.

M8 áp dụng DSP=0 đồng thời cho GEMM và non-GEMM, với operand GEMM ở FP16 và accumulation/output chủ yếu ở FP32. Theo implementation của báo cáo, GEMM gồm tám parallel FP16 stream và các datapath số học được tổng hợp bằng LUT/FF.

Do đó, M8 không cạnh tranh trực tiếp với HG-PIPE hoặc MDS-ViT về peak throughput. Ràng buộc DSP=0 làm giảm compute density nhưng cho phép khảo sát chi phí thực tế của LUT-based floating-point arithmetic trên FPGA.

Định vị phù hợp hơn là:

M8 khảo sát khả năng xây dựng một baseline ViT NPU end-to-end trong miền accelerator dưới ràng buộc DSP=0 và mixed FP16/FP32, thay vì tối đa hóa throughput bằng low-bit quantization và DSP-rich compute array.

Không nên từ phạm vi related work hiện tại tuyên bố đây là accelerator ViT DSP=0 đầu tiên nếu chưa thực hiện systematic literature search đủ rộng.

2.8.2 Tích hợp GEMM và non-GEMM trong cùng NPU

Một điểm tương đồng giữa M8 và xu hướng của các công trình gần đây là việc không coi GEMM là toàn bộ ViT accelerator.

M8 có các engine:

- GEMM;
- Vector;
- Layout;
- LayerNorm;
- Softmax;
- GELU;
- Argmax.

Điều này đưa thiết kế gần hơn với ViA, HG-PIPE và MDS-ViT so với một accelerator chỉ hỗ trợ matrix multiplication.

Tuy nhiên, cách tổ chức khác nhau. HG-PIPE spatially unroll nhiều operator thành pipeline riêng và truyền tensor trực tiếp giữa các stage. MDS-ViT tổ chức một configurable multi-data-stream flow. Trong khi đó M8 sử dụng **command/descriptor-based temporal execution**: sequencer lần lượt phát 249 command và engine dispatcher lựa chọn engine tương ứng.

Ưu điểm của cách tiếp cận M8 là giảm số lượng hardware block cần hoạt động đồng thời và phù hợp hơn với FPGA nhỏ. Nhược điểm là intermediate data movement và thời gian chuyển đổi giữa các operator trở nên quan trọng hơn.

2.8.3 Memory architecture là khác biệt lớn nhất về throughput

Related work cho thấy ba chiến lược bộ nhớ điển hình.

ViTA cố giữ activation on-chip và stream weight theo yêu cầu.

HG-PIPE sử dụng pipeline và FIFO/buffer để phần lớn intermediate activation không phải quay lại external memory.

MDS-ViT chấp nhận sử dụng DDR thường xuyên nhưng xây dựng toàn bộ architecture xung quanh DDR bằng multi-level cache, nhiều data stream, ping-pong buffering và dual-interface access.

M8 hiện tại gần với trường hợp thứ ba hơn ở mức tổ chức dữ liệu, nhưng chưa có cùng mức tối ưu memory frontend.

Đường dữ liệu của M8 hiện là:

DDR → AXI/SmartConnect → AXI memory adapter → memory frontend → operand bank → compute engine.

Performance counter của thiết kế cho thấy feeder stall ở mức cao và GEMM compute-active thấp, trong khi số GEMM stream hoạt động gần đầy khi operand đã có sẵn. Do đó, bottleneck hiện tại không thể chỉ được giải quyết bằng cách tăng thêm multiplier.

Ở khía cạnh này, MDS-ViT là related work có giá trị trực tiếp nhất cho hướng phát triển tiếp theo. Kết quả của MDS-ViT cho thấy tầng URAM/FIFO đóng vai trò như **bandwidth amplifier**, và việc bỏ URAM khiến throughput chỉ còn khoảng 27% cấu hình đầy đủ. Đây là bằng chứng cho thấy memory hierarchy và operand delivery có thể ảnh hưởng tới hiệu năng ở mức tương đương, thậm chí lớn hơn, việc mở rộng compute array.

2.8.4 M8 ưu tiên RTL-to-board validation và bottleneck characterization

Một khác biệt khác về phạm vi nghiên cứu là M8 không chỉ tập trung vào architecture-level throughput.

Thiết kế được thực hiện theo chuỗi:

RTL → simulation → synthesis → implementation → timing closure → bitstream/XSA → Genesys ZU-5EV board execution.

Hệ thống còn bao gồm PS-PL integration với:

- AXI4-Lite control path;
- AXI4 128-bit DDR data path;
- SmartConnect;
- register bank;
- sequencer;
- memory frontend;
- performance counter.

Kết quả implementation ghi nhận DSP=0, LUT utilization cao và critical path nằm trong shared FP32 non-GEMM datapath. Đồng thời runtime counter cho phép phân biệt **timing bottleneck** với **operand-delivery/runtime bottleneck**.

Vì vậy, giá trị chính của M8 ở giai đoạn hiện tại là **system-level characterization** hơn là peak-performance demonstration.

2.8.5 Giới hạn khi so sánh với các công trình đã công bố

Không nên trực tiếp đặt các con số như 7118 img/s của HG-PIPE, 28 fps của MDS-ViT hay 2,75 fps của ViTA cạnh latency M8 rồi kết luận về chất lượng kiến trúc.

Các công trình khác nhau đáng kể về:

- FPGA device;
- số DSP/LUT/BRAM/URAM;
- frequency;
- image size;
- ViT variant;
- precision;
- quantization;
- phạm vi operator nằm trên FPGA;
- memory hierarchy;
- batch và measurement protocol.

Ví dụ, HG-PIPE sử dụng low-bit quantization và một FPGA VCK190 có lượng tài nguyên lớn, trong khi M8 sử dụng mixed floating-point trên ZU-5EV và không sử dụng DSP. MDS-ViT sử dụng Alveo U250 với bốn accelerator kernel, còn M8 chỉ sử dụng fabric của ZU-5EV. Do đó, các giá trị FPS/TOPS chỉ nên dùng để mô tả **đặc tính của từng kiến trúc trong điều kiện thí nghiệm của chính công trình**, không phải benchmark công bằng giữa các thiết kế.

2.8.6 Định vị tổng thể của thiết kế M8

Từ các công trình được khảo sát, phạm vi của M8 có thể được định vị như sau:

Thiết kế trong báo cáo là một NPU chuyên dụng cho ViT-Base/Patch16-224 trên Zynq UltraScale+ theo hướng descriptor-driven, tích hợp GEMM và các non-GEMM operator chính trong PL, sử dụng mixed FP16/FP32 và áp đặt ràng buộc DSP=0. Khác với các kiến trúc ưu tiên peak throughput thông qua low-bit quantization, spatial pipeline hoặc DSP-rich matrix engines, mục tiêu của M8 là xây dựng và kiểm chứng một baseline RTL-to-board hoàn chỉnh, sau đó sử dụng timing report và runtime counter để xác định các giới hạn thực tế của compute datapath và memory/data-delivery path.

Nhìn từ tiến trình của related work, M8 đồng thời tiếp cận ba vấn đề đã được cộng đồng nhận diện:

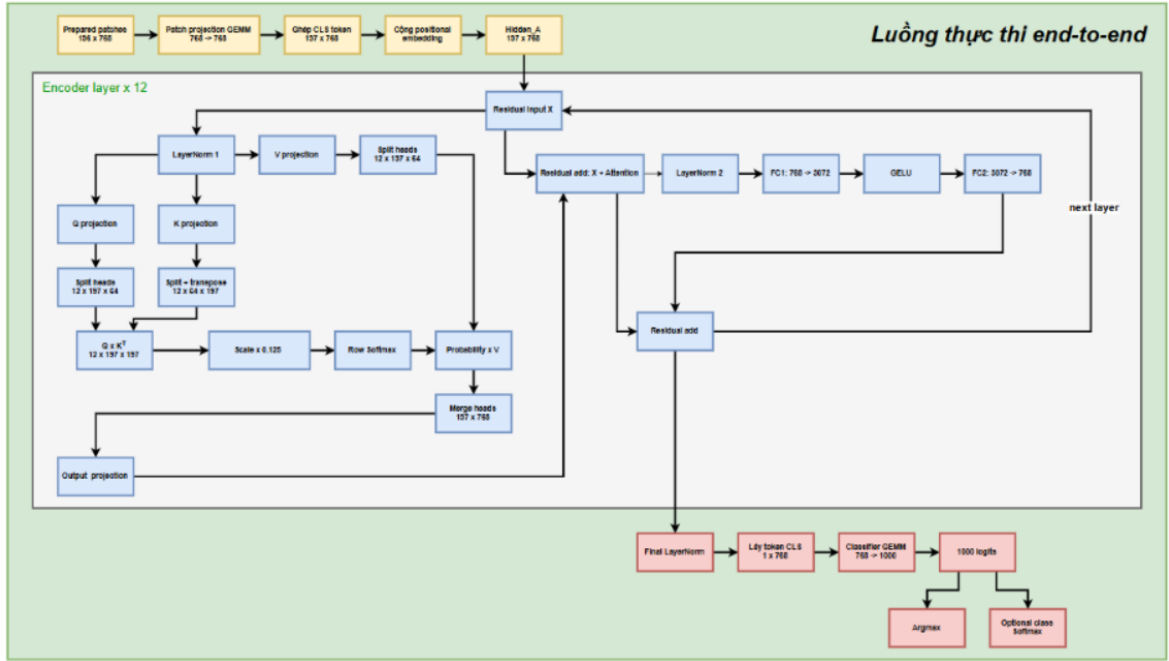
- (1) Operator coverage:** GEMM alone không đủ cho ViT, do đó LayerNorm, Softmax, GELU và các phép vector cần được xử lý trong accelerator.
- (2) Resource trade-off:** LUT có thể được khai thác để giảm sự phụ thuộc vào DSP, nhưng lựa chọn này tạo trade-off về logic depth, routing và compute density.
- (3) Data movement:** khi compute datapath đã hoạt động đúng, khả năng cung cấp operand từ DDR và reuse dữ liệu trở thành yếu tố quyết định utilization thực tế.

Do đó, khoảng trống cần tiếp tục xử lý của M8 không phải đơn thuần là mở rộng số lượng PE. Các kết quả từ ViTA, HG-PIPE và đặc biệt MDS-ViT cho thấy hướng phát triển hợp lý hơn là bổ sung **prefetch, ping-pong buffering, tăng data reuse, tối ưu AXI burst/outstanding transactions và giảm DDR round-trip** trước khi mở rộng compute parallelism.

Theo cách định vị này, M8 không được trình bày như một kiến trúc thay thế hoặc vượt trội các accelerator đã công bố về throughput. Thay vào đó, nó đóng vai trò một **research baseline có board validation** để khảo sát trade-off của mixed floating-point, DSP=0, PS-PL integration và memory bottleneck trên một nền tảng Zynq UltraScale+ có tài nguyên hữu hạn.

CHƯƠNG 3 — PHÂN TÍCH WORKLOAD VÀ ÁNH XẠ

3.1 Đồ thị thực thi end-to-end của ViT



3.2 Bảng tensor shape xuyên suốt mô hình

Ký hiệu shape dưới đây bỏ qua batch bằng 1 khi không gây nhầm lẫn. Mọi activation động, scratch và output của M8 được giữ dưới dạng FP32; riêng persistent GEMM-B có thể được lưu packed FP16 trong MODEL, như đã trình bày ở Mục 2.3.

Giai đoạn	Tensor vào	Tensor ra	Shape tensor ra	Kiểu dữ liệu	Không gian nhớ chính
Tiền xử lý	Ảnh RGB đã decode	pixel_values	[1,3,224,224]	FP32	Host RAM
Patchify	pixel_values	PATCH_A	[1,196,768]	FP32	INPUT

Patch projection	PATCH_A, W_patch, b_patch	LINEAR_TMP	[196,768]	FP32 output; W_patch packed FP16	SCRATCH, MODEL
Ghép token	CLS, LINEAR_TMP	HIDDEN_A	[197,768]	FP32	SCRATCH, MODEL
Positional embedding	HIDDEN_A, POS	HIDDEN_A	[197,768]	FP32	SCRATCH, MODEL
LN1/LN2/final LN	[197,768], gamma, beta	[197,768]	[197,768]	FP32	SCRATCH, MODEL
Q, K, V projection	token matrix, W _q /W _k /W _v , bias	LINEAR_TMP rồi head tensor	[197,768] rồi [12,197,64]	FP32 output; persistent B packed FP16	SCRATCH, MODEL
K transpose	K_HEAD	LINEAR_TMP (semantic K ^T)	[12,64,197]	FP32, bit-preserving	SCRATCH
Attention score	Q_HEAD, LINEAR_TMP chứa K ^T	SCORE_PROB	[12,197,197]	FP32 output	SCRATCH
Attention probability	score	SCORE_PROB in-place	[12,197,197]	FP32	SCRATCH
Attention context	probability, V_HEAD	Q_HEAD tái sử dụng	[12,197,64]	FP32 output	SCRATCH
Merge head và output projection	head workspace, W _o , bias	HIDDEN_B	[197,768]	FP32 output	SCRATCH, MODEL
MLP mở rộng	HIDDEN_A, W1, b1	FC1	[197,3072]	FP32 output	SCRATCH, MODEL
GELU	FC1	FC1 in-place	[197,3072]	FP32	SCRATCH
MLP thu hẹp	FC1, W2, b2	LINEAR_TMP	[197,768]	FP32 output	SCRATCH, MODEL

Chọn CLS	final-LN output	LINEAR_TMP	[1,768]	FP32, bit-preserving	SCRATCH
Classifier	CLS, W_cls, b_cls	LOGITS	[1,1000]	FP32 output	LOGITS, MODEL
Class Softmax tùy chọn	LOGITS	CLASS_PROB	[1,1000]	FP32	PROBS

Ba kích thước chi phối chi phí compute và scratch là 197×768 cho token matrix, $12 \times 197 \times 197$ cho attention score/probability, và 197×3072 cho FC1. Bảng này vì vậy cũng là cơ sở để xác định tensor nào cần không gian riêng trong SCRATCH và tensor nào có thể tái sử dụng sau khi consumer cuối cùng đã hoàn tất.

3.3 Phân loại GEMM và non-GEMM

3.3.1 Nhóm GEMM

Các phép GEMM đều có dạng $C = A \times B + \text{bias}$ khi bias được bật. Chúng dùng chung GEMM engine, nhưng khác nhau về shape, nguồn B và có/không có bias.

Phép GEMM	Shape logic	Nguồn B	Bias	Vai trò trong luồng
Patch projection	$[196, 768] \times [768, 768] \rightarrow [196, 768]$	Persistent MODEL	Có	Biến patch phẳng thành patch embedding
Q projection	$[197, 768] \times [768, 768] \rightarrow [197, 768]$	Persistent MODEL	Có	Tạo query trước split head
K projection	$[197, 768] \times [768, 768] \rightarrow [197, 768]$	Persistent MODEL	Có	Tạo key trước split/transpose
V projection	$[197, 768] \times [768, 768] \rightarrow [197, 768]$	Persistent MODEL	Có	Tạo value trước split head
QK^T	$[12, 197, 64] \times [12, 64, 197] \rightarrow [12, 197, 197]$	Dynamic SCRATCH	Không	Tạo attention score theo head

P×V	$[12,197,197] \times [12,197,64] \rightarrow [12,197,64]$	Dynamic SCRATCH	Không	Tổng hợp value theo attention probability
Output projection	$[197,768] \times [768,768] \rightarrow [197,768]$	Persistent MODEL	Có	Chiều context đã merge về hidden size
FC1	$[197,768] \times [768,3072] \rightarrow [197,3072]$	Persistent MODEL	Có	Mở rộng MLP
FC2	$[197,3072] \times [3072,768] \rightarrow [197,768]$	Persistent MODEL	Có	Thu hẹp MLP
Classifier	$[1,768] \times [768,1000] \rightarrow [1,1000]$	Persistent MODEL	Có	Sinh logits của 1.000 lớp

Persistent B được cấp ở format blocked/packed phù hợp GEMM; B động của QK^T và P×V được đọc từ SCRATCH. Cả hai trường hợp đều tuân theo boundary FP32→FP16 ở GEMM ingress khi cần, còn output GEMM trở lại FP32 trước khi trở thành tensor trung gian.

3.3.2 Nhóm non-GEMM

Non-GEMM không phải phần phụ có thể bỏ qua: chúng tạo đủ đường dữ liệu để ViT chạy end-to-end. M8 có sáu engine non-GEMM với vai trò sau. Bias của các phép GEMM được cộng trong epilogue của GEMM khi BIAS_ENABLE được bật, nên không được đếm như một command Vector độc lập.

Engine	Phép/điểm dùng trong ViT	Tensor điển hình	Đặc điểm ánh xạ
Vector	Positional add, attention scale $1/\sqrt{64}$, residual 1 và residual 2	$[197,768]$ hoặc $[12,197,197]$	FP32 add hoặc scale; attention mask bị tắt trong schedule M8; hỗ trợ in-place theo schedule

Layout	Copy CLS, copy patch, Q/K/V split, K transpose, head merge, select CLS	[197,768], [12,197,64], [12,64,197]	Không tính toán số học; copy word FP32 theo dim/stride
LayerNorm	LN1, LN2 của 12 encoder và final LN	[197,768]	Mean, biased variance, normalize và affine FP32
Softmax	12 attention Softmax và class Softmax tùy chọn	[12,197,197], [1,1000]	Max ổn định số, exp xấp xỉ, sum và normalize
GELU	Sau FC1 của mỗi encoder	[197,3072]	GELU approximant FP32, ghi đè FC1 sau khi đọc đủ packet
Argmax	Sau classifier	[1000]	So sánh numeric FP32, trả class index/logit qua sideband

Khi class Softmax được bật, full schedule gồm 98 GEMM, 37 Vector, 63 Layout, 25 LayerNorm, 13 Softmax, 12 GELU và 1 Argmax. Các count này là số lần thực thi command, không phải số lượng FSM hoặc số instance phần cứng.

3.4 Template lệnh, descriptor và lịch lệnh M8

M8 là accelerator chuyên dụng cho topology ViT-B/16 đã khóa. Nó không chạy compiler PyTorch tại thời điểm inference. Sequencer phát descriptor theo lịch cố định và chỉ chuyển sang descriptor kế tiếp khi command controller báo hoàn tất hoặc lỗi cho descriptor hiện tại.

4 template embedding
+ 20 template/encoder × 12 encoder
+ 5 template cuối khi class Softmax bật
= 29 template, 249 command instance

Nếu class Softmax bị tắt, ordinal cuối không phát và job có 248 command. Vì vậy, cụm từ 249 command chỉ mô tả số instance của full schedule có class Softmax; nó không có nghĩa thiết kế có 249 trạng thái FSM hoặc 249 loại opcode.

Pha	Nội dung	Số command	Ordinal 1-based
Embedding	Patch GEMM, copy CLS, copy patch, positional add	4	1–4
Mỗi encoder	LN1, attention path, residual 1, LN2, MLP path, residual 2	20	5 + 20L đến 24 + 20L
12 encoder	Lặp encoder với $L = 0 \dots 11$	240	5–244
Final	Final LN, select CLS, classifier, Argmax, class Softmax tùy chọn	5	245–249

Mỗi descriptor rộng 512 bit, tương đương 16 word u32. Địa chỉ tensor trong descriptor là logical word address, không phải byte address. Nhóm trường quan trọng được tóm tắt như sau.

Nhóm word	Nội dung	Ý nghĩa workload
0–1	Header và route	Opcode, subop, cờ, tag, tensor/memory space nguồn–đích
2–5	src0, src1, src2, dst base	Xác định tensor và vùng dữ liệu của command
6–9	dim0...dim3	GEMM dùng batch, M, K, N; non-GEMM dùng shape phù hợp opcode
10–14	stride0...stride4	Địa chỉ hóa batch, row, packed-B hoặc layout iterator
15	immediate	C-row stride của GEMM, scalar Vector hoặc epsilon LayerNorm

Các flag cần nêu trong Chương 3 là BIAS_ENABLE, MASK_ENABLE (tắt ở attention template M8), B_BLOCKED_K16_N2, GEMM_FP16 và B_FP16_PACKED2. Chi tiết bit field, handshake và lỗi admission được trình bày ở Chương 4; ở mức workload, descriptor là hợp đồng liên kết tensor shape với engine và vị trí dữ liệu.

3.5 Không gian dữ liệu và vòng đời tensor

3.5.1 MODEL

MODEL chứa package đã khóa: persistent GEMM-B, bias, CLS token, positional embedding, gamma/beta LayerNorm và các tham số cần cho engine. Đối với NPU adapter, MODEL là vùng chỉ đọc trong toàn bộ job. Model được nạp trước START và không được ghi đè theo từng layer hoặc từng ảnh.

3.5.2 INPUT

INPUT chứa đúng một prepared_input.bin, tức PATCH_A [1,196,768] gồm 150.528 word FP32. Host thay INPUT cho ảnh kế tiếp chỉ khi job trước đã quiescent; PL chỉ đọc vùng này ở patch projection. INPUT không phải nơi chứa ảnh JPEG/PNG gốc và không được NPU ghi.

3.5.3 TABLE/command parameter

TABLE là blob staging phía host, gồm header và 200 entry parameter. Nó không phải một selector dữ liệu của M_AXI mà NPU đọc liên tục trong khi inference. Trước START, host đọc các entry này, kiểm tra metadata/offset rồi nạp 8 global offset và 192 layer offset vào thanh ghi parameter table qua AXI-Lite. Sequencer sau đó sử dụng trạng thái parameter trong PL để tạo descriptor. Cách mô tả này tránh nhầm TABLE với một tensor trong SCRATCH.

3.5.4 SCRATCH/intermediate tensor

SCRATCH là workspace FP32 đọc/ghi duy nhất của NPU. Schedule cố định cho phép tái sử dụng vùng nhớ sau consumer cuối cùng thay vì cấp một bản sao tensor cho mọi layer. Các offset dưới đây là logical u32 offset từ SCRATCH_BASE; physical DDR base và bounds được trình bày ở Chương 5.

Vùng SCRATCH	Offset u32	Nội dung chính	Ví dụ vòng đời
HIDDEN_A	0x000000	Token matrix [197,768]	Nhận CLS+patch, positional add; là output residual 2 của mỗi encoder
HIDDEN_B	0x025000	Token matrix [197,768]	Output LN1 hoặc output projection/residual 1; được tái dùng theo pha
LINEAR_TMP	0x04A000	Linear/K ^T /merged-head/CLS tạm	Patch output, Q/K/V projection, K transpose, head merge, FC2 output và CLS cuối

Q_HEAD	0x06F000	Query theo head [12,197,64]	Được tạo sau Q projection và có thể tái dùng sau P×V
K_HEAD	0x094000	Key theo head [12,197,64]	Được tạo sau K projection; Layout đọc để ghi K^T vào LINEAR_TMP rồi vùng này hết hạn
V_HEAD	0x0B9000	Value theo head [12,197,64]	Tồn tại đến khi P×V hoàn tất
SCORE_PROB	0x0DE000	Score/probability [12,197,197]	QK^T ghi score; Vector và Softmax cập nhật in-place; P×V đọc
FC1	0x150000	MLP mở rộng [197,3072]	FC1 ghi, GELU ghi đè, FC2 đọc
LOGITS	0x1E4000	1.000 logit cuối	Classifier ghi; Argmax và class Softmax đọc
CLASS_PROB	0x1E5000	1.000 xác suất tùy chọn	Chỉ class Softmax ghi khi được bật

Ví dụ tái sử dụng trong một encoder là: $HIDDEN_A \rightarrow LN1 \rightarrow HIDDEN_B$; mỗi projection lần lượt dùng LINEAR_TMP rồi Layout tách vào Q/K/V; SCORE_PROB lần lượt mang score và probability; FC1 được GELU ghi đè; cuối cùng residual 2 ghi lại HIDDEN_A. Tái sử dụng này là hợp lệ vì schedule bảo đảm producer/consumer theo thứ tự; nó không suy ra rằng mọi alias tùy ý đều an toàn.

3.5.5 LOGITS, PROBS và đầu ra phân loại

LOGITS và PROBS là hai subregion có chủ ý của SCRATCH, không phải hai vùng dữ liệu độc lập với SCRATCH. Classifier ghi 1.000 word FP32 vào LOGITS. Argmax quét LOGITS và trả class_index cùng class_logit qua control sideband, không tạo tensor DDR mới. Khi class Softmax bật, engine đọc LOGITS và ghi 1.000 word FP32 vào CLASS_PROB; host chỉ thu thập output sau khi job kết thúc.

3.6 Phân hoạch host, PS và PL

Phân hoạch hiện tại được tóm tắt trong bảng sau. Mục đích của bảng là xác định ownership, không phải mô tả một standalone software runtime chưa được triển khai.

Thành phần	Trách nhiệm trong M8	Không thực hiện trong inner loop inference
------------	----------------------	--

Host tools	Decode/preprocess ảnh, tạo prepared_input.bin, nạp asset, kiểm tra manifest/hash, lưu log/output	Không nhân GEMM, Softmax, LayerNorm hoặc MLP cho job đang chạy
PS và control path	Khởi tạo board/DDR, cấu hình thanh ghi, phát START, poll trạng thái/IRQ, đọc output và counter	Không điều phối từng layer sau START
PL NPU	Sequencer, command controller, GEMM, six non-GEMM engine, memory frontend và AXI adapter	Không decode JPEG/PNG hoặc tiền xử lý ảnh thô
DDR	Lưu MODEL, INPUT, TABLE staging và SCRATCH/output	Không tự quyết định schedule hoặc chuyển kiểu dữ liệu

Trong flow hiện tại, host/XSCT qua JTAG điều phối job và FSBL trên A53 hỗ trợ khởi tạo PS/DDR. Không nên mô tả đây là A53 runtime standalone đã được xác nhận. Sau accepted START, PL tự chạy microprogram cố định; host/PS chỉ theo dõi trạng thái và thu nhận dữ liệu sau terminal state.

Về ownership, START chụp memory base/limit, global configuration và job configuration. Trong thời gian BUSY, protocol phần mềm không ghi đè MODEL, INPUT, SCRATCH hoặc TABLE staging; host chỉ poll control/status. Đây là quy tắc vận hành cần giữ trong báo cáo, không phải khẳng định rằng adapter M8 là một hardware firewall chặn mọi master khác của hệ thống ghi DDR.

3.7 Ánh xạ kiểu dữ liệu theo từng đường dữ liệu

Mục này tóm tắt việc ánh xạ precision theo workload; lý thuyết RNE, accumulator và các nguồn sai khác số học đã được giải thích ở Mục 2.3.

Đường dữ liệu	Kiểu lưu trữ	Kiểu xử lý/đầu ra	Ghi chú
Ảnh sau processor	FP32	FP32	pixel_values [1,3,224,224] ở host trước patchify
PATCH_A, HIDDEN, Q/K/V, score/prob, FC1 và output	FP32	FP32 ngoài GEMM ingress	Các tensor động giữ ở INPUT/SCRATCH dưới dạng word FP32
Persistent GEMM-B	Packed FP16 trong u32	FP16 term của GEMM	Mode 3 giải nén half trực tiếp từ MODEL

Activation A và B động của GEMM	FP32 trong SCRATCH	RNE FP32→FP16 tại ingress	QK^T và $P \times V$ dùng B động, không dùng persistent packed-B
Dot product và bias/output GEMM	Internal wide accumulator, rồi FP32	FP32	Kết quả cuối dot được chuẩn hóa về FP32 trước scratch writeback
Vector, Layout, LayerNorm, Softmax, GELU, Argmax	FP32	FP32	Layout copy bit-preserving, không ép kiểu
AXI/DDR transfer	Logical u32 opaque	Không chuyển kiểu	Adapter đóng gói/unpack beat AXI; converter nằm ở engine ingress

3.8 Ví dụ minh họa luồng xử lý từ ảnh thô

3.8.1 Mục tiêu, phạm vi và input mẫu

Ví dụ này theo dõi một ảnh ký hiệu I* từ file ảnh gốc đến sideband class/logit và các vùng output. Trong bản nộp cuối, thay I* bằng một ảnh thuộc snapshot M8 đã khóa và điền image ID, SHA-256 ảnh nguồn, SHA-256 prepared_input.bin, processor hash cùng đường dẫn log tương ứng. Ví dụ chỉ minh họa luồng dữ liệu và command; không đưa nhãn dự đoán hoặc số liệu kết quả vào chương này.

3.8.2 Định dạng ảnh gốc, resize, crop và chuẩn hóa

Host mở ảnh I*, decode bằng thư viện ảnh và ép về RGB ba kênh. Image processor đã khóa của dự án thực hiện resize bilinear về 224×224 , rescale theo 1/255 và normalize từng kênh với mean/std 0,5/0,5, tạo pixel_values FP32 có shape [1,3,224,224]. Luồng hiện tại không có một center-crop riêng; tiêu đề mục này được giữ để ghi rõ rằng crop không phải một bước độc lập trong preprocessing M8.

Các tham số processor phải được trích từ processor configuration/hash đi kèm campaign cuối, thay vì viết lại bằng một transform tự chọn. Điều này bảo đảm preprocess của ảnh minh họa, input đã nạp DDR và reference cùng dùng một contract.

3.8.3 Tạo tensor đầu vào và ghi vào vùng INPUT

pixel_values được patchify bằng unfold với kernel 16×16 và stride 16×16 , sau đó transpose thành PATCH_A [1,196,768]. Thứ tự phần tử được khóa là patch_y, patch_x, channel, kernel_y, kernel_x. Tensor được flatten thành FP32 little-endian, tương ứng 150.528 word hoặc 602.112 byte, và được lưu dưới tên prepared_input.bin.

Trước START, host kiểm tra manifest/hash của input rồi nạp payload vào INPUT. Trong retained-DDR flow, model và parameter table được giữ lại qua nhiều ảnh; mỗi job chỉ thay prepared_input.bin, xóa

scratch cần thiết, nạp configuration/offset và khởi động lại job sau khi NPU đã idle. Không gọi bước này là DMA: flow hiện tại dùng host/XSCT để ghi payload vào DDR.

3.8.4 Patchify, patch projection, CLS và positional embedding

Command đầu tiên đọc PATCH_A từ INPUT, đọc patch weight/bias từ MODEL và thực hiện patch-projection GEMM, ghi [196,768] vào LINEAR_TMP. Hai Layout command sau đó copy CLS token từ MODEL vào token 0 của HIDDEN_A và copy 196 patch embedding vào token 1...196. Vector command positional-add cộng positional embedding vào HIDDEN_A để tạo input [197,768] cho encoder đầu tiên.

3.8.5 Một encoder block minh họa

Với encoder thứ 1, sequencer phát 20 command theo thứ tự cố định. HIDDEN_A đi qua LN1 thành HIDDEN_B; ba GEMM Q/K/V lần lượt dùng LINEAR_TMP và Layout để tạo Q_HEAD, K_HEAD, V_HEAD. K được transpose, QK^T sinh SCORE_PROB, sau đó Vector nhân với $1/\sqrt{64} = 0,125$ (mask bị tắt) và Softmax cập nhật vùng này thành attention probability. GEMM $P \times V$, Layout merge-head và output projection tạo đầu vào cho residual 1. Sau LN2, FC1, GELU, FC2 và residual 2, output encoder trở lại HIDDEN_A để làm input cho encoder kế tiếp.

HIDDEN_A → LN1 → HIDDEN_B
→ Q/K/V GEMM + Layout → Q_HEAD/K_HEAD/V_HEAD
→ QK^T → scale '0,125' (mask disabled) → Softmax → $P \times V$ → merge → O projection
→ residual 1 → LN2 → FC1 → GELU → FC2 → residual 2 → HIDDEN_A

3.8.6 Final LayerNorm, classifier và Argmax

Sau encoder thứ 12, HIDDEN_A [197,768] đi qua final LayerNorm để tạo HIDDEN_B. Layout select-CLS copy token đầu tiên sang LINEAR_TMP [1,768]. Classifier GEMM ghi 1.000 logits FP32 vào LOGITS. Argmax quét LOGITS theo so sánh numeric FP32 và trả index/logit qua sideband. Nếu class_softmax_enable được bật, class Softmax chạy sau Argmax và ghi 1.000 probability FP32 vào CLASS_PROB.

3.8.7 Thu nhận logits/probability/nhãn dự đoán

Khi sequencer báo terminal state, host đọc status/error/counter và sideband class. Logits luôn có thể được thu từ LOGITS; probabilities chỉ được thu khi class Softmax được bật. Các record cần liên kết với ảnh I* gồm manifest preprocessing, input record, transcript control, output dump, status cuối và hash/log của artifact. Việc đối chiếu nhãn tham chiếu, Top-1 và numerical vector được trình bày ở Chương 6, không lặp lại trong ví dụ này.

3.8.8 Bảng trace ảnh thô → tensor → command → engine → đầu ra

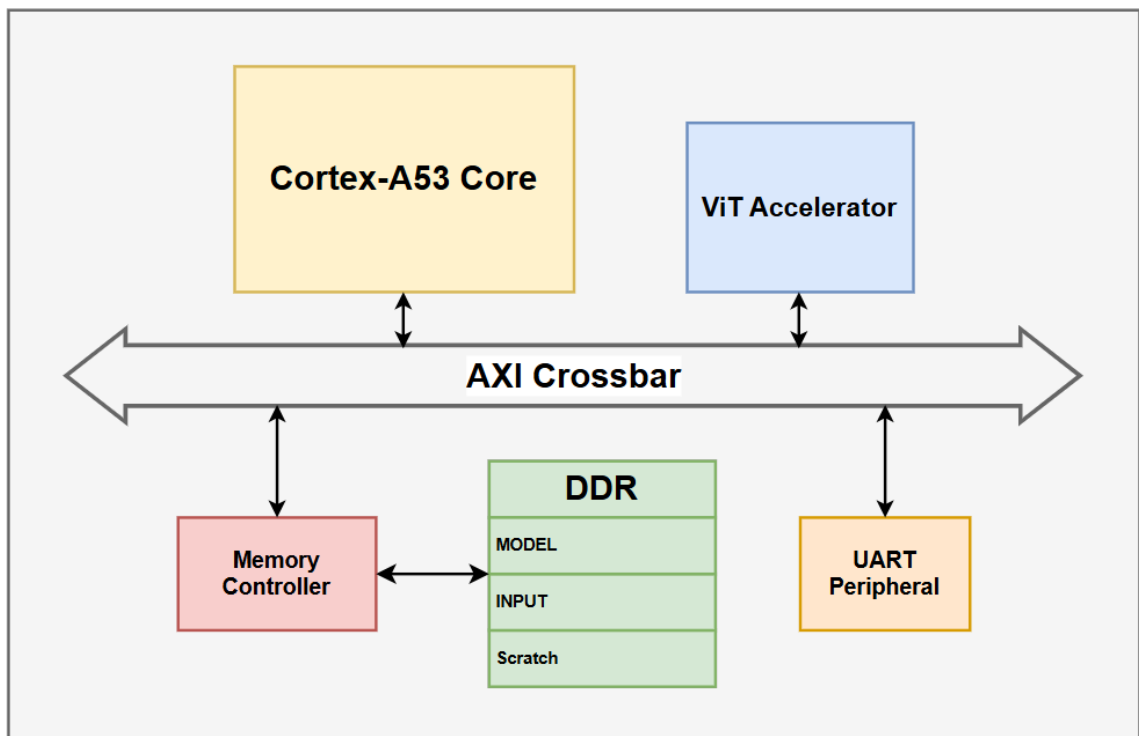
Bước	Dữ liệu vào	Biến đổi	Tensor/vùng nhớ ra	Command/engine	Bảng chứng cần lưu

1	File ảnh I*	Decode, RGB, resize/rescale/normalize	pixel_values [1,3,224,224] tại host	Python image processor	Source-image hash, processor hash, preprocess manifest
2	pixel_values	Patchify 16×16, stride 16, flatten FP32 LE	PATCH_A [1,196,768] / prepared_input.bin	Host preprocessing	Input SHA-256, shape, word count, byte order
3	prepared_input.bin, MODEL, TABLE	Nạp input; nạp/kiểm tra config; START	INPUT, parameter registers, SCRATCH sẵn sàng	Host/PS control	Input record, register readback, control transcript
4	PATCH_A, patch weight/bias	Patch projection; ghép CLS/patch; positional add	HIDDEN_A [197,768]	GEMM, Layout, Vector	Command ordinal 1–4 và trace tensor
5	HIDDEN_A	LN1, Q/K/V, head transform, attention, residual 1	HIDDEN_B và head/score workspace	LayerNorm, GEMM, Layout, Vector, Softmax	Command trace của encoder 1
6	Attention output	LN2, FC1, GELU, FC2, residual 2	HIDDEN_A [197,768] cho encoder kế tiếp	LayerNorm, GEMM, GELU, Vector	Command trace và checkpoint tensor nếu bật
7	HIDDEN_A sau encoder 12	Final LN, select CLS, classifier, Argmax, optional Softmax	LOGITS, class sideband, optional CLASS_PROB	LayerNorm, Layout, GEMM, Argmax, Softmax	Terminal status, output dump, artifact hash

8	Sideband/output regions	Đọc sau DONE và lưu record	Record gắn với I*	Host/PS control	Result record, counter CSV, transcript
---	-------------------------	----------------------------	-------------------	-----------------	--

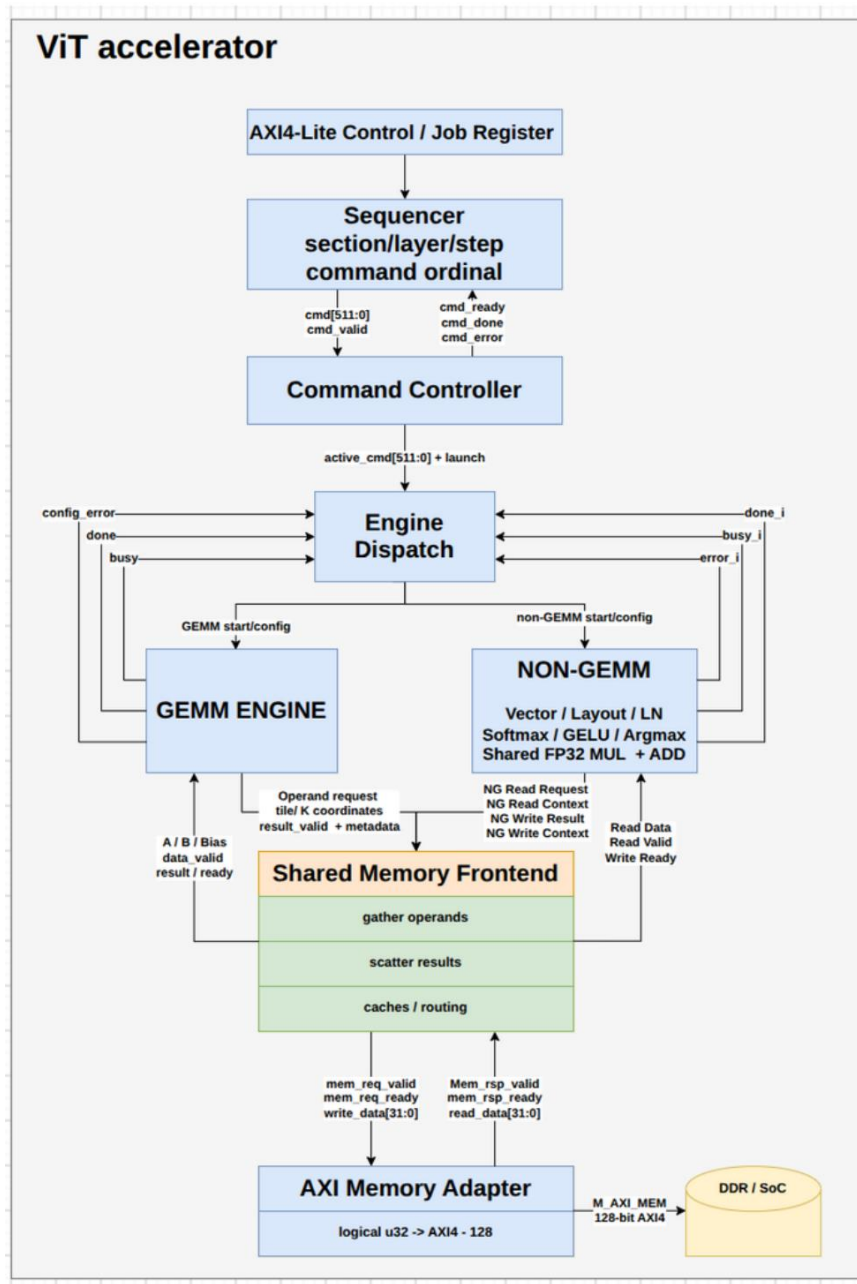
CHƯƠNG 4 — KIẾN TRÚC PHẦN CỨNG

Overall SoC architecture



4.1 Sơ đồ top-level NPU

- M8 là NPU chuyên dụng cho microprogram ViT-B/16 đã khóa, không phải một mảng tính toán được lập trình tự do ở thời điểm chạy. Trong PL, wrapper AXI bao quanh ba khối có chức năng khác nhau: control plane AXI4-Lite và parameter table, NPU thực thi command, và AXI memory adapter nối NPU với DDR. Bên trong NPU, sequencer sinh descriptor; engine top nhận descriptor, chọn đúng một engine và điều phối đọc/ghi qua memory frontend.
- Hai đường giao tiếp được tách biệt. AXI4-Lite dùng để cấu hình và thu trạng thái; đường M_AXI của NPU dùng cho tensor và tham số trong DDR. busy của NPU là OR của busy sequencer và busy engine. Không có command queue ẩn: tại một thời điểm chỉ một descriptor được active và dispatch chỉ khởi động một trong bảy compute engine (GEMM cộng sáu non-GEMM).



Hình 4.1. Kiến trúc top-level ViT accelerator: control plane, compute engines, Shared Memory Frontend và AXI Memory Adapter.

Khối	Vai trò thực tế	Không làm
AXI wrapper/control	Giữ CSR, snapshot job khi START, layer parameter table, status/IRQ	Không thực hiện phép toán tensor
Sequencer	Sinh descriptor theo pha/layer/bước ViT	Không tự đọc/ghi tensor DDR

Command controller/dispatch	Latch command, chọn engine, hợp nhất done/error	Không có FIFO tensor riêng
Compute engine	Thực hiện thuật toán của opcode đang active	Không tự sở hữu M_AXI transaction
Memory frontend/adapter	Chuyển yêu cầu logic thành đọc/ghi DDR và trả dữ liệu cho engine	Không quyết định topology ViT

4.2 Control plane

4.2.1 Thanh ghi điều khiển và trạng thái

Giao diện control là AXI4-Lite 32 bit, aperture PS-visible 0xA0000000...0xA0000FFF. Cổng chỉ cho tối đa một read và một write outstanding. Địa chỉ ghi AW và dữ liệu W có thể đến ở các chu kỳ khác nhau; register bank chỉ commit sau khi đã nhận cả hai, rồi giữ BVALID/BRESP đến khi master nhận response. Đọc cũng giữ RVALID/RDATA/RRESP khi bị backpressure. Vì vậy driver không được giả định AW và W cùng chu kỳ, hoặc ARREADY đồng nghĩa dữ liệu đọc xuất hiện ngay.

Nhóm CSR	Mục đích	Ý nghĩa khi chạy M8
IP_ID, IP_VERSION	Nhận diện ABI	Driver kiểm đúng phiên bản trước khi cấu hình job
CONTROL, STATUS	START, soft reset, clear error; idle/busy/done/error/IRQ	Là giao diện vòng đời một job
IRQ_ENABLE, IRQ_STATUS, ERROR_*	Ngắt và lỗi sticky	Phân biệt completion bình thường với lỗi cần recovery
Base/count MODEL, INPUT, SCRATCH	Khai báo byte base và kích thước theo word	Là context để adapter kiểm bounds cho descriptor logical address

Execution mode và global offsets	Chọn mode package, địa chỉ CLS/POS/weight/bias dùng chung	Mode hợp lệ tại START là exact 3 hoặc 5; M8 final dùng mode 3 packed-B
Job config, PATCH_A base, class result	Cấu hình phạm vi job và đọc kết quả Argmax sideband	Không thay thế LOGITS/PROBS trong DDR
Layer parameter table	192 layer offset cùng 8 global offset	Là metadata để sequencer materialize descriptor theo layer

CONTROL[0..3] lần lượt là START, SOFT_RESET, ABORT và CLEAR_ERROR. START là pulse; soft reset chỉ được nhận khi idle; ABORT hiện được khai báo nhưng không hỗ trợ. STATUS công bố idle, busy, done sticky, error sticky, IRQ và fallback-wait. Một AXI-Lite write trả OKAY chỉ nói transaction bus hợp lệ, không đồng nghĩa START được chấp nhận.

START chỉ tạo job khi wrapper đang idle, không ở recovery, execution mode hợp lệ và, với mode 3, MODEL base thỏa căn hàng 128 byte. Khi accepted, wrapper snapshot base/count MODEL-INPUT-SCRATCH, mode, global offset và job configuration thành active-job context. Vì vậy host không được sửa cấu hình trong lúc BUSY. Layer table là trường hợp đặc biệt: nó được sequencer đọc live nên write vào table bị khóa khi busy; các CSR cấu hình khác là shadow cho job kế tiếp.

4.2.2 Sequencer

Sequencer là microprogram controller cho graph ViT cố định. Nó không nhận graph PyTorch hay biên dịch lệnh mới trong inference. Với class Softmax bật, nó phát bốn command embedding, 20 command cho mỗi trong 12 encoder, rồi năm command cuối: tổng cộng 249 descriptor. Nếu class Softmax tắt, command tùy chọn cuối không phát và tổng là 248.

IDLE → LOAD_LAYER → ISSUE → WAIT_COMMAND → CHECKPOINT? → ADVANCE → DONE

State	Hành vi	Điều kiện chuyển quan trọng
IDLE	Chờ job_valid, snapshot context ban đầu	Reject pha/layer không hợp lệ trước khi phát command

LOAD_LAYER	Yêu cầu và latch 16-word parameter của encoder hiện tại	Giữ request đến khi parameter valid
ISSUE	Sinh descriptor 512 bit từ phase/layer/step đã đăng ký	Giữ cmd_valid và descriptor ổn định đến cmd_ready
WAIT_COMMA ND	Chờ controller retire command	cmd_error có ưu tiên cao hơn cmd_done
CHECKPOINT	Tùy chọn xuất metadata trace	Có thể stall theo checkpoint_ready
ADVANCE	Tăng step, layer hoặc chuyển phase	Quyết định kết thúc encoder/final theo schedule cố định
DONE	Terminal state một chu kỳ của job	Trả về IDLE sau khi wrapper quan sát terminal state

Sequencer không phát descriptor tiếp theo khi command hiện tại chưa done hoặc error. Điều này tạo thứ tự producer–consumer an toàn cho các vùng SCRATCH tái sử dụng ở Chương 3. Descriptor được tạo combinational từ context đã latch, nên vẫn ổn định trong toàn bộ thời gian ISSUE && !cmd_ready.

4.2.3 Command controller

Command controller nhận một descriptor từ sequencer và biến nó thành một transaction engine đơn. FSM gồm năm state:

IDLE → WAIT_PARAMETER? → LAUNCH → EXECUTE → REPORT → IDLE

State	Trách nhiệm
IDLE	Chỉ state có cmd_ready; latch một active_cmd khi accept
WAIT_PARAMETER	Giữ command nếu source route cần parameter và parameter chưa sẵn sàng
LAUNCH	Kiểm opcode, phát start cho engine được chọn
EXECUTE	Giữ engine active, chọn done/error/data request của đúng engine
REPORT	Phát đúng một pulse cmd_done hoặc cmd_error, rồi trở về IDLE

Parameter gate chỉ điều kiện hóa thời điểm launch; nó không bulk-transfer weight hay gamma/beta vào controller. Dữ liệu PARAM vẫn đi qua memory frontend và M_AXI khi engine cần. active_cmd giữ ổn định trong lúc controller busy, tránh việc một descriptor sau thay đổi base/shape của command đang chạy.

4.2.4 Command descriptor và cơ chế dispatch

Mỗi command là một phase_e_cmd_t rộng 512 bit gồm 16 word u32. Địa chỉ trong descriptor là **logical word address**, không phải byte address; adapter mới tạo địa chỉ vật lý base + (word_address << 2) sau khi kiểm route và bounds.

Word	Nhóm trường	Vai trò
0	Header	Opcode, subop, flags, tag và context phase/layer
1	Route	Tensor/memory space của src0, src1, src2, dst và step metadata
2–5	Base	Logical base của ba nguồn và một đích
6–9	Dimension	GEMM: batch,M,K,N; non-GEMM: shape theo opcode
10–14	Stride	Batch/row/tile stride hoặc iterator layout
15	Immediate	GEMM C-row stride, Vector scalar, LayerNorm epsilon, ...

Dispatch decode opcode 1...7 thành GEMM, Vector, Layout, LayerNorm, Softmax, GELU hoặc Argmax. Nó mux selected_done, selected_error, data_request, result_valid và số word native read/write của engine được chọn. Opcode không hợp lệ là lỗi local; Argmax còn đưa cờ nonfinite từ engine vào selected_error. Một descriptor không khởi động nhiều engine đồng thời.

Các cờ đặc biệt cần diễn giải theo đúng phạm vi: BIAS_ENABLE là epilogue của GEMM; B_BLOCKED_K16_N2 và B_FP16_PACKED2 áp dụng persistent packed-B; GEMM_FP16 chọn datapath GEMM FP16; MASK_ENABLE là khả năng Vector nhưng attention M8 đặt cờ này bằng 0; IN_PLACE chỉ là metadata, an toàn alias vẫn đến từ schedule và gather-before-scatter.

4.2.5 Handshake, lỗi và quy tắc hoàn tất command

Tất cả interface nội bộ dùng quy tắc valid/ready: producer giữ payload ổn định khi valid=1 và ready=0; consumer chỉ lấy dữ liệu ở cạnh có valid && ready. Các stall có thể kéo dài tùy ý; sequencer, controller và frontend không có watchdog/timeout nội bộ. Vì vậy liveness của job phụ thuộc engine, parameter path và AXI phản hồi theo hợp đồng.

Sự kiện	Quy tắc M8
Command ingress	Sequencer giữ descriptor đến cmd_ready; controller chỉ accept tại IDLE
Operand ingress	Frontend gather đủ bundle native rồi mới pulse *_data_valid cho engine
Result egress	Engine giữ result_valid; frontend scatter đủ output, nhận response cuối, sau đó mới pulse *_result_ready
Completion	Controller chỉ retire command sau selected_done; vì thế command không DONE trước write cuối
Memory error	memory_error_latched có ưu tiên hơn selected done trong EXECUTE
Error propagation	Adapter/frontend → controller cmd_error → sequencer terminal error context → wrapper sticky error/recovery
Reset/recovery	CLEAR_ERROR không tự xóa mọi trạng thái recovery; sau lỗi framing/protocol cần idle rồi soft reset trước job mới

Kết quả numerical lỗi của GEMM bị scheduler chặn khỏi đường store; error engine hoặc error memory đều trở thành command error. Đối với Argmax, class index/logit là sideband độc lập với writeback tensor. cmd_done và cmd_error là pulse mutually exclusive trong REPORT; nếu controller thấy trạng thái illegal, opcode invalid hoặc selected error thì đi REPORT-error thay vì im lặng advance schedule.

4.3 Khuôn giải thích bắt buộc cho mỗi thuật toán/phần tử xử lý

Để phần kiến trúc có thể đối chiếu trực tiếp với RTL, mỗi engine bên dưới được mô tả theo cùng một khuôn. Khuôn này tách rõ *phép toán logic của ViT* với *trình tự vi kiến trúc thực tế*, tránh suy diễn một công thức toán học thành một datapath không tồn tại.

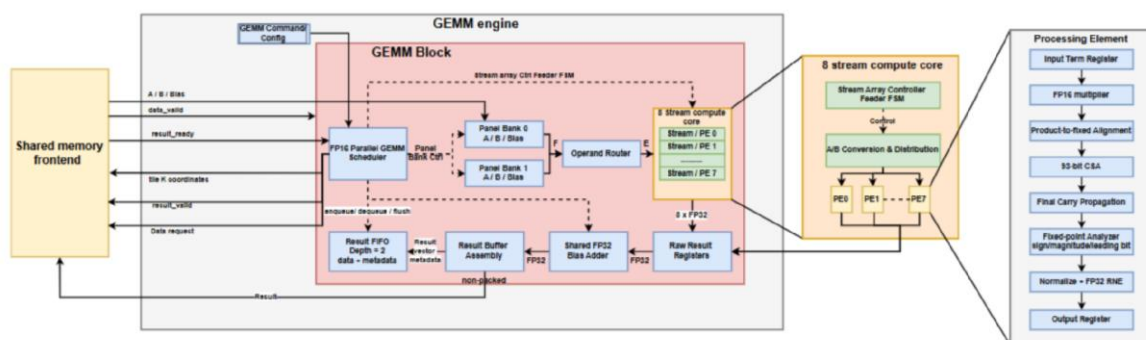
Thành phần cần có	Câu hỏi phải trả lời trong báo cáo
Vai trò workload	Nó nằm ở đâu trong ViT và command template nào gọi nó?
Tensor/shape	Đọc tensor nào, ghi tensor nào, route MODEL/INPUT/SCRATCH/PARAM ra sao?
Công thức	Thứ tự chính xác của FP32_ADD, FP32_MUL, RNE hoặc approximant là gì?

Dữ liệu và buffer	Mỗi packet/panel chứa bao nhiêu word, buffer nào giữ input/output, có in-place hay không?
Điều khiển	State hoặc vòng lặp chính, valid/ready, điều kiện done/error và backpressure?
Địa chỉ/layout	Iterator, stride, packed-half hoặc output address được tạo như thế nào?
Ví dụ vi mô	Một tile, packet, token hoặc row thực tế theo shape M8 ra sao?
Biên và kiểm chứng	Tail mask, NaN/Inf/subnormal, overflow, alias và artifact/RTL dùng để kiểm tra?

Ký hiệu FP32_ADD và FP32_MUL trong chương này chỉ leaf arithmetic của M8, không mặc nhiên đồng nghĩa IEEE 754 đầy đủ trong mọi corner case. Trên đường normal chúng làm tròn RNE, nhưng adder xử lý operand exponent-zero như zero còn multiplier flush subnormal input/underflow thành signed zero. Các công thức vì vậy phải được đọc cùng Mục 2.3, không thay reference RTL bằng một lời gọi thư viện floating-point tổng quát.

4.4 GEMM engine và các thuật toán GEMM

4.4.1 Vai trò và taxonomy GEMM trong ViT



Hình 4.2. Kiến trúc GEMM engine gồm scheduler, hai panel bank ping-pong, tám FP16 dot-product stream và datapath của một Processing Element.

GEMM là engine duy nhất thực hiện các phép tích ma trận trong ViT. Exact M8 chỉ elaborates nhánh GEMM FP16 stream; nhánh GEMM FP32 legacy bị loại ở build bằng INCLUDE_LEGACY_GEMM=0. Vì vậy không mô tả M8 như một systolic array FP32 cũ hoặc như hai datapath GEMM cùng chạy.

Với descriptor batch,M,K,N, phép logic là:

$$C[b,m,n] = \text{FP32_ADD}(\text{RNE32}(\sum_{k=0}^{K-1} \text{FP16}(A[b,m,k]) \times \text{FP16}(B[b,k,n])), \text{bias}[n])$$

trong đó hạng bias chỉ tồn tại khi BIAS_ENABLE=1. Mọi output động vẫn được ghi FP32. batch=12 của QK^T/PV là 12 attention head độc lập, không phải image batch 12.

GEMM	batch,M,K,N	B và bias	Số command trong full schedule	Đích chính
Patch projection	1,196,768,768	Persistent packed-FP16, có bias	1	LINEAR_TMP
Q projection	1,197,768,768	Persistent packed-FP16, có bias	12	LINEAR_TMP → Q_HEAD
K projection	1,197,768,768	Persistent packed-FP16, có bias	12	LINEAR_TMP → K_HEAD
V projection	1,197,768,768	Persistent packed-FP16, có bias	12	LINEAR_TMP → V_HEAD
QK^T	12,197,64,197	B động FP32 trong SCRATCH, không bias	12	SCORE_PROB
$P \times V$	12,197,197,64	B động FP32 trong SCRATCH, không bias	12	Q_HEAD tái sử dụng
Output projection	1,197,768,768	Persistent packed-FP16, có bias	12	HIDDEN_B
FC1	1,197,768,3072	Persistent packed-FP16, có bias	12	FC1
FC2	1,197,3072,768	Persistent packed-FP16, có bias	12	LINEAR_TMP

Classifier	1,1,768,1000	Persistent packed-FP16, có bias	1	LOGITS
------------	--------------	---------------------------------------	---	--------

Tổng là 98 GEMM command. Các projection/MLP/classifier dùng persistent B trong MODEL qua parameter route; QK^T/PV dùng B động trong SCRATCH. Khác biệt này quyết định đường nạp panel, không làm thay đổi mảng tám stream ở lõi tính.

4.4.2 Hình học logical tile và ánh xạ sang lượt tính vật lý

M8 khóa ARRAY_ROWS=8, ARRAY_COLS=2, PE_LANES=16, FP16_STREAMS=8. Tên đúng của geometry là logical tile R8/C2/K16: tối đa tám hàng output, hai cột output logic và 16 phần tử K trong một panel. Tuy nhiên chỉ có tám dot stream vật lý, nên một tile R8×C2 được hoàn tất bởi **hai** pass R8×C1, không phải 16 dot product song song.

A panel: $R8 \times K16$

packed B panel: $K16 \times C2$

Pass 0: chọn cột n

8 stream $\rightarrow C[m:m+7,n]$

Giữ tile context

rewind K về 0

Pass 1: chọn cột n+1

8 stream $\rightarrow C[m:m+7,n+1]$

n += 2; rồi m += 8; rồi batch

PE_LANES=16 chỉ là chiều dài panel K; một stream không thực hiện 16 phép nhân trong một chu kỳ. Ở mỗi compute-active cycle, mỗi trong tám stream nhận nhiều nhất một pair FP16 cho lane K hiện tại. Khi N lẻ, pass cột thứ hai bị mask; khi M hoặc K không đầy, token_valid, output_valid và lane_valid vô hiệu hóa phần tail. Lane K bị disable trước dot, do đó không tạo phép $0 \times \text{Inf}/\text{NaN}$ giả.

4.4.3 Scheduler, operand routing và vòng lặp GEMM

Scheduler duyệt batch, tile hàng m0 theo bước 8, tile cột logic n0 theo bước 2, rồi các panel k0 theo bước 16. Cách viết khái niệm sau biểu diễn đúng thứ tự dữ liệu, còn ready/valid có thể kéo dài mỗi bước:

```
for b in batch:
```

```
  for m0 in range(0, M, 8):
```

```
    for n0 in range(0, N, 2):
```

```

for physical_col in {n0, n0+1 nếu hợp lệ}:
    reset dot context
    for k0 in range(0, K, 16):
        nạp A[8,16], B[16,2] và mask
        phát 16 lane K vào tám stream
    finalize, cộng bias nếu bật, xuất pass hiện tại

```

Operand router tạo bundle theo command route và mask. Với panel đầy, A có $8 \times 16 = 128$ word FP32; B không-packed có $2 \times 16 = 32$ word FP32; B packed có 16 word u32, mỗi word chứa hai half; bias gồm hai word FP32 và chỉ cần ở panel K cuối. Địa chỉ output logic là:

$C_word_address = dst_base + batch * stride4 + row * immediate + column$

Write router chỉ phát phần tử có output mask hợp lệ. Scheduler không advance tile nếu bridge/stream/result store bị stall. Các FSM load panel, start bridge, send chunk, wait result và done đều tuân valid/ready; không có cycle count cố định hay watchdog nội bộ.

4.4.4 Nạp activation, weight và bias

Activation A trong INPUT/SCRATCH bắt đầu ở FP32 và được chuyển FP32→FP16 theo RNE ở ingress stream array. Với B động của QK^T/PV , B FP32 trong SCRATCH cũng đi qua converter đó. Persistent B của mode 3 khác biệt: 74 tensor WEIGHT_B được package v3 lượng từ FP16 RNE và lưu hai half trong một word u32; stream array lấy half trực tiếp, không thực hiện vòng FP16→FP32→FP16. Các tensor model còn lại, input chuẩn bị và scratch vẫn FP32.

Với weight B[K,N], mapping packed $K16 \times N2$ có thể viết:

```

n_tile = n / 2 ; col = n % 2
k_chunk = k / 16 ; lane = k % 16
u32_index = ((n_tile * ceil(K/16) + k_chunk) * 16) + lane
half = low16 khi col=0, high16 khi col=1

```

Một block $K16 \times N2$ vì vậy chứa 32 half trong 16 word. B_BLOCKED_K16_N2 và B_FP16_PACKED2 nói rõ format này; không được suy ra mọi tensor MODEL là FP16.

Activation cache có tám bank hàng, mỗi bank sâu tối đa 3072 word, để replay A qua các cột khi descriptor cho phép cache-safe. Bias cache sâu 3072 word hỗ trợ replay bias. Không có B-cache độc lập: B được xem là bypass traffic. Bias được cộng trong epilogue FP32 của GEMM, không phát thành Vector command riêng.

4.4.5 Packed-B panel và ping-pong buffer

Ping-pong chỉ được bật ở đường B persistent packed (B_FP16_PACKED2=1), không áp dụng cho QK^T/PV. Hai bank panel có ownership rõ ràng:

FREE → RESERVED → READY → COMPUTE → FREE

Producer reserve một bank FREE, giữ request nạp panel đến khi data_valid, sau đó bank trở thành READY. Scheduler có thể claim một bank READY cho stream array trong khi producer đang nạp bank còn lại. Khi chunk_valid && chunk_ready, bank COMPUTE được trả FREE. Như vậy overlap là **nạp B panel kế tiếp với tính panel hiện tại**, không phải hai cột output chạy đồng thời, cũng không phải double-buffer toàn bộ activation hay scratch.

Chỉ có một native DDR load request của producer tại một thời điểm. Nếu cả hai bank READY, scheduler ưu tiên panel K nhỏ hơn; nếu không có panel sẵn sàng, compute chờ. Ownership này giữ payload và tọa độ panel ổn định qua stall, đồng thời tránh dùng nhầm panel thuộc tile/command khác.

4.4.6 FP16 dot-stream array

Stream array có tám instance dot stream độc lập. Trong một pass cột vật lý, stream r nhận activation của hàng m0+r và cùng một weight của cột được chọn:

stream r, lane k: $A[b, m0+r, k] \times B[b, k, n_selected] \rightarrow \text{accumulator } r$

Sau 16 lane của một panel, stream array nhận panel tiếp theo; sau lane cuối cùng của K cuối, mỗi stream trả một result. Result collector đợi đủ stream hợp lệ, sau đó chạy bias epilogue tuần tự cho tám hàng và xuất một tile data 512 bit cùng token/output mask. Nếu output backpressure, toàn bộ tile data, mask và numerical flags giữ ổn định.

Đường A/B có ba dạng ingress: A FP32→FP16 RNE; B động FP32→FP16 RNE; B packed chọn half FP16 trực tiếp. FP16 là 16 bit, không phải lane 8 bit. Các multiplier term của tám stream là riêng; shared FP32 arithmetic ở phần non-GEMM không thay thế chúng.

4.4.7 Thuật toán một dot stream: khởi tạo, multiply, accumulate, kết thúc

Mỗi dot stream có pipeline term accept → FP16 product → alignment → CSA accumulation → finalize → output. Với một dot valid, ý tưởng số học có thể biểu diễn như sau:

```
sum = 0; carry = 0; flags = 0
for k = 0 .. K-1:
    if lane_enable[k]:
        product = FP16_MUL(a[k], b[k])
        addend = align_to_fixed_point(product)
    else:
```

```

    addend = 0
    sum, carry = CSA3_2(sum, carry, addend)
    flags |= product/alignment flags

on TLAST:
    total = carry_propagate(sum, carry)
    out = pack_binary32_RNE(total, flags)

```

CSA thực hiện:

```
sum_next = sum XOR carry XOR addend
```

```
carry_next = ((sum AND carry) OR (sum AND addend) OR (carry AND addend)) << 1
```

M8 dùng accumulator 93 bit, ACC_LSB = -48, và giới hạn K_MAX_TERMS=3072. Quan trọng nhất, TLAST snapshot **cả** sum_next/carry_next trước khi reset accumulator, nên product cuối không bị mất vì thứ tự nonblocking. Sau đó carry-propagate, normalize và RNE sang FP32 chỉ xảy ra một lần cho toàn dot; bias FP32 được cộng sau đó.

Khi out_valid && !out_ready, global pipeline của stream giữ nguyên, đồng thời hạ term_ready; không có skid path cho phép term mới vượt qua output bị chặn. Cờ invalid, overflow/alignment, subnormal-flushed và length-error đi kèm result. Scheduler fail-closed nếu result numerical error, còn disabled tail chỉ đóng góp zero nhưng vẫn giữ semantics TLAST/length.

4.4.8 Accumulation, conversion, rounding và result FIFO

Mixed precision của GEMM không phải FP32 MAC feedback ở từng term. Product FP16 hữu hạn được căn hàng chính xác lên lưới fixed-point, CSA giữ thông tin rộng trong suốt K, rồi kết quả cuối mới chuẩn hóa/RNE sang FP32. Vì vậy nguồn sai khác chính là conversion ingress FP32→FP16, special-value contract và một lần rounding cuối dot, không phải hàng trăm lần làm tròn FP32 accumulator. Chi tiết subnormal và RNE được kế thừa từ Mục 2.3.

Đường packed-B có thêm result FIFO depth 2. Mỗi entry 692 bit snapshot nguyên tử generation, batch/token/output coordinate, absolute word base, masks và tile data 512 bit. Generation tag buộc result cũ không thể dùng destination của command mới. FIFO cho phép stream array enqueue một pass hoàn tất trước khi frontend scatter entry cũ, nhưng packed scheduler chỉ DONE khi FIFO đã drain rỗng.

QK^T/PV là non-packed path: bridge result được giữ trực tiếp đến khi memory frontend nhận result. Không gán result FIFO hoặc ping-pong packed-B cho hai GEMM này.

4.4.9 Đường ghi kết quả và đồng bộ với controller

Sau khi stream array xuất tile, memory frontend snapshot destination identity của result GEMM: base, generation, batch, token/output coordinate và mask. Nó gửi từng word FP32 hợp lệ qua write router và

logical adapter, chờ write response theo contract, rồi mới pulse gemm_result_ready. Snapshot này ngăn tile đang chờ store thừa hưởng dst_base của descriptor kế tiếp.

stream result / FIFO head

- memory frontend result ownership
- write router chọn C_word_address có mask
- logical write request / AXI write response
- result_ready
- scheduler tile advance hoặc command DONE

Memory error, generation mismatch hoặc result-coordinate mismatch được frontend latch và ưu tiên hơn normal done. Command controller reset engine, báo cmd_error; sequencer dừng job ở terminal error. Kết quả là một GEMM chỉ hoàn tất sau khi output hợp lệ đã được scatter theo protocol, không phải ngay sau multiply cuối.

4.4.10 Patch projection GEMM

Patch projection đọc PATCH_A [196,768] từ INPUT, weight/bias patch persistent từ MODEL/PARAM, và ghi LINEAR_TMP [196,768]. Descriptor dùng batch=1, M=196, K=768, N=768; do đó có 48 K16 panel cho mỗi pass cột. Output này chưa phải token sequence hoàn chỉnh: hai Layout command tiếp theo ghép CLS và 196 patch token vào HIDDEN_A, rồi Vector cộng positional embedding.

4.4.11 Q projection GEMM

Q projection nhận HIDDEN_B [197,768] sau LN1, nhân W_q [768,768] và cộng bq [768]. Nó ghi tạm LINEAR_TMP [197,768]; Layout Q-split biến token-major layout thành Q_HEAD [12,197,64]. Q được giữ đến QK^T , nên LINEAR_TMP có thể tái dùng ngay cho K projection.

4.4.12 K projection GEMM

K projection có shape và packed-B path giống Q, nhưng output tạm được Layout thành K_HEAD [12,197,64]. K transpose sau đó đọc K_HEAD và ghi layout semantic K^T [12,64,197] vào LINEAR_TMP; K_HEAD hết vòng đời trước QK^T .

4.4.13 V projection GEMM

V projection cũng lấy HIDDEN_B, dùng W_v /bv, ghi LINEAR_TMP rồi Layout V-split sang V_HEAD [12,197,64]. V_HEAD không được ghi đè cho đến $P \times V$ vì đó là B động của GEMM attention context.

4.4.14 QK^T GEMM

QK^T là GEMM batched theo 12 head: $Q_HEAD [12,197,64] \times K^T [12,64,197] \rightarrow SCORE_PROB [12,197,197]$. B chính là LINEAR_TMP chứa K transpose, không phải weight persistent; bias tắt; A và

B đầu FP32 SCRATCH rồi chuyển FP16 ở ingress. Sau GEMM, Vector nhân toàn SCORE_PROB với $1/\sqrt{64}=0,125$ (mask tắt) và Softmax ghi probability in-place.

4.4.15 PV GEMM

$P \times V$ thực hiện SCORE_PROB [12,197,197] $\times$ V_HEAD [12,197,64] $\rightarrow$ [12,197,64]. Không bias và B là V động trong SCRATCH. Q_HEAD cũ không còn cần sau QK^T , nên output context ghi đè Q_HEAD; Layout head-merge sau đó chuyển nó về LINEAR_TMP [197,768].

4.4.16 Output projection GEMM

Output projection đọc merged context LINEAR_TMP [197,768], persistent Wo [768,768] và bo [768], rồi ghi HIDDEN_B [197,768]. Vùng HIDDEN_B được dùng lại an toàn vì Q/K/V branch đã tiêu thụ LN1 output. Vector residual 1 cộng HIDDEN_A vào HIDDEN_B tại chỗ; LN2 sau đó ghi normalized result sang HIDDEN_A.

4.4.17 FC1 GEMM

FC1 nhận HIDDEN_A [197,768] sau LN2, nhân Wfc1 [768,3072], cộng bfc1 [3072] và ghi FC1 [197,3072]. Đây là GEMM có K lớn nhất trong projection/MLP output path và dùng packed persistent B; GELU đọc/ghi FC1 in-place chỉ sau khi FC1 command đã retire.

4.4.18 FC2 GEMM

FC2 nhận FC1 [197,3072] sau GELU, nhân Wfc2 [3072,768], cộng bfc2 [768] và ghi LINEAR_TMP [197,768]. Vector residual 2 cộng LINEAR_TMP với HIDDEN_B rồi ghi HIDDEN_A, tạo input của encoder kế tiếp hoặc final LayerNorm.

4.4.19 Classifier GEMM

Final LayerNorm ghi HIDDEN_B; Layout select-CLS copy token 0 thành LINEAR_TMP [1,768]. Classifier thực hiện 1×768 nhân 768×1000 , cộng bias 1.000 chiều và ghi LOGITS [1,1000]. Argmax đọc LOGITS ngay sau đó; class Softmax, nếu bật, vẫn đọc cùng LOGITS nên classifier không ghi đè logits.

4.4.20 Ví dụ minh họa một GEMM tile/panel

Pha	A/activation	B/weight	Buffer/FIFO	Tính toán	Kết quả/địa chỉ ghi
1. Chọn tile	Patch projection: m0=0, n0=0, shape toàn lệnh 196×768×768	Weight patch panel K16×N2	Scheduler tạo context logical R8/C2/K16	Chọn tám patch đầu và hai cột embedding đầu	Chưa có output

2. Nạp panel K đầu	A[0:7,0:15]: 128 word FP32 từ INPUT	16 word packed chứa B[0:15,0:1]	Bank 0 FREE→RESERVED →READY	Chưa nạp bias vì chưa phải K panel cuối	Payload panel ổn định chờ compute
3. Pass cột 0	Tám giá trị A[r,k] mỗi lane	Chọn half cột 0 trực tiếp	Bank 0 COMPUTE; tám stream	Với lane k=0...15, stream r cộng A[r,k]×B[k,0] vào CSA	Sau 48 panel, TLAST finalize rồi cộng b[0]
4. Store pass 0	Không đọc A mới sau finalize	Không đọc B mới	Result FIFO packed hoặc result ownership frontend	Tile mask 2'b01; chỉ tám hàng/cột 0 hợp lệ	C[r,0] tại dst + r×768 + 0, r=0...7
5. Pass cột 1	Rewind K về 0, replay A/cache hoặc nạp lại theo router	Chọn half cột 1 của các panel	Ping-pong bank có thể nạp panel kế tiếp song song	Cùng tám stream tính A[r,k]×B[k,1], rồi cộng b[1]	Mask 2'b10; ghi C[r,1]
6. Advance	Sau hai pass tăng n0 += 2; khi hết N tăng m0 += 8	—	FIFO phải drain trước DONE của packed command	Lắp đến output cuối	Tile cuối patch có m0=192, chỉ bốn token hợp lệ

4.5 Non-GEMM engines và các thuật toán non-GEMM

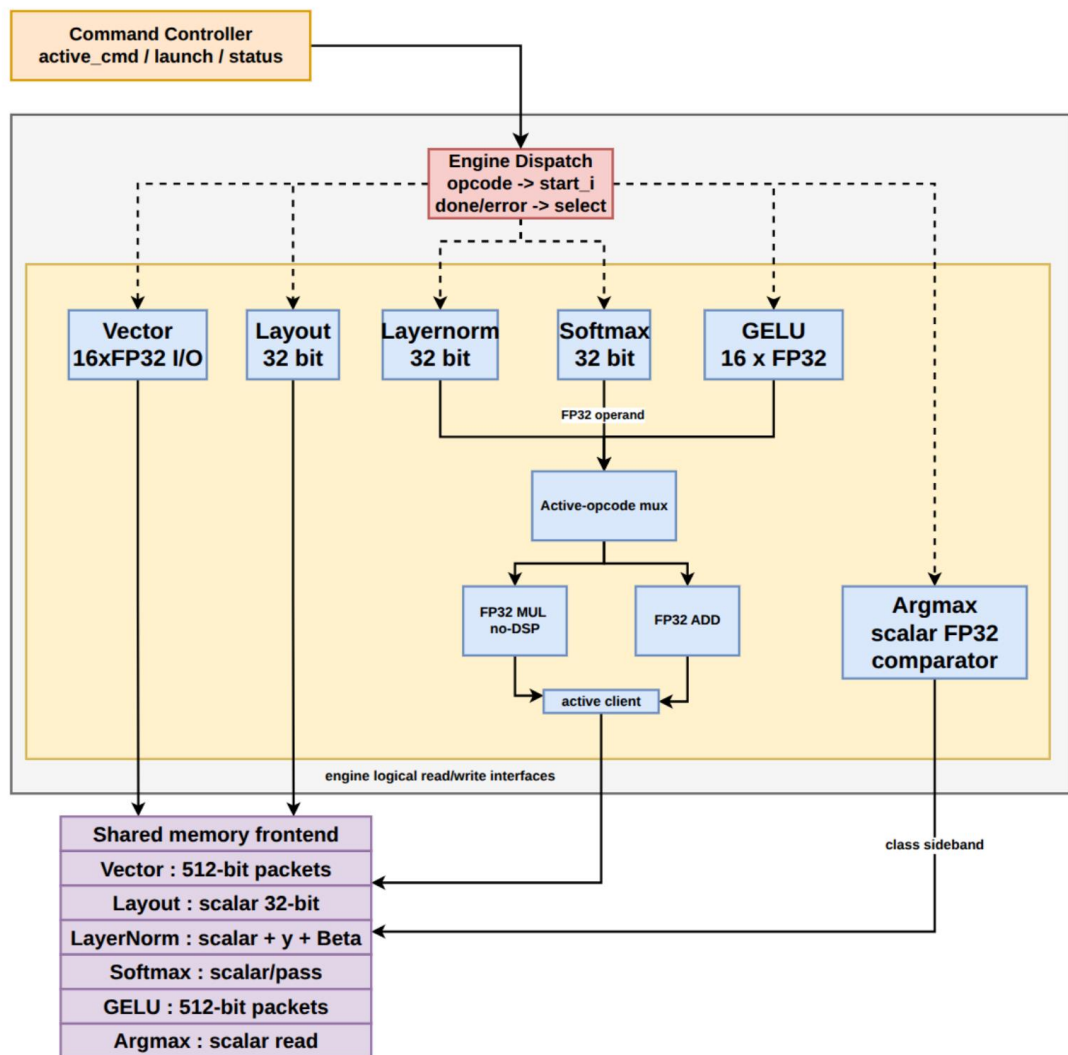
4.5.1 Tổ chức chung, shared arithmetic và buffer cục bộ

Sáu engine non-GEMM hoàn tất phần còn lại của đồ thị ViT: Vector, Layout, LayerNorm, Softmax, GELU và Argmax. Khi class Softmax bật, schedule có 151 command non-GEMM: 37 Vector, 63 Layout, 25 LayerNorm, 13 Softmax, 12 GELU và 1 Argmax. Đây là số instance command, không phải số chu kỳ hay số ALU.

Một command chỉ active một engine. Vector, LayerNorm, Softmax và GELU multiplex một FP32 multiplier và một FP32 adder ở engine top theo thời gian; vì vậy packet 16 word không có nghĩa 16 phép FP32 chạy song song. Layout chỉ copy bits, Argmax chỉ dùng comparator numeric FP32. Các buffer cục bộ giảm DDR reread nhưng không thay đổi công thức/precision contract.

Engine	Buffer/trạng thái cục bộ	Thuật toán chính
Vector	Input/output packet 16 word, lane mask	ADD hoặc SCALE_MASK tuần tự từng lane
Layout	Một word data, iterator và descriptor validator	Copy/permutation theo dim-stride
LayerNorm	Ba BRAM 1024×32: sample, gamma, beta	Mean → biased variance → rsqrt → affine
Softmax	Một BRAM row 1024×32	Max → exp/sum → reciprocal → normalize
GELU	Packet 16 word và scalar core	Erf-style approximant tuần tự từng lane
Argmax	Best value/index, cờ finite/error	Scan strict-maximum 1.000 logit

Giống GEMM, frontend gather đủ input bundle trước khi engine nhận data_valid; khi engine phát result, frontend scatter output và chờ response cuối trước result_ready. Quy tắc này là điều kiện cho các command in-place như SCORE_PROB, FC1 và HIDDEN_A/HIDDEN_B.



Hình 4.3. Kiến trúc cụm Non-GEMM: sáu engine, engine dispatch, shared FP32 arithmetic fabric và Shared Memory Frontend.

4.5.2 Vector engine

Vector có hai subop: ADD và SCALE_MASK. Bias không phải một Vector command: bias của các linear layer đã được GEMM cộng trong epilogue khi BIAS_ENABLE=1.

ADD: $y[i] = \text{FP32_ADD}(a[i], b[i])$

SCALE_MASK: $s = \text{FP32_MUL}(a[i], \text{scalar})$

$y[i] = \text{mask_enable} ? \text{FP32_ADD}(s, b[i]) : s$

4.5.2.1 Positional add

Sau copy CLS và patch embedding, Vector thực hiện $HIDDEN_A[i] = FP32_ADD(HIDDEN_A[i], POS[i])$ trên $197 \times 768 = 151.296$ word. Đây là command Vector đầu tiên của schedule, có thể ghi đè $HIDDEN_A$ vì frontend đã gather packet nguồn trước writeback.

4.5.2.2 Residual add

Mỗi encoder có hai residual:

residual 1: $HIDDEN_B = FP32_ADD(HIDDEN_B, HIDDEN_A)$

residual 2: $HIDDEN_A = FP32_ADD(LINEAR_TMP, HIDDEN_B)$

Cả hai có length 151.296. Residual 1 là in-place ở $HIDDEN_B$; residual 2 ghi $HIDDEN_A$ từ hai source khác. Schedule bảo đảm operand còn sống cho đến khi packet cuối được tiêu thụ.

4.5.2.3 Attention scale (mask disabled) và phép toán từng phần tử

Sau QK^T , Vector nhân $SCORE_PROB$ với $1/\sqrt{64}=0,125$ trên $12 \times 197 \times 197 = 465.708$ word. $SCALE_MASK$ có khả năng thêm mask, nhưng command attention thực tế đặt $mask_enable=0$, $src1=NONE$; vì vậy M8 không thực hiện causal/attention mask trong path này. Với mask tắt, engine bỏ adder và không yêu cầu read source thứ hai.

4.5.2.4 Luồng đọc-tính-ghi và điều kiện kết thúc

FSM rút gọn là $IDLE \rightarrow LOAD \rightarrow ADD$ hoặc $MULTIPLY \rightarrow ADD \rightarrow WRITE \rightarrow DONE$. $LOAD$ gather tối đa 16 lane; arithmetic xử lý từng lane; $WRITE$ giữ packet và lane mask đến khi frontend nhận. Tail lane có mask 0 không tạo read/write DDR. Ví dụ packet cuối của scale có base 465.696: 12 lane hợp lệ tính $score \times 0,125$, bốn lane còn lại không được ghi. Config length bằng 0, memory error hoặc stall đều ngăn completion; không có timeout nội bộ.

4.5.3 Layout engine

Layout là data mover bit-preserving: không đổi FP32 thành định dạng khác và không làm rounding. Với iterator ($i0, i1, i2$), nó đọc source theo stride và ghi destination contiguous:

```
src = src_base + i0*stride0 + i1*stride1 + i2*stride2
dst = dst_base + linear_output_index
dst_bits = src_bits
```

FSM $IDLE \rightarrow VALIDATE \rightarrow REQUEST \rightarrow WRITE \rightarrow \dots \rightarrow DONE$ giữ source address hoặc destination/data ổn định khi memory backpressure. Validator kiểm dimension, product và overflow địa chỉ u32; nó không chứng minh mọi semantic shape, capacity vùng hay overlap tùy ý. An toàn chỉ được khẳng định cho mapping fixed schedule của M8.

4.5.3.1 Copy CLS và copy patch layout

Embedding dùng hai copy:

CLS: dims=[1,1,768], strides=[0,0,1]

patch: dims=[1,196,768], strides=[0,768,1]

CLS từ MODEL/PARAM vào token 0 HIDDEN_A; 196 patch embedding từ LINEAR_TMP vào token 1...196. Sau final LayerNorm, select-CLS dùng lại layout [1,1,768] để copy token 0 HIDDEN_B sang LINEAR_TMP phục vụ classifier.

4.5.3.2 Split Q/K/V theo attention head

Mỗi projection token-major [197,768] được split thành [12,197,64] với output dims [12,197,64] và source strides [64,768,1]. Phần tử head=h, token=t, dim=d đọc LINEAR_TMP[t, h×64+d]. Đây là biến đổi layout, không phải reshape metadata thuần túy, vì dữ liệu được ghi lại trong SCRATCH head-major.

4.5.3.3 K transpose

K transpose đọc K_HEAD[h,t,d] và phát layout [h,d,t] vào LINEAR_TMP. Output dims là [12,64,197], source strides [12608,1,64]; 12608=197×64. Nhờ vậy QK^T đọc B theo K dimension liên tục logic mà không cần transpose tính toán trong GEMM.

4.5.3.4 Head merge

Sau $P \times V$, context ở Q_HEAD [12,197,64] được merge thành LINEAR_TMP [197,768]. Output dims [197,12,64] và source strides [64,12608,1] thực hiện ánh xạ [h,t,d] $\rightarrow$ [t,h,d], là inverse layout của split head.

4.5.3.5 Địa chỉ, stride và iterator layout

Iterator chỉ advance sau khi write hiện tại được frontend nhận. Vì destination của Layout luôn contiguous, descriptor cần mô tả stride source chứ không cần stride destination. Đặc điểm này giúp command template diễn đạt copy CLS/patch, split, transpose, merge và select CLS bằng cùng một opcode; đổi lại, Layout đi theo nhịp read-one-word/write-one-word nên không phải engine compute song song.

4.5.4 LayerNorm engine

LayerNorm chạy 25 lần: LN1/LN2 của 12 encoder và final LayerNorm. Mỗi command xử lý 197 token, hidden size 768, gamma/beta từ PARAM và epsilon 1e-12. Toàn bộ đường này FP32; không có FP16 ingress.

4.5.4.1 Mean

Với token t và H=768, engine tuần tự tích lũy:

```

sum = 0
for c = 0..767:
    sum = FP32_ADD(sum, x[t,c])
mean = FP32_MUL(sum, recip_u32(768))

```

Thứ tự cột cố định là một phần contract số học. Trong fast path, mean pass cũng ghi toàn bộ row vào sample BRAM để các pass sau không phải đọc lại DDR.

4.5.4.2 Variance

Variance là biased variance, chia cho H chứ không phải H-1:

```

var_sum = 0
for c = 0..767:
    centered = FP32_ADD(x[t,c], -mean)
    var_sum = FP32_ADD(var_sum, FP32_MUL(centered, centered))
variance = FP32_MUL(var_sum, recip_u32(768))

```

Không dùng FMA; mỗi cộng/nhân là một rounding point FP32 riêng. Sample row được replay từ BRAM ở pass này khi hidden 768.

4.5.4.3 Reciprocal square root/normalization

Engine tạo $a = \text{FP32_ADD}(\text{variance}, \text{epsilon})$ rồi xấp xỉ $\text{inv_std} = \text{rsqrt}(a)$. Seed là $0x5F375A86 - (a \gg 1)$ và đúng ba Newton update:

$$y \leftarrow y \times (1,5 - 0,5 \times a \times y \times y)$$

Mỗi phép trong update được thực hiện tuần tự bởi shared arithmetic. Sau đó $\text{normalized} = \text{FP32_MUL}(\text{FP32_ADD}(x, -\text{mean}), \text{inv_std})$. Nhánh special của rsqrt xử lý NaN/âm, zero và infinity theo RTL; reference cần tái hiện contract này thay vì thay bằng hàm thư viện.

4.5.4.4 Gamma, beta và output writeback

Affine cuối là:

```

scaled = FP32_MUL(normalized, gamma[c])
y[t,c] = FP32_ADD(beta[c], scaled)

```

Ba BRAM sample/gamma/beta đều sâu 1024 word. Với token 0, engine nạp gamma/beta để cache; token 1...196 replay sample/gamma/beta local. Dù có cache, writeback vẫn theo handshake từng output word và chỉ hoàn tất token khi response hợp lệ.

4.5.4.5 Ví dụ một token LayerNorm

Với token 0, hardware tính theo đúng thứ tự $\text{sum}_0 = \text{ADD}(0, x[0])$, $\text{sum}_1 = \text{ADD}(\text{sum}_0, x[1])$, ..., sum_{767} ; nhân với $1/768$ tạo mean; sau đó scan cùng row để tạo variance; cuối cùng scan lần ba để center $\rightarrow$ normalize $\rightarrow$ gamma $\rightarrow$ beta và ghi $y[0, c]$. Ví dụ này thể hiện ba pass tuần tự thực tế, không nên giản lược thành một vector operation duy nhất.

4.5.5 Softmax engine

Softmax chạy 12 command attention và một command class tùy chọn. Attention có $\text{row_count} = 12 \times 197 = 2364$, $\text{row_length} = 197$; class Softmax có một row dài 1.000. Cả hai row length đều fit row buffer 1024 word.

4.5.5.1 Tìm max theo hàng

Với row $x[0 \dots N-1]$, engine quét max trước:

```
m = max(x[0], ..., x[N-1])
```

Ở buffer path, pass này đồng thời đọc raw input từ DDR vào BRAM. Tất cả raw của một row có mặt local trước khi output probability đầu tiên được ghi.

4.5.5.2 Trừ max; attention scale/mask ở lệnh Vector trước Softmax

Attention scale 0,125 đã được Vector thực hiện trước Softmax; Softmax không thực hiện mask trong schedule M8. Nó tạo centered value $z = \text{FP32_ADD}(x[i], -m)$ để exp làm việc trên miền không dương. Nếu z dương do trường hợp numeric bất thường, RTL trả qNaN thay vì tính exp ngoài miền dự kiến.

4.5.5.3 Exponential/xấp xỉ theo phương pháp M8

M8 không gọi expf. Với $z \leq 0$, exp approximant dùng range reduction theo $1/\ln(2)$, hiệu chỉnh remainder, Horner polynomial bậc 8 với hệ số C8...C0, rồi scale 2^{-q} . Nhánh special trả 1 cho signed zero, 0 cho -Inf hoặc underflow sâu, và qNaN cho NaN/miền không hợp lệ. Đây là xấp xỉ xác định bởi constant và thứ tự FP32 RTL.

4.5.5.4 Sum, reciprocal và normalization

Sau khi tính $e[i]$, engine cộng dồn FP32 tuần tự, dùng seed $0x7EF311C3$ - sum rồi bốn Newton update $y \leftarrow y \times (2 - \text{sum} \times y)$ để xấp xỉ $1/\text{sum}$. Cuối cùng ghi:

```
p[i] = FP32_MUL(e[i], reciprocal)
```

Ở fast path, BRAM lần lượt chứa raw x sau max pass và e sau sum pass; output pass replay e. Vì vậy Softmax attention có thể đọc/ghi SCORE_PROB in-place mà không phá score chưa đọc. Với row dài hơn 1024, RTL fallback reread/recompute nhưng shape M8 hiện tại không đi vào nhánh đó.

4.5.5.5 Ví dụ một hàng attention score

Một row attention có 197 score của một (head, query-token). Vector đã scale toàn score; Softmax đọc 197 word vào BRAM và tìm m, overwrite lần lượt BRAM bằng $e[0 \dots 196]$ đồng thời tạo sum, rồi nhân mỗi e với reciprocal và ghi 197 probability trở lại cùng row SCORE_PROB. Khi row này hoàn tất, engine chuyển row kế tiếp; không có 197 exp unit song song.

4.5.6 GELU engine

GELU chạy một command sau FC1 trong mỗi encoder, in-place trên FC1[197,3072] gồm 605.184 word. M8 dùng erf-style approximant, không phải tanh-GELU và cũng không phải mathematical erf exact.

4.5.6.1 Công thức và phương pháp hiện thực

Với $z = |x|/\sqrt{2}$, scalar core xấp xỉ reciprocal $\approx 1/(1+pz)$, dùng polynomial năm hạng và $\exp_approx(-z^2)$ để tạo erf_approx, rồi tính:

```
erf_mag  $\approx$  clamp(1 - P(t)*exp_approx(-z2), 0, 1)
erf_sgn = sign(x) ? -erf_mag : erf_mag
GELU(x)  $\approx$  0,5*x*(1 + erf_sgn)
```

Reciprocal dùng bốn Newton update; exponential dùng range reduction và polynomial bậc 8 giống họ approximant của Softmax. Constant đều là FP32 hex cố định trong RTL, nên “bit-exact” chỉ có nghĩa tái hiện cùng graph arithmetic M8.

4.5.6.2 Trình tự tính toán một phần tử

Packet dispatcher đọc tối đa 16 word, nhưng scalar core chạy lần lượt lane 0 đến lane cuối: scale $|x|$, reciprocal, polynomial, z^2 , exp, clamp/sign, rồi output GELU. Sau khi các lane active hoàn tất, dispatcher giữ toàn packet output đến khi frontend ghi xong. Tail lane không launch scalar core. NaN và -Inf đi qNaN; +Inf giữ +Inf; cả +0/-0 canonicalize thành +0.

4.5.6.3 Chuỗi FC1 $\rightarrow$ GELU $\rightarrow$ FC2

FC1 GEMM phải retire hoàn toàn trước GELU. GELU gather cả packet FC1 trước khi ghi in-place, nhờ đó output lane đầu không phá input lane sau. Khi GELU command DONE, FC2 GEMM đọc FC1 đã activation và ghi LINEAR_TMP. Đây là dependency tuần tự thực tế, không phải fusion FC1/GELU/FC2 trong một engine.

4.5.7 Argmax engine

Argmax chỉ chạy sau classifier và trước class Softmax tùy chọn.

4.5.7.1 Khởi tạo max và chỉ số lớp

Engine đặt `have_best=false`, `best_index=0` và bắt đầu scan 1.000 logit FP32 từ LOGITS. Không cấp destination DDR cho command này.

4.5.7.2 So sánh/cập nhật tuần tự

```
for i = 0..999:  
    if logits[i] không finite: sticky_nonfinite_error = 1  
    else if !have_best or logits[i] > best_value:  
        best_value = logits[i]; best_index = i; have_best = true
```

Comparator là numeric FP32, không sắp xếp raw word unsigned. Update strict > giữ index nhỏ nhất khi equal, kể cả +0 và -0. NaN và cả hai infinity không được chọn làm winner và làm command error fail-closed.

4.5.7.3 Xuất nhãn và trạng thái hoàn tất

Sau scan, engine giữ {class_index, class_logit} đến result_ready; command controller capture sideband này rồi phát class_result_valid. LOGITS không bị ghi đè, vì vậy class Softmax tùy chọn vẫn có thể đọc 1.000 logit. Nếu toàn bộ input nonfinite, RTL có placeholder {0,qNaN} nhưng command vẫn kết thúc error, không được diễn giải thành dự đoán lớp 0 hợp lệ.

4.6 Đồng bộ giữa controller, GEMM, non-GEMM và memory frontend

M8 dùng một chuỗi ownership rõ ràng thay vì để engine truy cập DDR trực tiếp:

DDRMemory frontend/adapted engineCommand controllerSequencerDDRMemory
frontend/adapted engineCommand controllerSequencerdescriptor valid/readylaunch selected
opcodedata_requestlogical read → AXI readresponsecomplete data bundle validcompute / buffer /
packet or panelresult validlogical write → AXI responsewrite acceptedresult readydone or
errorcmd_done or cmd_error

Memory frontend có state đọc SELECT → REQUEST → RESPONSE → DELIVER và đối xứng cho write. Nó arbitrate operand load với result store; packed GEMM có tie-break round-robin khi load/store đồng thời, còn path khác ưu tiên load theo contract. Với GEMM, frontend snapshot generation và destination của result trước store; với non-GEMM, nó gather packet/row trước writeback. Vì vậy mọi engine có cùng quy tắc: không nhận input chưa đủ, không retire output chưa được frontend chấp nhận.

Nếu adapter phát hiện bounds/alignment/permission error hoặc lỗi framing response, frontend latch memory error và quarantine request mới. Command controller ưu tiên lỗi memory hơn normal done, reset selected engine và báo error cho sequencer. Wrapper đưa lỗi lên status sticky/recovery. Software phải để NPU idle rồi soft-reset theo recovery policy trước job mới; không giả định CLEAR_ERROR đơn lẻ khôi phục mọi trạng thái protocol.

4.7 Danh sách hình/bảng kỹ thuật của chương

Các hình/bảng sau nên được giữ trong bản dàn trang cuối vì chúng giúp kiểm chứng mối liên hệ giữa thuật toán và RTL, không lặp lại bảng kết quả Chương 6.

Mã	Hình/bảng	Nội dung cần thể hiện
H4.1	Top-level NPU	Wrapper, sequencer, controller/dispatch, bẫy engine, frontend và adapter
H4.2	Luồng một command	ISSUE → LAUNCH → EXECUTE → REPORT, cùng đường valid/ready
H4.3	Descriptor 16 word	Header, route, base, dimension, stride, immediate và flag quan trọng
H4.4	GEMM R8/C2/K16	Hai pass vật lý R8/C1 qua tám stream
H4.5	Packed-B ping-pong	Ownership FREE/RESERVED/READY/COMPUTE của hai bank
H4.6	Một dot stream	FP16 multiply → fixed-point CSA → finalize/RNE → bias FP32
H4.7	Non-GEMM organization	Shared FP32 arithmetic và local BRAM/buffer của sáu engine
H4.8	Error/completion chain	Adapter/frontend → controller → sequencer → wrapper status
B4.1	GEMM taxonomy	Mười loại GEMM, shape, B source, bias và command count
B4.2	Non-GEMM mapping	Operator ViT → engine → tensor → buffer → output
B4.3	Trace memory/handshake	Gather input, compute, scatter output và terminal condition

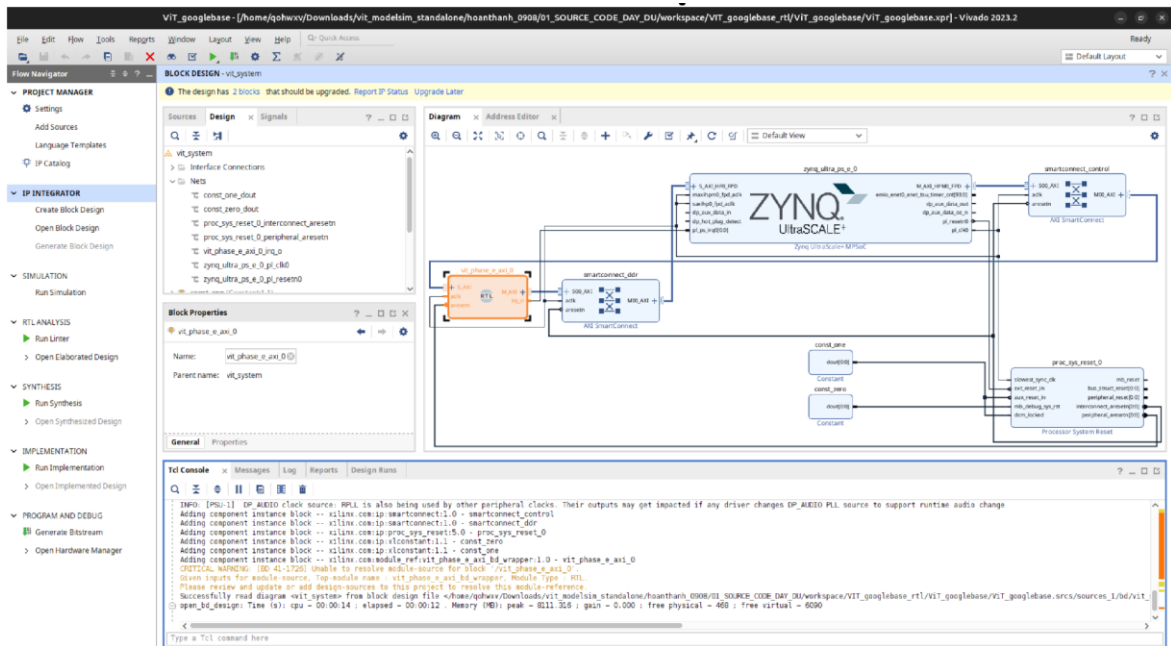
CHƯƠNG 5 — BỘ NHỚ, AXI, TÍCH HỢP SOC VÀ RUNTIME

Chương này trình bày cách kiến trúc M8 được tích hợp thành hệ thống SoC thực tế trên Genesys ZU-5EV, tập trung vào đường PS–PL, tổ chức DDR, Shared Memory Frontend, AXI Memory Adapter và runtime. Các compute engine ở Chương 4 được xem như client của memory subsystem; nội dung dưới đây giải thích cách dữ liệu đi từ DDR tới engine và quay trở lại bộ nhớ.

5.1 Kiến trúc PS–PL và Vivado Block Design

Thiết kế được tích hợp trên Zynq UltraScale+ MPSoC của board Genesys ZU-5EV. Processing System (PS) cung cấp Cortex-A53, DDR controller và các cổng AXI; Programmable Logic (PL) chứa ViT accelerator. Block Design thực tế tách hai đường SmartConnect theo chức năng, thay vì dùng một interconnect chung cho cả control và tensor traffic.

- Control path: PS M_AXI_HPM0_FPD → SmartConnect control → NPU S_AXI → AXI4-Lite register bank.
- Data path: NPU M_AXI → SmartConnect DDR → PS S_AXI_HP0_FPD → PS DDR controller → DDR.
- Interrupt/status: irq_o của NPU được đưa về phía PS; runtime hiện tại vẫn chủ yếu poll trạng thái qua XSCT/JTAG.



Hình 5.1. Block Design thực tế trong Vivado 2023.2 với Zynq UltraScale+, SmartConnect control, SmartConnect DDR và IP ViT accelerator.

Tách control path và data path giúp traffic cấu hình nhỏ không cạnh tranh trực tiếp với lưu lượng tensor. CPU/host chỉ cần cấu hình job và các base address, trong khi accelerator có AXI master riêng để chủ động đọc/ghi DDR trong quá trình thực thi.

5.2 Clock, reset và các miền đồng bộ

Clock chính của PL được lấy từ PS qua pl_clk0 và cấu hình ở 50 MHz, tương ứng chu kỳ 20 ns. proc_sys_reset tạo các tín hiệu reset đồng bộ cho interconnect và peripheral trong PL. Phiên bản M8 cuối chủ yếu sử dụng một miền clock PL chung cho control, AXI adapter và compute path, qua đó tránh thêm phức tạp CDC ở top-level.

$$T_{\text{clk}} = 1 / 50 \text{ MHz} = 20 \text{ ns}$$

Reset vật lý của SoC và soft reset của job có vai trò khác nhau. Reset hệ thống đưa các block logic về trạng thái khởi tạo, trong khi soft reset/recovery của NPU chỉ được sử dụng theo policy của register/control logic khi accelerator đang ở trạng thái hợp lệ.

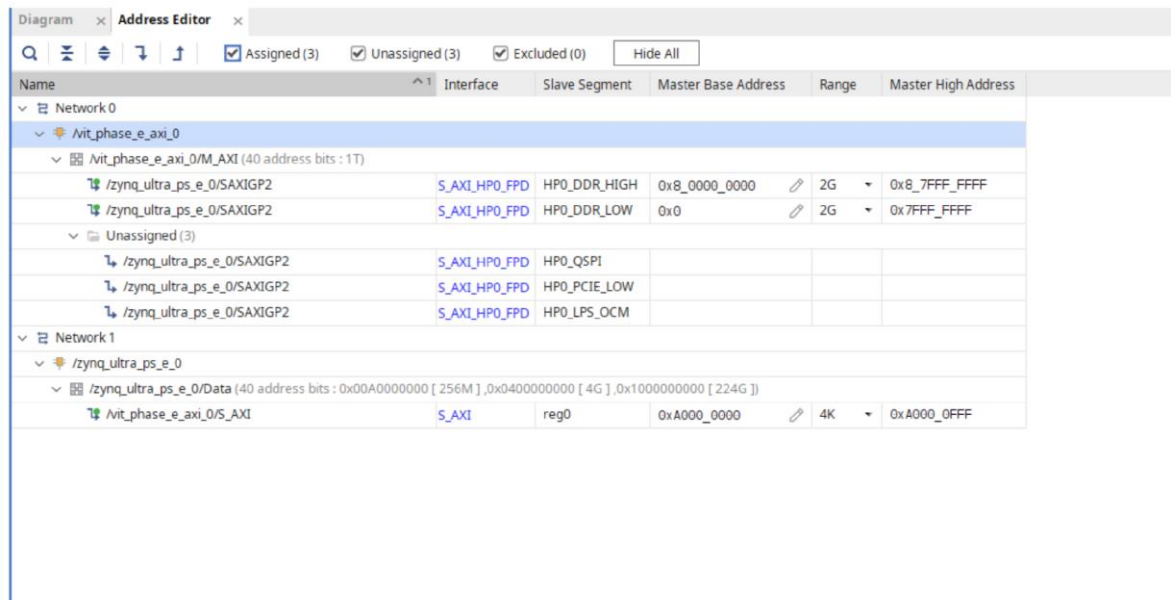
5.3 Bản đồ địa chỉ và vai trò từng vùng DDR

Model, input và tensor trung gian có dung lượng lớn được lưu trong DDR. Descriptor của M8 sử dụng logical word address theo đơn vị u32; AXI Memory Adapter chuyển địa chỉ logic thành byte address vật lý bằng quan hệ:

$$\text{physical_address} = \text{region_base} + (\text{word_address} \ll 2)$$

Bản đồ dưới đây là memory map của snapshot board cuối. TABLE là vùng staging phía host: host đọc table binary và nạp parameter cần thiết vào register/table RAM của IP trước START; NPU không xem TABLE như một tensor M_AXI được đọc liên tục trong inference.

Vùng	Base	Dung lượng	Quyền truy cập	Vai trò
Control MMIO	0xA0000000	4 KiB	PS/host ↔ NPU	Register, status, counter, parameter table
MODEL	0x10000000	0x0A5A3C00 byte ≈ 165.64 MiB	NPU đọc	Persistent weight/parameter
INPUT	0x24B00000	0x00093000 byte ≈ 0.574 MiB	Host ghi, NPU đọc	Prepared input, 150,528 FP32 word
TABLE staging	0x25000000	0x00003280 byte ≈ 12.63 KiB	Host	Header và parameter entries
SCRATCH	0x27000000	0x00798000 byte ≈ 7.594 MiB	NPU đọc/ghi	Activation và tensor trung gian
LOGITS	0x27790000	1,000 word FP32	NPU ghi / host đọc	Classifier output
PROBABILITIES	0x27794000	1,000 word FP32	NPU ghi / host đọc	Class Softmax tùy chọn



Hình 5.2. Vivado Address Editor: M_AXI của accelerator được ánh xạ tới DDR qua S_AXI_HP0_FPD; vùng AXI4-Lite control đặt tại 0xA0000000.

Trong SCRATCH, các workspace được tái sử dụng theo vòng đời tensor. Các offset nổi bật gồm HIDDEN_A 0x000000, HIDDEN_B 0x025000, LINEAR_TMP 0x04A000, Q/K/V lần lượt 0x06F000/0x094000/0x0B9000, SCORE_PROB 0x0DE000, FC1 0x150000, LOGITS 0x1E4000 và CLASS_PROB 0x1E5000. Việc tái sử dụng làm giảm dung lượng scratch nhưng đòi hỏi sequencer duy trì đúng thứ tự producer-consumer.

5.4 Memory frontend

Shared Memory Frontend là lớp trung gian giữa các compute engine và AXI Memory Adapter. Engine không trực tiếp phát AXI transaction; thay vào đó engine phát request logic kèm context, còn frontend chịu trách nhiệm gather operand, route response, cache dữ liệu có khả năng tái sử dụng và scatter kết quả.

5.4.1 Arbitration và ownership

Tại một thời điểm chỉ có một high-level command active, nhưng frontend vẫn phải quản lý nhiều loại request bên trong command, ví dụ activation, weight, bias, gamma/beta và result writeback. Quyền sở hữu được gắn với active command/engine để response không bị giao nhầm consumer. Ở GEMM packed mode, hai panel bank tuân theo vòng đời FREE → RESERVED → READY → COMPUTE → FREE, tạo khả năng overlap load panel kế tiếp với compute panel hiện tại khi điều kiện scheduling cho phép.

5.4.2 Address context

Address context mang region base, logical word address, stride, kích thước và quyền truy cập của request. Frontend kiểm tra request thuộc MODEL, INPUT hay SCRATCH trước khi chuyển xuống adapter. Write chỉ hợp lệ với vùng được phép ghi, chủ yếu là SCRATCH và các subregion output.

5.4.3 Activation cache và bias cache

GEMM sử dụng cache cục bộ để giảm số lần đọc lại dữ liệu. Snapshot implementation cho thấy activation cache chiếm 32 RAMB36 và bias cache chiếm 4 RAMB36. Activation cache hỗ trợ replay activation khi cùng A tile được dùng cho nhiều cột output; bias cache tránh đọc lại bias cho từng nhóm kết quả. LayerNorm và Softmax cũng có local buffer riêng để replay dữ liệu trong các pha nhiều lượt đọc.

Khối bộ nhớ cục bộ	RAMB36	Vai trò
GEMM activation cache	32	Giữ/replay activation panel
GEMM bias cache	4	Giữ bias dùng lại theo output channel
LayerNorm local buffers	3	Sample, gamma, beta
Softmax row buffer	1	Lưu/replay một hàng Softmax
Layer parameter table	1	Parameter/config cục bộ
Tổng	41	Khớp post-implementation utilization

5.4.4 Read router, write router và result path

Read router phân phối response tới đúng engine và đúng loại operand. Đường ghi tập trung kết quả của GEMM/Non-GEMM về frontend trước khi phát write request xuống adapter. Một command

chỉ được báo DONE sau khi dữ liệu kết quả đã được chấp nhận và write response cuối cùng liên quan đến command đã hoàn tất. Quy tắc này ngăn controller chuyển sang command sau trong khi tensor producer trước vẫn chưa được commit vào SCRATCH.

5.5 AXI memory adapter

AXI Memory Adapter chuyển giao diện logic theo word u32 của M8 thành giao tiếp AXI4 Memory-Mapped 128 bit. Cấu hình implementation cuối được tóm tắt trong Bảng 5.3.

Tham số	Giá trị	Ý nghĩa
M_AXI data width	128 bit	16 byte/beat
M_AXI address width	40 bit	Không gian địa chỉ vật lý của PS
AXI ID width	1 bit	Số ID hạn chế
Max burst length	4 beat	Tối đa 64 byte/burst ở bus 128 bit
Read outstanding	2	Tối đa hai read transaction đang chờ
Write outstanding	1	Một write transaction đang chờ
Request FIFO	Depth 2	Tách frontend khỏi AXI issue
Response FIFO	Depth 2	Tách AXI response khỏi consumer

5.5.1 Read request FIFO

Read request FIFO nhận request logic từ frontend và giữ chúng cho tới khi AXI read-address channel chấp nhận. FIFO depth 2 cho phép frontend tạm thời phát trước request kế tiếp mà không phụ thuộc trực tiếp vào ARREADY trong từng chu kỳ.

5.5.2 Read response FIFO

Read response FIFO nhận dữ liệu từ R channel và đưa về giao diện logic của frontend. Khi consumer chưa sẵn sàng, FIFO giữ response và tạo backpressure hợp lệ thay vì làm mất beat. Depth nhỏ phản ánh trade-off tài nguyên và khả năng hấp thụ burst.

5.5.3 Scalar read, line-fill và read-ahead

Adapter hỗ trợ scalar word read và line-fill. Với bus 128 bit, một beat chứa bốn word 32 bit. Line-fill tận dụng tính liên tục của tensor để lấy nhiều word trong cùng beat/burst, còn read-ahead cố gắng chuẩn bị dữ liệu trước khi compute engine yêu cầu. Mức read-ahead hiện tại vẫn bị giới hạn bởi burst tối đa 4 beat, số outstanding nhỏ và pattern gather/stride của từng engine.

5.5.4 AXI beat, burst và addressing

Một transfer hợp lệ xảy ra khi VALID và READY cùng bằng 1. Đối với read burst, adapter phát ARADDR/ARLEN rồi nhận tối đa bốn beat trên R channel. Với DATA_WIDTH = 128 bit, mỗi beat tương đương 16 byte và burst 4 beat tương đương tối đa 64 byte dữ liệu liên tiếp.

$$\text{bytes_per_beat} = 128/8 = 16 \text{ byte}, \quad \text{max_burst_payload} = 4 \times 16 = 64 \text{ byte}$$

5.5.5 Đường ghi, write response và backpressure

Write path phát AW và W theo handshake độc lập của AXI, sau đó chờ B response để xác nhận transaction. Nếu downstream chưa thể nhận dữ liệu, READY có thể hạ xuống và backpressure lan ngược về frontend/engine. Result FIFO và write path giúp tách ngăn hạn compute khỏi DDR writeback, nhưng không làm tăng bandwidth trung bình của hệ thống.

5.6 Ví dụ minh họa một luồng xử lý liên quan đến bộ nhớ

5.6.1 Mục tiêu, command mẫu và tensor mẫu

Xét Q projection trong một encoder layer. Phép logic là $Q = XW_Q + b_Q$, với X có shape [197,768], W_Q [768,768] và Q [197,768]. Descriptor chỉ định opcode GEMM, địa chỉ nguồn/đích, M/N/K và các flag packed-B/bias tương ứng.

$$Q = X W_Q + b_Q$$

5.6.2 Sinh địa chỉ input/activation/weight/bias

X được đọc từ SCRATCH ở dạng FP32. W_Q là persistent GEMM-B trong MODEL, lưu blocked K16/N2 và packed hai FP16 trong một word u32. Bias Q giữ trên đường FP32. Scheduler tạo tile context, còn frontend chuyển logical address của từng operand thành region/base context.

5.6.3 Read request → AXI transaction → read response

Frontend phát request operand xuống adapter. Adapter đóng gói request thành AXI read transaction, nhận beat 128 bit từ DDR, tách word/half cần thiết và trả response kèm metadata về frontend. Nếu consumer chưa thể nhận, response FIFO giữ dữ liệu và tạo backpressure.

5.6.4 Nạp dữ liệu vào cache/panel/buffer ping-pong

Activation được đưa vào activation cache/panel phù hợp. Packed-B được nạp vào hai panel bank theo cơ chế ping-pong. Một bank chỉ chuyển sang READY sau khi panel cần thiết đã đủ; bank COMPUTE không bị ghi đè cho tới khi scheduler trả nó về FREE.

5.6.5 Cấp toán hạng cho GEMM hoặc non-GEMM

Khi panel READY, Operand Router cấp từng cặp FP16 A/B cho tám stream. Mỗi stream nhận tối đa một cặp operand trong một compute-active cycle; K16 là số term theo chiều K trong panel, không phải 16 multiplier trên một stream. Khi command kế tiếp là Non-GEMM, cùng Shared Memory Frontend chuyển sang native read width và access pattern tương ứng của engine đó.

5.6.6 Result FIFO → scratch writeback

Sau khi dot-product finalize và bias được xử lý, kết quả FP32 đi vào result path. Result FIFO depth 2 trên đường GEMM giữ một lượng nhỏ kết quả đang chờ store. Frontend sinh địa chỉ destination trong SCRATCH, adapter phát write transaction và chờ B response.

5.6.7 Non-GEMM đọc tensor trung gian kế tiếp

Khi Q projection hoàn tất, sequencer phát command kế tiếp theo fixed schedule, ví dụ Layout để split Q thành 12 head. Layout engine đọc tensor Q vừa ghi trong SCRATCH, biến đổi index/stride và ghi lại ở vùng head workspace. Do command trước chỉ retire sau write completion, consumer không đọc dữ liệu chưa được commit.

5.6.8 Timeline request → response → compute → writeback

Thời điểm	Request/response	Vùng dữ liệu	Buffer/FIFO	Engine	Tensor/kết quả
T0	Launch GEMM command	SCRATCH/MODEL	Address context	GEMM scheduler	Q tile chưa tính
T1	Read request A/B/bias	SCRATCH + MODEL	REQ FIFO	Memory frontend	Gather operand
T2	AXI read response	DDR	RSP FIFO/cache/panel	Memory frontend	Panel lắp dần
T3	Panel READY	On-chip	Ping-pong bank	8 FP16 streams	Compute K panel
T4	Finalize + bias	On-chip	Result FIFO	GEMM	FP32 result
T5	AXI write + B response	SCRATCH	Write path	Memory frontend	Q tile committed
T6	Issue Layout	SCRATCH	Layout read path	Layout engine	Split Q theo head

5.7 Luồng runtime cho một ảnh

Runtime cuối vẫn là host-driven qua hw_server/XSCT/JTAG. Cortex-A53 và firmware PS được dùng cho bring-up/DDR initialization, nhưng chưa có application inference hoàn chỉnh chạy độc lập trên A53. Vì vậy cần phân biệt cold setup với per-image inference.

5.7.1 Chuẩn bị model, command table và input

Host chuẩn bị mixed-precision model package, layer/parameter table và prepared input. Persistent GEMM-B đã được pack FP16 theo layout K16/N2 trước khi tải lên DDR. Input mỗi ảnh được preprocess ở host về tensor đúng format trước khi ghi vào INPUT.

5.7.2 Khởi tạo và kích hoạt job

Cold setup gồm kết nối JTAG, nạp firmware PS khi cần, program bitstream, khởi tạo/kiểm tra DDR, nạp MODEL và INPUT, zero SCRATCH và cấu hình register/table. Sau đó host ghi START; sequencer tự phát chuỗi 249 command khi class Softmax cuối được bật.

1. Kết nối hw_server/XSCT và khởi tạo PS/DDR.
2. Program bitstream và nạp model package vào MODEL.
3. Nạp prepared input, zero/initialize SCRATCH và parameter table.
4. Ghi START; poll STATUS cho tới DONE hoặc ERROR.
5. Đọc class index, logits/probabilities và performance counters.

5.7.3 Theo dõi trạng thái thực thi

Trong lúc job chạy, host theo dõi busy/done/error và performance counter. Các counter ở Chương 7 phản ánh panel load/compute, feeder stall, operand-bank empty, result backpressure và mức sử

dụng tám stream. Counter là công cụ profiling; chúng không thay thế kiểm chứng đúng/sai của tensor.

5.7.4 Thu thập đầu ra và log bằng chứng

Sau DONE, host đọc class index và các vùng output cần thiết, lưu predicted label, ground-truth, pass/fail và runtime vào CSV/log. Trong campaign nhiều ảnh, model được giữ trong DDR giữa các lần chạy (retained-DDR) để không tính thời gian nạp model qua JTAG vào latency inference. Mỗi ảnh mới chủ yếu thay INPUT, khởi tạo vùng scratch cần thiết, START và thu output.

Giới hạn runtime hiện tại: XSCT/JTAG phù hợp cho bring-up và validation nhưng chưa phải production runtime. Bare-metal/Linux application trên Cortex-A53 là hướng phát triển, không phải chức năng đã hoàn tất của snapshot này.

CHƯƠNG 6 — PHƯƠNG PHÁP KIỂM CHỨNG VÀ KẾT QUẢ CUỐI CÙNG

Chương 6 tổng hợp bằng chứng từ mức primitive/engine tới implementation và chạy board. Ba câu hỏi được tách riêng: phần cứng có hoàn tất job hay không, kết quả phân loại có đúng ground-truth hay không, và vector số học có trùng reference trong phạm vi tolerance hay không.

6.1 Quy tắc trình bày snapshot cuối cùng

Snapshot cuối được khóa theo project Vivado 2023.2 cho part xczu5ev-sfvc784-1-e, target clock 50 MHz và ràng buộc DSP = 0. Số liệu implementation/timing lấy từ implemented design; kết quả tập ảnh lấy từ campaign 1.000 ảnh đầu tiên của ImageNet validation được tổng hợp trong slide báo cáo cuối kỳ.

- MEASURED: số liệu lấy trực tiếp từ Vivado report, runtime log hoặc kết quả tổng hợp.
- RTL/IMPLEMENTATION VERIFIED: thông tin được đối chiếu từ RTL, hierarchy, parameter hoặc Block Design.
- DERIVED: giá trị tính từ số liệu measured/verified, ví dụ 128 bit = 16 byte/beat.
- PROPOSED: hướng tối ưu chưa implement; không được trình bày như kết quả đã đạt.

Phần so sánh với bài báo tham chiếu được để trống ở Mục 6.8, chờ chốt công trình và metric so sánh.

6.2 Chiến lược kiểm chứng

6.2.1 Mức phép toán

Leaf-level verification kiểm tra primitive số học và chuyển đổi định dạng trước khi tích hợp engine. Các nhóm quan trọng gồm FP16 multiply, FP32 add/mul no-DSP, FP32↔FP16 conversion, comparator, reciprocal/iterative arithmetic và logic alignment/rounding của dot stream. Mục tiêu là cô lập lỗi arithmetic khỏi lỗi scheduling hoặc memory.

6.2.2 Mức engine

Engine-level verification kiểm tra GEMM và từng Non-GEMM engine với descriptor/shape phù hợp. GEMM được kiểm các trường hợp persistent packed-B, dynamic B, bias enable/disable, tail M/N/K và result handshake. Non-GEMM được kiểm Vector, Layout, LayerNorm, Softmax, GELU và Argmax theo trình tự state thực tế.

6.2.3 Mức tích hợp

Integration verification kiểm tra Sequencer → Command Controller → Engine Dispatch → Memory Frontend → AXI adapter. Khi class Softmax cuối được bật, một job gồm 249 command: 4 command embedding, 20 command cho mỗi trong 12 encoder và 5 command cuối. Điều kiện pass ở mức hệ thống là schedule đi đến DONE không có command/memory error và output cuối đọc lại được.

$$4 + 12 \times 20 + 5 = 249 \text{ command/job}$$

6.2.4 Mức implementation

Sau khi RTL/integration pass, project được chạy synthesis và implementation trên Vivado. Tiêu chí chính gồm không có failed route, write_bitstream hoàn tất, resource utilization nằm trong khả năng device và timing constraints được đáp ứng. DSP usage được kiểm ở post-implementation thay vì chỉ dựa trên attribute trong RTL.

6.2.5 Mức board và tập dữ liệu

Board-level verification kiểm tra cold bring-up, retained-DDR session, START/DONE và output trên Genesys ZU-5EV. Sau khi một ảnh chạy ổn định, cùng protocol được lặp trên 1.000 ảnh tuần tự để đánh giá Top-1 và tính ổn định của flow.

6.3 Phương pháp đánh giá tập ảnh

6.3.1 Nguồn dữ liệu, số lượng ảnh và tiêu chí chọn mẫu

Tập kiểm chứng sử dụng 1.000 ảnh đầu tiên, index 1–1000, của ImageNet ILSVRC2012 validation. Đây là một subset tuần tự của tập validation 50.000 ảnh, không phải kết quả trên toàn bộ ImageNet validation.

6.3.2 Tiền xử lý và nhãn tham chiếu

Ảnh được preprocess ở host theo pipeline của model để tạo prepared input trước khi nạp vào INPUT. Ground-truth class lấy từ bộ nhãn ILSVRC2012 tương ứng. Decode/resize/normalize nằm ngoài PL trong phạm vi hiện tại; vì vậy thời gian của các bước này không được đồng nhất với compute latency của NPU.

6.3.3 Quy trình chạy một ảnh và thu log

1. Chọn ảnh và tạo prepared input.
2. Ghi INPUT, reset/zero các vùng scratch cần thiết và cấu hình job.

3. Phát START và chờ DONE/ERROR.
4. Đọc class index/logits, ghi predicted label và runtime vào log/CSV.
5. So sánh predicted class với ground-truth và cập nhật correct/incorrect.

6.3.4 Quy tắc ghi nhận pass/fail, lỗi và dữ liệu thiếu

Hardware PASS nghĩa là job hoàn tất theo protocol mà không báo lỗi control/memory. Classification PASS nghĩa là predicted Top-1 trùng ground-truth. Hai khái niệm này không được gộp. Nếu job lỗi hoặc output không đọc được, sample phải được ghi riêng là hardware failure thay vì tự động tính là classification wrong.

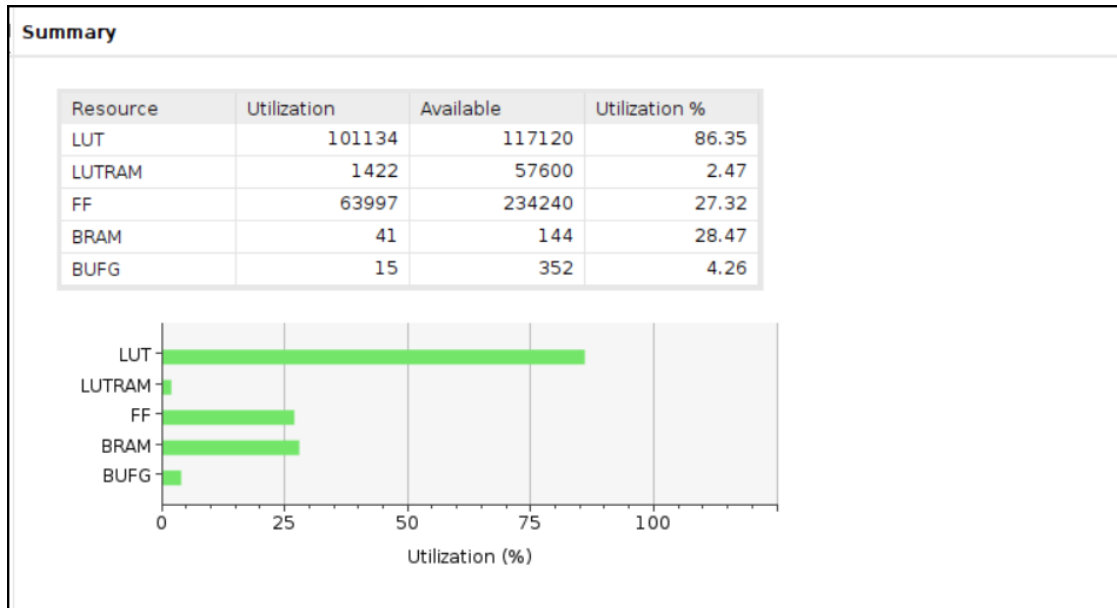
6.3.5 Khả năng tái lập kết quả

Để tái lập kết quả cần có định bitstream/XSA, model package, execution mode, command table, preprocessing, dải index ảnh và script runtime. Log cuối nên lưu tối thiểu image index, predicted class, ground-truth, pass/fail và runtime. Ảnh chụp terminal của campaign 1.000 ảnh được dùng làm bằng chứng tổng hợp cho correct/incorrect trong bản báo cáo này.

6.4 Kết quả implementation và timing cuối cùng

Implementation cuối đạt trạng thái write_bitstream Complete trên Vivado 2023.2. Bảng tài nguyên cho thấy thiết kế sử dụng phần lớn LUT/CLB nhưng vẫn giữ DSP bằng 0, đúng với ràng buộc ban đầu.

Tài nguyên	Sử dụng / Có sẵn	Tỷ lệ
CLB LUT	101,134 / 117,120	86.35%
LUTRAM	1,422 / 57,600	2.47%
Registers / FF	63,997 / 234,240	27.32%
CARRY8	5,521 / 14,640	37.71%
Physical CLB	14,547 / 14,640	99.36%
RAMB36	41 / 144	28.47%
URAM	0	0%
DSP48E2	0 / 1,248	0%



Hình 6.1. Resource utilization từ implemented design.

Physical CLB đạt khoảng 99.36%, cho thấy placement gần sát capacity của device mặc dù CLB LUT là 86.35%. Điều này phản ánh áp lực placement/routing của datapath LUT-based và là lý do không nên chỉ nhìn phần trăm LUT để kết luận còn nhiều không gian mở rộng.

Chỉ số timing	Kết quả	Diễn giải
Target frequency	50 MHz	Clock period 20 ns
WNS	+0.045 ns	Setup timing met; margin rất nhỏ
TNS	0.000 ns	Không có setup violation
WHS	+0.010 ns	Hold timing met
THS	0.000 ns	Không có hold violation
WPWS	+8.500 ns	Pulse-width timing met
Worst data path	≈19.58 ns	Gần giới hạn 20 ns
Logic levels	≈52	Đường tổ hợp sâu
Routing contribution	≈70%	Routing chi phối critical path

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.045 ns	Worst Hold Slack (WHS): 0.010 ns	Worst Pulse Width Slack (WPWS): 8.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 197161	Total Number of Endpoints: 197161	Total Number of Endpoints: 66188

All user specified timing constraints are met.

Hình 6.2. Design Timing Summary: setup, hold và pulse-width đều đạt yêu cầu.

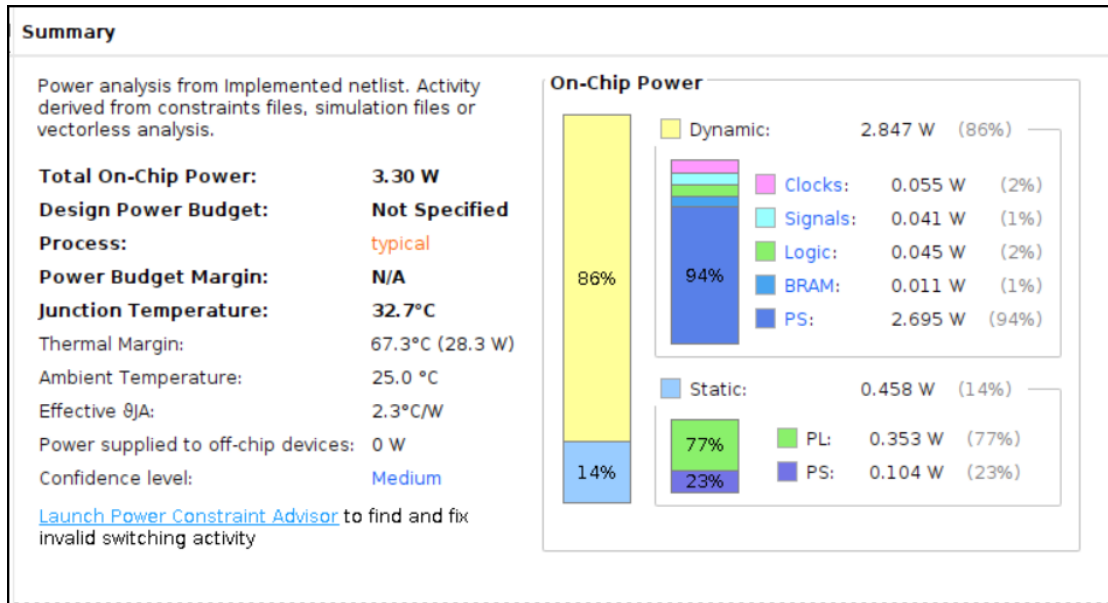
Intra-Clock Paths

Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)	WPWS (ns)	TPWS (ns)	Failing Endpoints (TPWS)	Total Endpoints (TPWS)
clk_pl_0	rise - rise	0.045	0.000	0	197161	rise - rise	0.010	0.000	0	197161	8.500	0.000	0	66188

Hình 6.3. Intra-clock timing của clk_pl_0 trong implemented design.

Kết luận timing ở mức implementation là PASS tại target 50 MHz. Tuy nhiên WNS chỉ +0.045 ns, tức margin gần bằng 0. Phân tích Chương 7 cho thấy timing hotspot liên quan shared FP32 Non-GEMM path và routing; đây là vấn đề khác với runtime starvation của GEMM.

Vivado Power Analysis ước lượng Total On-Chip Power khoảng 3.30 W, trong đó Dynamic khoảng 2.847 W và Static khoảng 0.458 W; junction temperature ước lượng 32.7 °C ở ambient 25 °C. Confidence level của tool là Medium, vì vậy đây chỉ là tool-estimated power, không phải công suất đo trực tiếp trên board.



Hình 6.4. Vivado Power Summary của implemented design; số liệu được sử dụng như ước lượng của công cụ.

Name	Constraints	Status	WNS	TNS	WHS	ITS	WSS	TSS	Total Power	Failed Routes	Methodology	High Score	QoR Suggestions	QoR	FF	Block	Design	QoR	Status
synth_1 (active)	constrs_1	synth_design Complete!													0	0	0	0	8/9/26, 3:46 AM
impl_1	constrs_1	write_bitstream Complete!	0.045	0.000	0.010	0.000		0.000	3.305	0				101134	63997	41	0	0	8/9/26, 3:50 AM

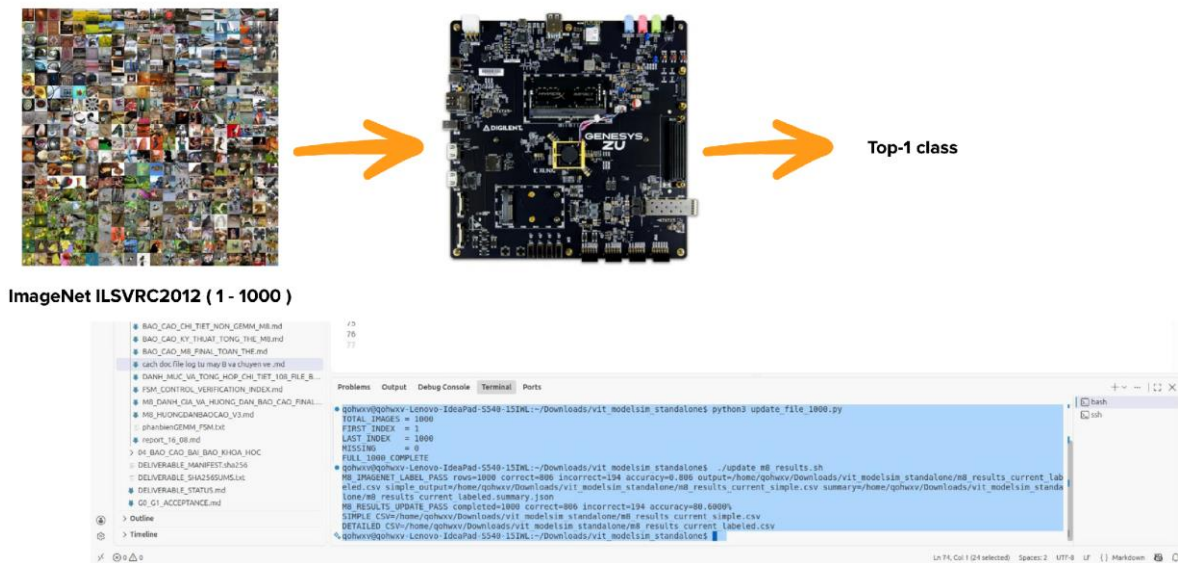
Hình 6.5. Design Runs cho thấy synthesis complete, implementation/write_bitstream complete và DSP = 0.

6.5 Bảng kết quả cuối cùng trên tập ảnh

Campaign cuối chạy 1.000 ảnh đầu tiên của ImageNet validation và tổng hợp kết quả bằng script. Terminal ghi nhận TOTAL IMAGES = 1000, FIRST INDEX = 1, LAST INDEX = 1000 và FULL_1000_COMPLETE; sau đó predicted labels được so khớp ground-truth.

Metric	Kết quả cuối
Dataset	ImageNet ILSVRC2012 validation
Phạm vi sample	Index 1–1000
Số ảnh	1,000
Correct	806
Incorrect	194
Top-1 accuracy	80.6%
Command/job	249 khi bật class Softmax
Runtime prototype	xấp xỉ 7 phút/ảnh trong flow host-driven hiện tại

$$\text{Top-1} = (806 / 1000) \times 100\% = 80.6\%$$



Hình 6.6. Phương pháp kiểm chứng board và log tổng hợp campaign ImageNet 1–1000.

Kết quả 80.6% cho thấy hệ thống có thể thực hiện end-to-end classification trên board với mixed-precision datapath hiện tại. Đây là accuracy trên subset 1.000 ảnh và không được diễn giải như official ImageNet-50k accuracy.

6.5.1 Tham chiếu với software CPU

Nền tảng	Top-1	Latency
Laptop CPU reference	~85%	~490 ms/ảnh
FPGA prototype	80.6%	~7 phút/ảnh
Chênh lệch	~4.4 điểm phần trăm	~857× theo giá trị làm tròn 7 phút/490 ms

Bảng trên chỉ cung cấp context cho prototype, không phải benchmark speedup công bằng. Runtime FPGA hiện bao gồm flow host-driven, nhiều lượt truy cập DDR, scheduling và serialization được ưu tiên để fit ZU-5EV dưới ràng buộc DSP=0. Mục tiêu của phiên bản cuối là chứng minh functional deployment và xác định bottleneck, không phải chứng minh FPGA nhanh hơn CPU.

6.6 So sánh vector số và kết quả phân loại

Kết quả Top-1 và kiểm chứng vector số phải được tách riêng. Một sample có thể cho đúng class dù logits khác software reference một lượng nhỏ; ngược lại, một job có thể hoàn tất protocol nhưng vẫn có sai khác số học ở tensor trung gian.

Trong phạm vi artifact hiện được cung cấp cho bản hoàn thiện này, bằng chứng implementation và Top-1 campaign đã rõ, nhưng chưa có một bộ raw tensor checkpoint cuối đủ để kết luận toàn graph bit-exact hoặc đạt một tolerance thống nhất tại mọi layer. Vì vậy báo cáo không tuyên bố full numerical-vector sign-off.

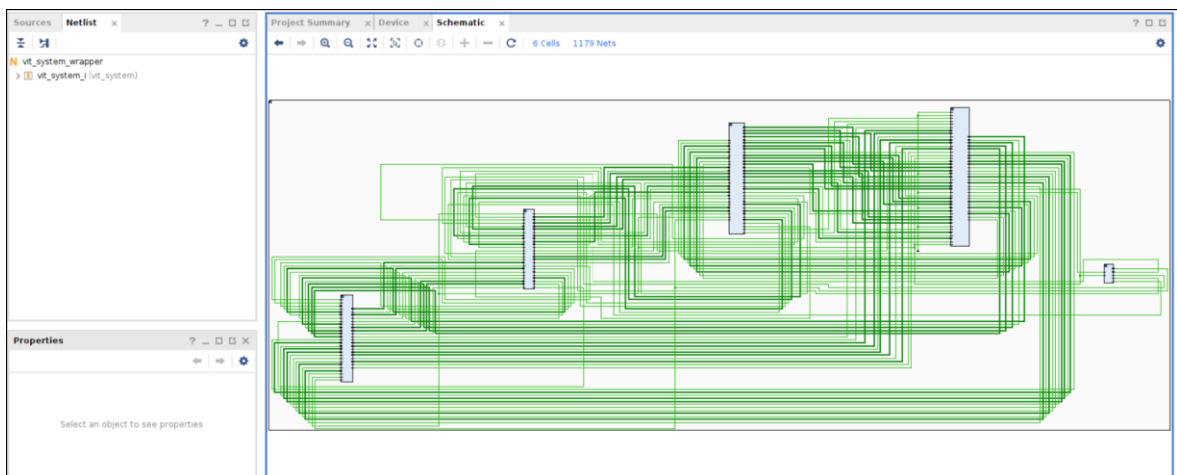
- GEMM: sai khác có thể xuất phát từ FP32→FP16 tại ingress, packed FP16 persistent B, special-value policy, thứ tự tích lũy và RNE ở cuối dot product.

- LayerNorm/Softmax/GELU: sai khác có thể xuất phát từ thứ tự FP32 operation và các approximant reciprocal/exp/erf.
- Layout: về nguyên tắc bit-preserving; mismatch thường chỉ ra lỗi address/stride hoặc data movement.
- Argmax: chỉ quan tâm ordering của logits; không cần class Softmax nếu mục tiêu chỉ là Top-1.

Phương pháp tiếp tục numerical sign-off là so sánh checkpoint sau projection, QK^T , Softmax, $P \times V$, LayerNorm, GELU và logits; ghi max absolute error, relative error có epsilon bảo vệ và số phần tử vượt ngưỡng. Checkpoint đầu tiên lệch giúp phân biệt precision/approximation với lỗi memory/layout/control.

6.7 Hình, bảng, log và receipt minh chứng

Nhóm bằng chứng	Artifact	Điều được chứng minh
Block Design	Vivado IP Integrator screenshot	PS, SmartConnect control/data, NPU và reset
Address map	Vivado Address Editor	S_AXI control và M_AXI→DDR mapping
Utilization	Implemented Design / Report Utilization	LUT, FF, BRAM, DSP=0
Timing	Report Timing Summary	WNS/TNS/WHs/THs và timing closure
Power	Vivado Power Summary	Tool-estimated on-chip power; không phải board measurement
Design Runs	Vivado run status	synth_1 complete; impl_1 write_bitstream complete
Bitstream/XSA	Hardware artifacts	Export/program board
Dataset	Terminal/CSV summary 1–1000	806 correct, 194 wrong, 80.6%
Runtime profiling	Performance counters	Bottleneck analysis ở Chương 7



Hình 6.7. Schematic/netlist tổng thể do Vivado tạo. Hình này được dùng làm evidence tích hợp, không thay thế sơ đồ kiến trúc logic ở Chương 4.

CHƯƠNG 7 — PHÂN TÍCH KỸ THUẬT VÀ GIỚI HẠN

Trên cơ sở các kết quả đó, Chương 7 tập trung phân tích ý nghĩa kỹ thuật của các số liệu thu được, xác định các điểm nghẽn chính của hệ thống và giới hạn phạm vi mà các kết quả hiện tại có thể chứng minh.

Hai vấn đề cần được phân biệt xuyên suốt chương này là **timing bottleneck** và **runtime bottleneck**. Timing bottleneck xác định đường logic giới hạn chu kỳ clock của thiết kế, trong khi runtime bottleneck phản ánh mức độ hiệu quả mà các tài nguyên tính toán thực sự được sử dụng trong quá trình suy luận. Hai hiện tượng này có nguyên nhân khác nhau và do đó cần các chiến lược tối ưu khác nhau.

7.1 Phân bố thời gian thực thi và runtime counters

Kết quả thực thi trên phiên bản M8 cho thấy một lần suy luận đầy đủ phải đi qua toàn bộ chuỗi lệnh của ViT-Base/Patch16-224. Sequencer điều phối tổng cộng **249 command**, bao phủ giai đoạn embedding, 12 Transformer Encoder layer và phần xử lý cuối gồm final LayerNorm, classifier, Argmax và class Softmax.

Thời gian thực thi của prototype cuối được ghi nhận ở mức xấp xỉ 7 phút cho một ảnh trong luồng host-driven hiện tại. Latency tổng này bao gồm toàn bộ chuỗi điều phối và data movement của job, nên không phản ánh trực tiếp phần thời gian mà GEMM engine thực sự thực hiện phép tính. Các performance counter được sử dụng ở Chương 7 để tách thời gian load, compute và stall.

Một snapshot profiling được ghi nhận như sau:

Runtime counter	Giá trị quan sát
Panel/load active	khoảng 88,5%
Panel compute active	khoảng 10,11%
Feeder stall	khoảng 87,9%
Operand bank empty	khoảng 60,1%

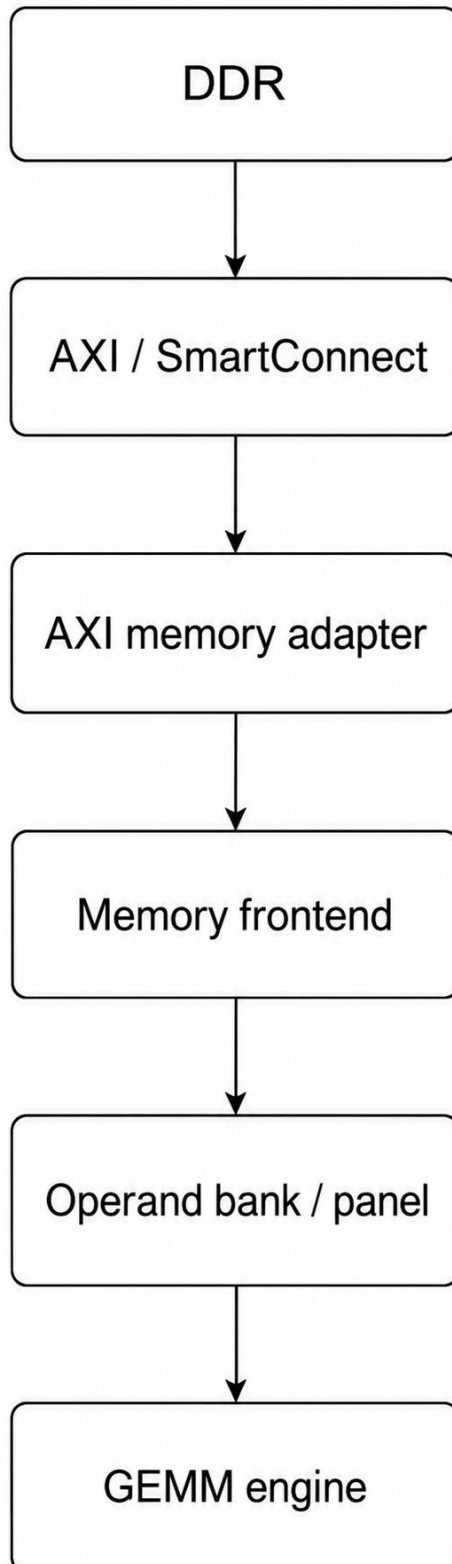
GEMM compute active	khoảng 10,1%
GEMM stream active khi đang tính	khoảng 7,84/8

Các counter này mô tả những trạng thái khác nhau của hệ thống và không nên được cộng trực tiếp với nhau như các thành phần độc lập của một biểu đồ thời gian. Ví dụ, một chu kỳ có thể đồng thời thuộc một trạng thái panel hoạt động và một điều kiện feeder stall. Vì vậy, giá trị quan trọng của các counter không nằm ở việc cộng chúng thành 100%, mà ở việc xác định **mối quan hệ giữa khả năng tính toán và khả năng cung cấp dữ liệu**.

Counter `gemm_compute_active` cho thấy GEMM datapath chỉ thực sự ở trạng thái tính toán trong khoảng 10% thời gian quan sát. Tuy nhiên, khi datapath đã ở trạng thái active, số stream được sử dụng đạt khoảng **7,84 trên tổng số 8 stream**. Điều này cho thấy phần tính toán song song không phải lúc nào cũng hoạt động ở mức thấp; ngược lại, khi operand đã được cung cấp đầy đủ, các stream tính toán có thể được khai thác gần mức tối đa của cấu hình hiện tại.

Vấn đề chính nằm ở khoảng thời gian giữa các cửa sổ compute-active. Giá trị `feeder_stall` xấp xỉ 87,9% và `operand_bank_empty` khoảng 60% cho thấy compute engine thường phải chờ hệ thống phía trước cung cấp toán hạng.

Có thể mô tả hành vi tổng quát như sau:



Nếu dữ liệu đến operand bank đủ nhanh, tám GEMM stream có thể hoạt động gần đầy. Nếu operand bank cạn dữ liệu, GEMM không thể tiếp tục dù các đơn vị số học vẫn còn khả năng tính toán.

Do đó, latency hiện tại không nên được diễn giải đơn giản là “GEMM quá chậm”. Số liệu profiling cho thấy vấn đề quan trọng hơn nằm ở việc **duy trì dòng operand liên tục tới GEMM engine**.

Đây là một kết luận có ý nghĩa đối với hướng tối ưu tiếp theo. Nếu chỉ tăng số lượng multiplier hoặc mở rộng độ rộng GEMM trong khi tốc độ cấp dữ liệu không được cải thiện tương ứng, phần cứng bổ sung có nguy cơ tiếp tục ở trạng thái chờ và không tạo ra mức tăng throughput tỷ lệ thuận với chi phí tài nguyên.

7.2 Mức sử dụng GEMM và đường cấp toán hạng

GEMM là thành phần tính toán chính của ViT, xuất hiện trong patch embedding, các phép chiếu Q, K, V, attention matrix multiplication, output projection, hai lớp MLP và classifier. Vì vậy, mức sử dụng GEMM có ảnh hưởng trực tiếp đến hiệu năng tổng thể của accelerator.

Trong M8, GEMM được tổ chức dưới dạng nhiều FP16 computation stream hoạt động song song, với accumulation và phân xử lý sau ở miền precision rộng hơn. Thiết kế sử dụng chiến lược mixed precision nhằm giảm lượng dữ liệu cho operand GEMM nhưng vẫn duy trì FP32 ở các phần cần độ chính xác cao hơn.

Kết quả profiling cho thấy hai trạng thái rất khác nhau.

Khi operand chưa sẵn sàng:

Operand bank empty

↓

Feeder stall

↓

GEMM không thể nhận term mới

↓

Compute utilization thấp

Khi operand đã sẵn sàng:

Operand available

↓

GEMM compute active

↓

$\approx 7,84 / 8$ stream hoạt động

Sự khác biệt này cho thấy **khả năng tính toán nội tại của GEMM datapath và utilization toàn job không phải cùng một khái niệm.**

Một datapath có thể đạt utilization cao trong cửa sổ active nhưng vẫn tạo throughput thấp nếu các cửa sổ active xuất hiện không liên tục. Đây chính là trường hợp được quan sát trong snapshot M8.

Đường cấp toán hạng bắt đầu từ DDR. Weight, activation và intermediate tensor được đọc thông qua AXI4 master. Memory frontend sau đó tổ chức dữ liệu thành định dạng phù hợp với GEMM engine. Những bước như tính địa chỉ, gather operand, chuyển đổi layout, điền operand bank và đồng bộ trạng thái đều nằm trên đường dữ liệu trước khi phép nhân thực sự được thực hiện.

Do đó, thuật ngữ phù hợp hơn để mô tả runtime bottleneck là: end-to-end operand/data-delivery bottleneck thay vì quy toàn bộ vấn đề cho riêng DDR hoặc riêng SmartConnect.

Từ dữ liệu hiện tại, chưa có phép đo đủ chi tiết để tách riêng latency của từng thành phần:

- DDR controller;
- cổng PS-PL;
- SmartConnect;
- AXI memory adapter;
- address generation;
- memory frontend;
- operand bank;
- synchronization giữa feeder và GEMM.

Vì vậy, không nên kết luận rằng một thành phần đơn lẻ, chẳng hạn SmartConnect, là nguyên nhân duy nhất của bottleneck.

Điều có thể kết luận chắc chắn hơn từ các counter là:

1. GEMM không được cung cấp operand liên tục.
2. Operand-bank-empty xuất hiện với tỷ lệ đáng kể.
3. Feeder stall chiếm phần lớn thời gian quan sát.
4. Khi đã có dữ liệu, tám stream GEMM được khai thác gần đầy.

Những kết quả này chỉ ra rằng bước tối ưu hợp lý trước khi mở rộng compute array là tăng hiệu quả của đường cấp dữ liệu.

7.3 Memory/AXI, data movement và backpressure

Memory subsystem của M8 sử dụng DDR làm nơi lưu các vùng dữ liệu chính gồm model, input và scratch/intermediate tensor. NPU truy cập DDR thông qua giao tiếp **AXI4 master 128 bit**, trong khi CPU điều khiển NPU bằng AXI4-Lite.

Cấu hình AXI được ghi nhận trong thiết kế gồm:

Thông số	Giá trị
AXI data width	128 bit
Burst length tối đa	4 beat
Read outstanding	2
Write outstanding	1

Ở tần số 50 MHz, giao tiếp vật lý 128 bit có bandwidth lý thuyết đáng kể nếu mỗi beat được sử dụng liên tục. Tuy nhiên, bandwidth lý thuyết không đồng nghĩa với bandwidth hiệu dụng của workload.

Trong quá trình thực thi ViT, memory traffic không chỉ gồm việc đọc model một lần. Các tensor trung gian được tạo ra, đọc lại và ghi trở lại scratch memory qua nhiều operator khác nhau. Đặc biệt, attention, LayerNorm, GELU, residual operation và MLP tạo ra các pattern truy cập không hoàn toàn giống một luồng đọc tuần tự dài.

Các yếu tố làm giảm efficiency của đường dữ liệu có thể bao gồm:

- burst tương đối ngắn;
- số lượng outstanding transaction hạn chế;
- thời gian tạo request;
- các access có stride hoặc gather;
- chuyển đổi giữa logical word và physical AXI beat;

- đồng bộ giữa frontend và compute engine;
- thiếu prefetch;
- thiếu overlap đầy đủ giữa load và compute;
- nhiều lần ghi và đọc intermediate tensor từ scratch DDR.

Trong AXI, dữ liệu chỉ được truyền tại thời điểm cả VALID và READY đồng thời được assert. Khi slave chưa sẵn sàng hoặc master chưa thể phát request tiếp theo, pipeline dữ liệu sẽ tạo ra khoảng trống. Các khoảng trống này truyền ngược về phía compute engine dưới dạng **backpressure**.

Ở M8, backpressure cuối cùng có thể biểu hiện dưới dạng:

AXI không trả dữ liệu kịp

↓

Memory frontend thiếu word

↓

Operand bank không được refill

↓

Feeder không phát term mới

↓

GEMM stall

Cần lưu ý rằng đây là chuỗi quan hệ ở mức hệ thống. Counter hiện tại cho thấy stall tại phía feeder và operand bank, nhưng chưa cô lập được tỷ lệ stall phát sinh riêng từ từng tầng.

Một đặc điểm quan trọng khác là **data movement cost**. Trong accelerator cho mạng nơ-ron, phép nhân thường được xem là workload chính, nhưng năng lượng và thời gian dành cho việc di chuyển dữ liệu có thể trở nên đáng kể nếu operand không được reuse hiệu quả.

Đối với ViT, điều này đặc biệt rõ ở các ma trận weight lớn và intermediate activation có kích thước đáng kể. Nếu cùng một tile weight hoặc activation phải được đọc lại từ DDR nhiều lần, compute throughput lý thuyết của GEMM engine khó được duy trì.

Do đó, memory optimization cần tập trung không chỉ vào bandwidth thô mà còn vào ba mục tiêu:

Thứ nhất, tăng transaction efficiency.

Giảm số lượng request nhỏ và tận dụng burst dài hơn khi access pattern cho phép.

Thứ hai, tăng mức overlap giữa communication và computation.

Trong khi GEMM đang xử lý một tile, tile tiếp theo nên được chuẩn bị nếu tài nguyên buffer cho phép.

Thứ ba, tăng data reuse tại bộ nhớ on-chip.

Những operand được sử dụng nhiều lần nên được giữ tại BRAM/cache/operand bank đủ lâu thay vì quay lại DDR.

Với cấu hình hiện tại, các cải tiến như prefetch, ping-pong buffering, coalesced burst và tăng request concurrency là những hướng có cơ sở trực tiếp từ kết quả profiling.

7.4 Timing, critical path và tài nguyên

Khác với runtime bottleneck, timing bottleneck được xác định từ routed timing report của Vivado.

Thiết kế M8 được implementation tại:

- tần số mục tiêu: **50 MHz**;
- chu kỳ clock: **20 ns**;
- WNS: khoảng **+0,045 ns**;
- TNS: **0 ns**.

Như vậy, thiết kế đáp ứng timing constraint tại điểm implementation 50 MHz. Tuy nhiên, WNS chỉ dương khoảng 0,045 ns, cho thấy **timing margin rất nhỏ**.

Không nên từ kết quả này suy ra rằng 50 MHz là Fmax thực nghiệm chính xác của thiết kế. Báo cáo hiện tại chỉ chứng minh thiết kế đạt constraint tại 50 MHz; việc xác định Fmax cần sweep tần số hoặc implementation lại với constraint chặt hơn.

Critical path được xác định nằm trong **shared FP32 non-GEMM datapath**, đi qua các khối số học và multiplexing phục vụ GELU, Softmax, LayerNorm hoặc các operator liên quan.

Worst path có khoảng:

- **52 logic levels**;
- routing delay chiếm khoảng **70%** tổng path delay.

Điều này mang hai ý nghĩa.

Thứ nhất, bottleneck về clock không nằm ở GEMM array hay giao diện AXI mà nằm trong logic số học non-GEMM được chia sẻ.

Thứ hai, tỷ lệ routing delay cao cho thấy vấn đề không chỉ nằm ở độ phức tạp Boolean của các phép toán FP32. Việc nhiều engine cùng sử dụng shared arithmetic fabric tạo fanout, muxing và khoảng cách routing đáng kể sau placement.

Về resource utilization, tài liệu M8 ghi nhận:

Resource	Used	Available	Utilization
LUT	khoảng 51.042	59.040	khoảng 86,4%
FF	khoảng 38.106	118.080	khoảng 32,3%
BRAM36	41	144	khoảng 28,5%
DSP	0	1.248	0%

Hai điểm nổi bật là tỷ lệ LUT cao và việc không sử dụng DSP.

Ràng buộc DSP=0

DSP=0 là một lựa chọn kiến trúc có chủ đích của M8. Các phép toán floating-point được xây dựng bằng LUT/logic thay vì phụ thuộc vào DSP block.

Lợi ích chính là:

- giữ toàn bộ DSP cho ứng dụng khác;
- khảo sát khả năng hiện thực NPU bằng logic khả trình;
- cung cấp một baseline để đánh giá chi phí của LUT-based arithmetic.

Tuy nhiên, kết quả implementation cũng cho thấy chi phí của quyết định này.

FP32 arithmetic bằng LUT tạo ra:

- số lượng logic level lớn;
- routing phức tạp;
- LUT utilization cao;
- timing margin nhỏ.

Do đó, DSP=0 không nên được diễn giải là lựa chọn tối ưu tuyệt đối. Đây là một **điểm thiết kế trong design space** nhằm phục vụ mục tiêu nghiên cứu cụ thể.

Hai hướng tối ưu timing có thể được khảo sát sau này.

Hướng thứ nhất là **pipeline sâu hơn shared FP32 datapath**. Việc chèn thêm register có thể làm giảm combinational delay trên từng stage, đổi lại latency của operator tăng thêm một số cycle.

Hướng thứ hai là **selective DSP**. Chỉ các operator nằm trên critical path mới sử dụng DSP, trong khi GEMM hoặc các datapath khác vẫn giữ LUT-based implementation.

Tuy nhiên, cả hai phương án đều là hướng phát triển. Báo cáo hiện tại chưa có implementation comparison để khẳng định phương án nào hiệu quả nhất.

Timing bottleneck và runtime bottleneck

Kết quả tổng hợp cho thấy:

TIMING BOTTLENECK

Shared FP32 non-GEMM datapath

↓

Giới hạn chu kỳ clock

RUNTIME BOTTLENECK

Operand/data-delivery path

↓

Giới hạn GEMM utilization

Đây là hai vấn đề độc lập.

Nếu chỉ pipeline non-GEMM để tăng Fmax nhưng memory frontend vẫn không cấp đủ operand, throughput có thể không tăng tương ứng.

Ngược lại, nếu chỉ tối ưu memory frontend mà không cải thiện timing path, hệ thống có thể sử dụng compute tốt hơn tại 50 MHz nhưng vẫn khó nâng tần số.

Vì vậy, hướng phát triển cần xử lý hai nhóm vấn đề riêng biệt.

7.5 Sai khác số học và phạm vi diễn giải

Một điểm quan trọng của kết quả M8 là cần phân biệt **system execution correctness**, **classification result** và **numerical equivalence**.

Trong phép thử board được ghi nhận, hệ thống:

- thực hiện đủ full schedule 249 command;
- không xuất hiện hardware/protocol error;
- đạt trạng thái DONE;
- cho Top-1 class trùng với golden class của mẫu kiểm thử.

Tuy nhiên, phép so sánh vector đầu ra cho thấy:

- **465/1000 logits** vượt tolerance được sử dụng;
- **3/1000 probability** vượt tolerance;
- trạng thái tổng thể của numerical verification là **NOT_PROMOTED**.

Do đó, kết quả phải được diễn giải theo ba cấp:

Full hardware schedule: PASS

Top-1 classification của mẫu kiểm thử: PASS

Full numerical-vector equivalence: FAIL / chưa sign-off

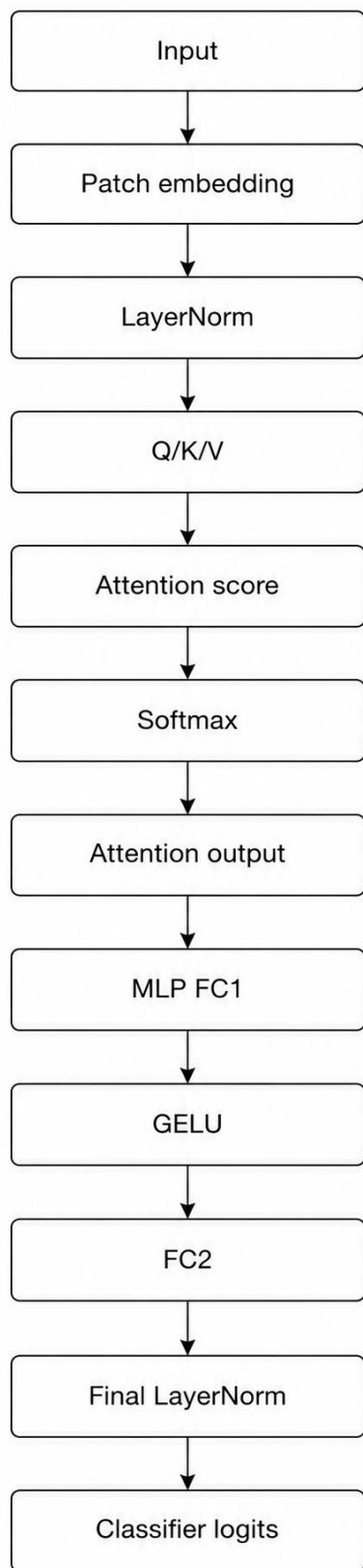
Việc Top-1 đúng không chứng minh rằng toàn bộ 1000 logits đã tương đương với software reference. Argmax chỉ phụ thuộc vào phần tử lớn nhất. Một số sai lệch ở các logit khác có thể không làm thay đổi class index nhưng vẫn thể hiện sai khác số học đáng kể.

M8 sử dụng mixed precision nên có nhiều vị trí có khả năng tạo numerical drift, bao gồm:

1. chuyển đổi FP32 sang FP16 tại ingress của GEMM;
2. rounding trong phép nhân FP16;
3. accumulation;
4. chuyển đổi hoặc rounding ở output;
5. tính mean và variance của LayerNorm;
6. approximation hoặc implementation của GELU;
7. exponential và normalization trong Softmax.

Tài liệu hiện tại chưa cô lập được đóng góp của từng nguồn sai khác. Vì vậy, các nguyên nhân trên chỉ nên xem là **các giả thuyết cần kiểm tra**, không phải root cause đã được chứng minh.

Để xác định root cause, cần thực hiện checkpoint comparison tại nhiều vị trí trong inference:



Tại checkpoint đầu tiên xuất hiện divergence lớn, có thể tiếp tục thu hẹp xuống operation hoặc arithmetic primitive tương ứng.

Phạm vi của kết quả dataset

Tài liệu M8 còn ghi nhận một snapshot multi-image gồm khoảng **1000 kết quả**, với Top-1 accuracy khoảng **80,6%**. Tuy nhiên, kết quả này được chính tài liệu phân loại ở mức **USER-REPORTED**, chưa phải bằng chứng độc lập tương đương Vivado timing report hoặc raw board log đã được kiểm chứng đầy đủ.

Do đó, con số này chỉ nên được trình bày như snapshot thực nghiệm hiện có, không nên sử dụng để khẳng định accuracy cuối cùng của accelerator cho toàn bộ ImageNet validation set.

Để đưa ra kết luận accuracy chính thức cần:

- lưu raw prediction của từng ảnh;
- lưu exact image index và golden label;
- cố định preprocessing;
- cố định bitstream/model package;
- lưu execution status;
- kiểm tra số lượng sample;
- tính accuracy bằng script tái lập được.

7.6 Các giới hạn của phạm vi

Phiên bản M8 là một research prototype nhằm kiểm chứng chuỗi thiết kế từ RTL tới board. Do đó, một số giới hạn cần được nêu rõ khi diễn giải kết quả.

Giới hạn về mô hình

Thiết kế hiện tập trung vào **ViT-Base/Patch16-224**, batch size bằng 1 và geometry cố định.

Sequencer sử dụng schedule compile-time với 249 command. Vì vậy thiết kế hiện không thể tự động chạy các kiến trúc khác như:

- ViT-Large;
- DeiT;
- Swin Transformer;
- BERT;
- các cấu hình patch hoặc hidden dimension khác.

Việc hỗ trợ model generalization sẽ yêu cầu compiler/lowering framework, descriptor generation linh hoạt và memory allocator động.

Giới hạn về precision

Thiết kế sử dụng mixed FP16/FP32 và chưa đạt full-vector numerical sign-off.

Do đó, báo cáo chỉ có thể khẳng định mức correctness tương ứng với những phép thử đã thực hiện, không thể khẳng định accelerator bit-exact với PyTorch.

Giới hạn về runtime

Runtime hiện tại vẫn là **host-driven** thông qua XSCT/JTAG.

Host chịu trách nhiệm:

- chuẩn bị input;
- nạp dữ liệu vào DDR;
- cấu hình register;
- phát START;
- poll trạng thái;
- đọc kết quả.

Do đó, M8 chưa phải standalone embedded inference system chạy trực tiếp bằng Cortex-A53.

Một runtime như:

A53 application



AXI4-Lite control



NPU



Interrupt



A53 result handling

mới chỉ nằm trong phạm vi phát triển tiếp theo.

Giới hạn về throughput

Latency hiện tại còn cao và profiling cho thấy compute resource không được sử dụng liên tục.

M8 vì vậy không nên được định vị như một accelerator ViT tối ưu về throughput. Giá trị kỹ thuật chính nằm ở việc xác lập được một baseline có thể triển khai trên board và có số liệu để chỉ ra bottleneck.

Giới hạn về timing

Thiết kế đạt timing tại 50 MHz nhưng WNS chỉ khoảng +0,045 ns. Margin này rất nhỏ.

Không có frequency sweep nên chưa xác định chính xác Fmax của design.

Giới hạn về tài nguyên

LUT utilization khoảng 86% cho thấy việc mở rộng trực tiếp số đơn vị tính toán sẽ gặp áp lực tài nguyên và routing.

Do đó, không thể mặc định rằng tăng gấp đôi compute width sẽ tạo ra speedup tương ứng hoặc vẫn fit trên cùng FPGA.

Giới hạn về phân tích memory

Counter hiện tại chứng minh operand delivery là một bottleneck quan trọng nhưng chưa cho phép phân rã latency chính xác thành:

DDR vs. PS port vs. SmartConnect vs. AXI adapter vs. Memory frontend vs. Operand bank

Vì vậy, cần tránh kết luận một thành phần đơn lẻ là root cause cho toàn bộ memory stall.

7.7 Hướng tối ưu/khảo sát tiếp theo

Từ kết quả profiling, thứ tự tối ưu hợp lý không phải bắt đầu bằng việc mở rộng GEMM array, mà nên ưu tiên đường dữ liệu trước.

7.7.1 Tối ưu memory frontend

Ưu tiên đầu tiên là giảm thời gian compute engine phải chờ operand.

Các hướng gồm:

- tăng khả năng gom nhiều word liên tiếp vào cùng transaction;
- tối ưu address generation;
- giảm request nhỏ;
- giảm các lần truy cập scratch không cần thiết;
- tăng data reuse trong local buffer.

7.7.2 Tăng burst efficiency

Burst length hiện tương đối ngắn. Với những vùng dữ liệu tuần tự, có thể khảo sát burst dài hơn để giảm overhead địa chỉ và response trên mỗi byte dữ liệu.

Việc tăng burst cần được thực hiện theo access pattern thực tế, không phải tất cả tensor đều phù hợp với burst dài.

7.7.3 Tăng request concurrency

Cấu hình hiện tại giới hạn số outstanding transaction.

Việc tăng số lượng read/write request đang chờ có thể giúp che latency DDR nếu memory controller và interconnect hỗ trợ.

Cần đo lại sau mỗi thay đổi để xác định mức concurrency tối ưu, vì tăng outstanding transaction cũng có thể làm tăng buffer và control complexity.

7.7.4 Prefetch

Memory frontend có thể dự đoán operand của tile tiếp theo và bắt đầu request trước khi compute tile hiện tại hoàn thành.

Mục tiêu là chuyển:

LOAD → COMPUTE → LOAD → COMPUTE

thành:

LOAD tile 0

↓

COMPUTE tile 0 + LOAD tile 1

↓

COMPUTE tile 1 + LOAD tile 2

Qua đó giảm khoảng trống giữa các cửa sổ GEMM compute-active.

7.7.5 Ping-pong buffering

Hai buffer có thể được sử dụng luân phiên:

Buffer A: compute

Buffer B: load

sau đó

Buffer A: load

Buffer B: compute

Kỹ thuật này phù hợp với mục tiêu giảm operand_bank_empty nếu đủ BRAM và đường truy cập độc lập.

7.7.6 Giảm scratch DDR round-trip

Một số intermediate tensor có thể được giữ lại on-chip nếu consumer kế tiếp sử dụng chúng ngay.

Điều này có thể tránh pattern:

Compute

↓

Write DDR

↓

Read DDR

↓

Next operator

và thay bằng:

Compute

↓

Local buffer

↓

Next operator

Đây là một hướng có khả năng cải thiện cả latency lẫn AXI traffic.

7.7.7 Pipeline shared FP32 non-GEMM datapath

Để cải thiện timing, shared FP32 arithmetic cần được chia thành nhiều pipeline stage hơn.

Mục tiêu là giảm logic depth và routing pressure trên một chu kỳ clock.

Đồng lại, controller của LayerNorm, GELU và Softmax phải được điều chỉnh để xử lý latency mới.

7.7.8 Khảo sát selective DSP

Một phiên bản thiết kế khác có thể cho phép sử dụng DSP chỉ ở các operator nằm trên critical path.

Việc này cần được thực hiện như một design-space exploration độc lập và so sánh:

- LUT;
- FF;
- DSP;
- WNS;
- Fmax;
- power;
- throughput;
- numerical behavior.

Không nên mặc định selective DSP chắc chắn tốt hơn trước khi có implementation report.

7.7.9 Standalone Cortex-A53 runtime

Một hướng phát triển quan trọng khác là chuyển khỏi XSCT/JTAG runtime.

A53 có thể thực hiện:

- cấu hình NPU qua AXI4-Lite;

- quản lý input/model memory;
- phát START;
- nhận interrupt;
- đọc class/logits.

Khi đó hệ thống mới tiến gần hơn tới embedded deployment thực tế.

7.7.10 Numerical checkpoint analysis

Trước khi thực hiện các thay đổi precision lớn như INT8 hoặc INT4, nên giải quyết numerical mismatch hiện tại.

Việc xác định checkpoint đầu tiên lệch với software reference sẽ giúp phân biệt lỗi do:

- precision;
- rounding;
- approximation;
- algorithm implementation;
- memory/layout;
- hoặc control/data movement.

Sau khi baseline mixed-precision đã được hiểu rõ, các kỹ thuật quantization mới có thể được đánh giá trên nền tảng đáng tin cậy hơn.

CHƯƠNG 8 — KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 Tóm tắt phạm vi báo cáo

- Báo cáo đã xây dựng và kiểm chứng một baseline NPU chuyên dụng cho ViT-Base/Patch16-224 trên Digilent Genesys ZU-5EV. Phạm vi hiện thực bao phủ chuỗi xử lý từ tensor đầu vào đã được tiền xử lý trong DDR tới logits, Argmax và lớp dự đoán cuối. Các bước decode ảnh, resize/normalize và đóng gói model vẫn thuộc phía host; phần PL tập trung vào inference graph đã khóa của ViT-B/16.
- Ở mức kiến trúc, hệ thống được tổ chức theo hướng descriptor-driven. Sequencer phát lịch lệnh cố định, Command Controller và Engine Dispatch chọn đúng một engine cho mỗi descriptor, trong khi Shared Memory Frontend và AXI Memory Adapter chịu trách nhiệm đưa operand từ DDR tới compute engine và ghi kết quả trở lại SCRATCH. Đường control sử dụng AXI4-Lite, còn đường tensor sử dụng AXI4 Memory-Mapped 128 bit qua SmartConnect và cổng HP của Zynq UltraScale+.
- Phần tính toán gồm một GEMM engine mixed FP16/FP32 và sáu engine non-GEMM: Vector, Layout, LayerNorm, Softmax, GELU và Argmax. GEMM sử dụng tám FP16 dot-product stream, tích lũy rộng trước khi chuẩn hóa về FP32; các phép non-GEMM chủ yếu

hoạt động trên dữ liệu FP32. Toàn bộ implementation cuối được ràng buộc DSP=0, vì vậy các datapath số học được ánh xạ lên LUT, FF, carry logic và BRAM.

- Kết quả cuối cho thấy thiết kế đã đi hết luồng RTL → synthesis → implementation → bitstream và chạy trên board. Tại clock mục tiêu 50 MHz, implemented design đạt WNS +0,045 ns và TNS 0 ns; hold timing cũng đạt với WHS +0,010 ns. Resource utilization ghi nhận 101.134 LUT, 63.997 FF, 41 RAMB36 và 0 DSP. Trên subset 1.000 ảnh đầu tiên của ImageNet ILSVRC2012 validation, hệ thống ghi nhận 806 dự đoán đúng và 194 dự đoán sai, tương ứng Top-1 accuracy 80,6%.
- Giá trị quan trọng nhất của phiên bản M8 không nằm ở peak throughput mà ở việc hình thành một baseline RTL-to-board có thể quan sát và đo đạc. Các performance counter cho thấy khi operand đã sẵn sàng, tám GEMM stream được sử dụng gần đầy, nhưng phần lớn thời gian hệ thống lại nằm ở pha cấp dữ liệu. Do đó, báo cáo đã tách rõ hai vấn đề: timing bottleneck ở shared FP32 non-GEMM path và runtime bottleneck ở đường operand/data delivery.

Bảng 8.1. Tóm tắt các kết quả chính của implementation M8.

Hạng mục	Kết quả cuối	Ý nghĩa
Nền tảng	Genesys ZU-5EV, xczu5ev-sfvc784-1-e	Board validation trên Zynq UltraScale+ MPSoC
Mô hình	ViT-Base/Patch16-224, batch 1	Topology và schedule cố định
Compute	GEMM + 6 non-GEMM engine	Bao phủ các operator chính của inference
Precision	Mixed FP16/FP32	FP16 tại GEMM term; output/intermediate chủ yếu FP32
Clock	50 MHz	Timing closure đạt
Resource	101.134 LUT; 63.997 FF; 41 RAMB36; DSP=0	Thiết kế LUT-heavy, giữ toàn bộ DSP chưa sử dụng
Timing	WNS +0,045 ns; TNS 0 ns	PASS nhưng margin setup nhỏ
Dataset snapshot	1.000 ảnh ImageNet val, index 1–1000	806 đúng / 194 sai
Top-1	80,6%	Kết quả trên subset 1.000 ảnh, không phải ImageNet-50k
Runtime prototype	xấp xỉ 7 phút/ảnh trong flow host-driven	Dùng để characterization, chưa phải production runtime

8.2 Các giới hạn cần lưu ý khi đọc kết quả cuối cùng

- Các kết quả ở trên cần được đọc trong đúng phạm vi của prototype. Thứ nhất, M8 là accelerator chuyên dụng cho ViT-Base/Patch16-224 với batch size 1 và fixed schedule. Thiết kế chưa phải model-agnostic accelerator, chưa có compiler tự sinh descriptor và chưa hỗ trợ dynamic shape hoặc nhiều mô hình Transformer khác.
- Thứ hai, kết quả Top-1 80,6% chỉ được đo trên 1.000 ảnh đầu tiên của ImageNet validation. Con số này không được trình bày như official ImageNet-50k accuracy. Ngoài ra, classification pass không đồng nghĩa với full numerical equivalence. Báo cáo hiện có

bằng chứng end-to-end và Top-1, nhưng chưa có bộ checkpoint tensor cuối đủ để tuyên bố toàn bộ graph bit-exact hoặc đạt cùng một tolerance ở mọi layer.

- Thứ ba, mixed precision và các approximant tạo ra những nguồn sai khác số học có chủ đích. Persistent GEMM-B và operand động có thể đi qua boundary FP32→FP16; LayerNorm, Softmax và GELU sử dụng chuỗi operation/approximation theo RTL. Vì vậy software reference cần tái hiện đúng thứ tự operation, rounding và special-value contract trước khi quy một mismatch cho lỗi phần cứng.
- Thứ tư, timing đã đóng ở 50 MHz nhưng WNS chỉ +0,045 ns và Physical CLB ở mức rất cao. Điều này cho thấy design vẫn nhạy với placement/routing và không có nhiều margin để tăng clock hoặc mở rộng thêm logic theo cách trực tiếp. Power 3,30 W trong Chương 6 là ước lượng của Vivado với confidence level Medium, không phải phép đo công suất trực tiếp trên board.
- Thứ năm, runtime hiện tại vẫn host-driven qua XSCT/JTAG. Latency xấp xỉ 7 phút/ảnh bao gồm overhead điều phối, truy cập DDR và serialization của prototype, nên không phù hợp để dùng như một benchmark công bằng với CPU/GPU hoặc các accelerator đã công bố. Cortex-A53 chưa chạy một application inference standalone hoàn chỉnh trong snapshot cuối.
- Cuối cùng, performance counter cho thấy runtime bottleneck chính nằm ở data delivery. Panel load active khoảng 88,48%, feeder stall khoảng 87,89% và operand bank empty khoảng 59,95%, trong khi compute-active chỉ khoảng 10,11%. Ngược lại, khi compute thực sự hoạt động, accepted stream slots đạt khoảng 7,84/8. Vì vậy không nên kết luận rằng tăng số PE/stream ngay lập tức sẽ làm throughput tăng tương ứng.

8.3 Hướng phát triển

- **Hướng phát triển nên ưu tiên** theo thứ tự bottleneck đã đo, thay vì mở rộng compute một cách cơ học.
- **Ưu tiên đầu tiên** là cải thiện memory frontend và AXI data delivery. Các thay đổi cần khảo sát gồm tăng độ sâu prefetch/read-ahead, tăng số transaction outstanding khi tải nguyên cho phép, khai thác burst dài hơn hoặc hiệu quả hơn, mở rộng ping-pong/double buffering cho các operand phù hợp và tăng data reuse để giảm số lần quay lại DDR.
- **Ưu tiên thứ hai** là cải thiện timing margin của shared FP32 arithmetic fabric. Các đường FP32 dùng chung cho Vector, LayerNorm, Softmax và GELU nên được xem xét lại về pipeline boundary, fan-out, mux depth và placement. Mục tiêu trước tiên là tạo margin timing an toàn hơn tại 50 MHz; chỉ sau đó mới đánh giá khả năng nâng tần số. Việc tối ưu phải đi kèm kiểm tra resource/routing vì Physical CLB hiện đã gần giới hạn device.
- **Ưu tiên thứ ba** là hoàn thiện numerical sign-off. Cần bổ sung checkpoint sau projection, QK^T , Softmax, $P \times V$, LayerNorm, GELU và logits; mỗi checkpoint nên ghi max absolute error, relative error có epsilon bảo vệ, số phần tử vượt ngưỡng và special-value statistics. Checkpoint đầu tiên phân kỳ sẽ giúp phân biệt lỗi packing/addressing với sai khác do FP16, rounding hoặc approximant.
- **Ưu tiên thứ tư** là thay thế flow XSCT/JTAG bằng runtime thực thi trên PS. Một bare-metal hoặc Linux application trên Cortex-A53 có thể đảm nhiệm nạp input, cấu hình register/table, phát START, chờ IRQ hoặc poll STATUS, đọc class/logits và quản lý nhiều

ảnh liên tiếp. Khi đó có thể đo latency/throughput theo protocol ổn định hơn và tách cold-start khỏi per-image execution.

- **Sau khi memory delivery, timing và numerical baseline đã ổn định**, mới nên khảo sát mở rộng compute parallelism hoặc precision khác. Các hướng có thể gồm tăng số stream/PE, dùng DSP theo chế độ tùy chọn, INT8/INT4 quantization, model compression hoặc hỗ trợ thêm các biến thể ViT. Những hướng này cần được đánh giá bằng cùng một hệ thống bằng chứng: accuracy, resource, timing, throughput và memory behavior, thay vì chỉ so sánh số phép tính lý thuyết.
- **Về phương pháp đánh giá**, bước tiếp theo nên mở rộng campaign từ subset 1.000 ảnh sang toàn bộ 50.000 ảnh validation khi runtime đã đủ ổn định, đồng thời bổ sung phép đo công suất trực tiếp nếu cần energy/inference. Khi đó M8 có thể chuyển từ một board-validated research baseline sang một nền tảng so sánh kiến trúc có protocol đo đầy đủ và tái lập được.

TÀI LIỆU THAM KHẢO

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021, pp. 1–21.
- [2] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, and H. Chen, “ViA: A Novel Vision-Transformer Accelerator Based on FPGA,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4088–4099, Nov. 2022, doi: 10.1109/TCAD.2022.3197489.
- [3] S. Nag, G. Datta, S. Kundu, N. Chandrachoodan, and P. A. Beerel, “ViTA: A Vision Transformer Inference Accelerator for Edge Applications,” *arXiv preprint arXiv:2302.09108*, Feb. 2023.
- [4] T. Li, F. Zhang, G. Xie, X. Fan, Y. Gao, and M. Sun, “A High Speed Reconfigurable Architecture for Softmax and GELU in Vision Transformer,” *Electronics Letters*, vol. 59, no. 5, 2023, doi: 10.1049/ell2.12751.
- [5] Q. Guo, J. Wan, S. Xu, M. Li, and Y. Wang, “HG-PIPE: Vision Transformer Acceleration with Hybrid-Grained Pipeline,” *arXiv preprint arXiv:2407.17879*, ver. 2, Aug. 2024.

- [6] W. Ye, H. Li, X. Zhou, D. Pan, and K. Li, “MDS-ViT: A Multi-Data Stream FPGA-Based Vision Transformer Accelerator,” *Journal of Systems Architecture*, vol. 178, Art. no. 103898, 2026, doi: 10.1016/j.sysarc.2026.103898.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [8] H. T. Kung, “Why Systolic Architectures?” *Computer*, vol. 15, pp. 37–46, 1982.
- [9] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.
- [10] AMD/Xilinx, *Zynq UltraScale+ MPSoC Data Sheet*, DS891, 2023.
- [11] Arm Ltd., *AMBA AXI Protocol Specification*, Arm Architecture Specifications.
- [12] AMD/Xilinx, *Vivado Design Suite User Guide*, Vivado Design Suite, 2023.
- [13] Digilent Inc., *Genesys ZU: Zynq UltraScale+ MPSoC Development Board Reference Manual*.

PHỤ LỤC (bổ sung sau)

Phụ lục A — Ký hiệu, tensor shape và quy ước dữ liệu

Phụ lục B — Command descriptor, template lệnh và lịch lệnh

Phụ lục C — Bản đồ bộ nhớ, thanh ghi điều khiển và quy tắc ownership

Phụ lục D — Pseudocode/trace chi tiết các thuật toán GEMM

Phụ lục E — Pseudocode/trace chi tiết các thuật toán non-GEMM

Phụ lục F — Trace minh họa luồng xử lý từ ảnh thô

Phụ lục G — Trace minh họa luồng xử lý liên quan đến bộ nhớ

Phụ lục H — Bảng kết quả đầy đủ của tập ảnh cuối cùng

Phụ lục I — Danh mục hình, waveform, log, receipt và source mapping

Phụ lục J — Ma trận tổng hợp bài báo liên quan

